基于Harness架构的AI Coding项目

讲师：肖斌

强烈建议：用基于DeepSeek的Claude 桌面来跟着做项目

https://berryreload.blog.csdn.net/article/details/161533177

三个基于Harness架构的Agent开发框架：
DeepAgents

Claude agent sdk

Openai agent sdk

agentscope： 阿里

第一章、项目总览介绍

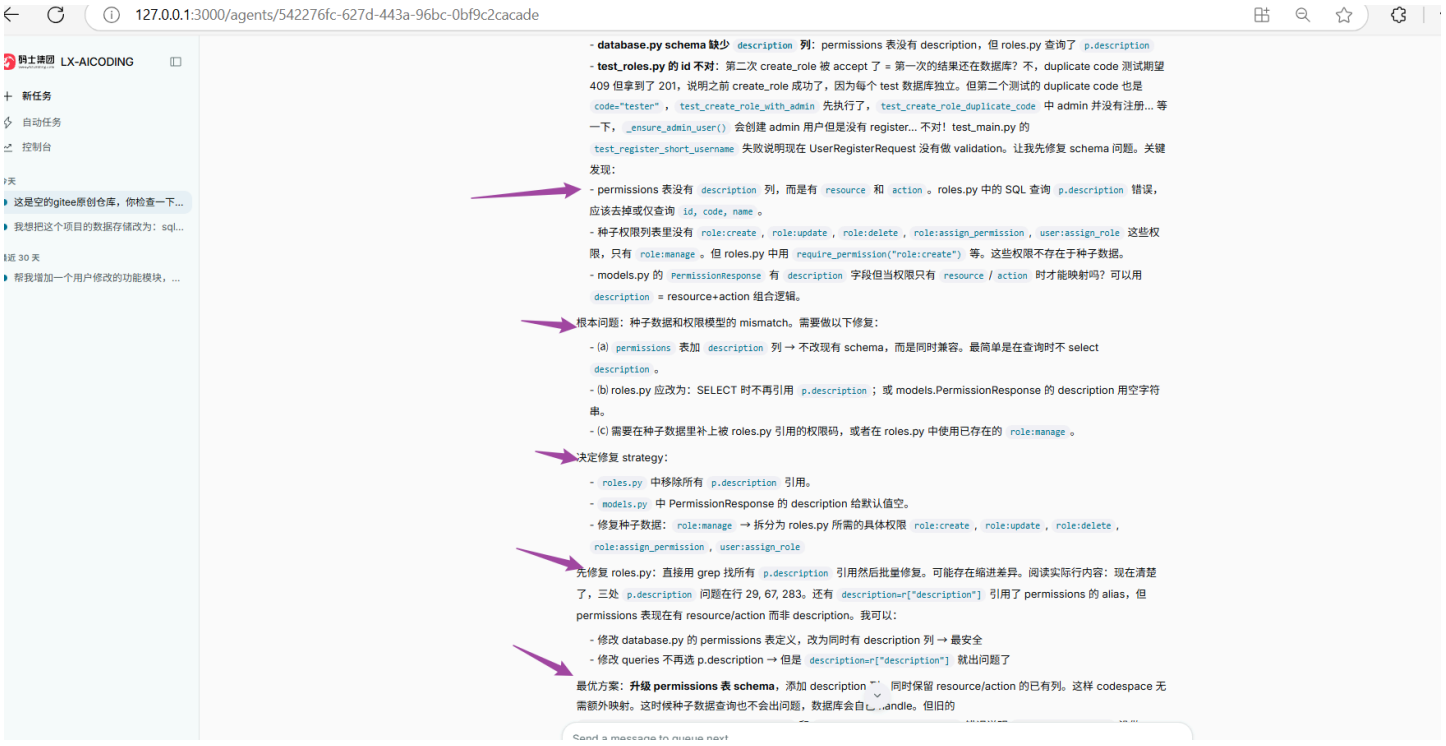
1、课程目标

本项目不是讲“如何调用一次大模型”，而是讲一个真实 Agent Coding 项目的工程化搭建顺序。

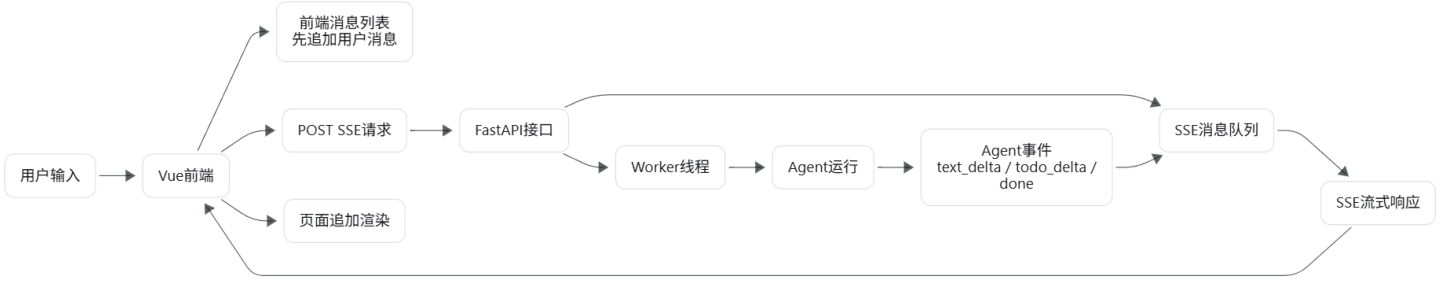
完成课程后，我们需要能掌握：

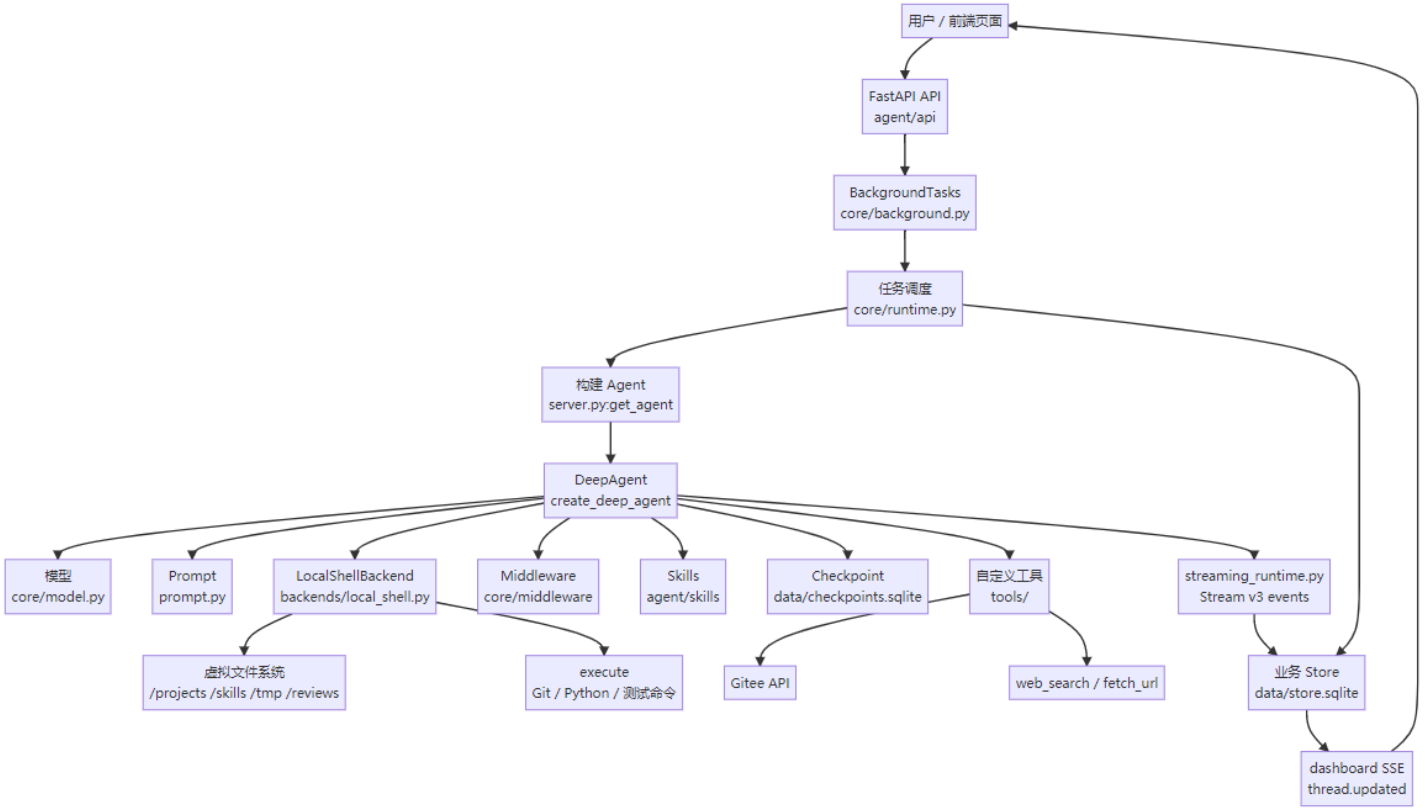
能力	学完后应达到的理解
Agent Loop	模型如何在“思考、调用工具、读取反馈、继续行动”之间循环
DeepAgents	create_deep_agent 如何把模型、工具、文件系统、子 Agent、skills 和 middlewar 合成 harness
文件后端	为什么 Agent 需要一个受控的 backend，而不能直接读写全盘
长期记忆	记忆、skills、prompt、checkpoint，store 分别解决什么问题
中间件	如何在工具调用前清洗参数、在工具失败后恢复运行
运行调度	为什么要先生成方案、等待确认，再进入编码实现
流式展示	如何把 DeepAgents raw event 转成页面可见步骤
持久化	为什么业务 Store 和 LangGraph checkpoint 要分开


127.0.0.1:3000/agents/465c1882-a30d-456f-95a0-9835dce20090
LX-AICODING
新任务
自动任务
控制台
今天
请归纳一下当前项目一共有多少个AP...
帮我增加一个用户修改的功能模块，...
17 POST /upload 正常 非 .xlsx→拒绝；解析 Excel 并存入 uploads + upload_rows 表
四、技术架构总结
- 框架：FastAPI（异步支持）
- 数据库：SQLite（app.db），含 5 张表：departments、users、query_history、uploads、upload_rows
- 模板引擎：Jinja2（14 个 HTML 模板）
- 验证码：captcha 库生成 PNG，存储在内存 SESSION_STORE（基于 x-session-id 头）
- 密码存储：⚠ 明文存储（未做哈希），仅适合教学/演示场景
- 依赖：fastapi、uvicorn、captcha、openpyxl、jinja2、python-multipart
结论：该项目一共 17 个 API 接口（GET 11 + POST 6），全部接口逻辑正确，项目自带测试覆盖完整。
任务状态：completed 仓库：https://gitee.com/msb-goldbin/ai_coding.git
任务计划
5 out of 5 tasks completed
1. 定位并确认仓库对应本地目录
2. 扫描项目结构，找出所有 API 接口定义
3. 归纳总结 API 接口总数和详情
4. 对每个 API 接口进行测试
5. 汇总测试结果并回答用户
有详细的任务清单跟踪能力
任务已创建
解析 Gitee 仓库
继续输入指令


127.0.0.1:3000/agents/542276fc-627d-443a-96bc-0bf9c2cacade
LX-AICODING
新任务
自动任务
控制台
这是空的gitee原创仓库，你检查一下...
我想把这个项目的数据存储改为：sql...
帮我增加一个用户修改的功能模块，...
- database.py schema 缺少 description 列：permissions 表没有 description，但 roles.py 查询了 p.description
- test_roles.py 的 id 不对：第二次 create_role 被 accept 了 = 第一次的结果还在数据库？不，duplicate code 测试期望 409 但拿到了 201，说明之前 create_role 成功了，因为每个 test 数据库独立。但第二个测试的 duplicate code 也是 code="tester"，test_create_role_with_admin 先执行了，test_create_role_duplicate_code 中 admin 并没有注册... 等一下，_ensure_admin_user() 会创建 admin 用户但是没有 register... 不对！test_main.py 的 test_register_short_username 失败说明现在 UserRegisterRequest 没有做 validation。让我先修复 schema 问题。关键发现：
- permissions 表没有 description 列，而是有 resource 和 action。roles.py 中的 SQL 查询 p.description 错误，应该去掉或仅查询 id, code, name。
- 种子权限列表里没有 role:create, role:update, role:delete, role:assign_permission, user:assign_role 这些权限，只有 role:manage。但 roles.py 中用 require_permission("role:create") 等。这些权限不存在于种子数据。
- models.py 的 PermissionResponse 有 description 字段但当权限只有 resource / action 时才能映射吗？可以用 description = resource+action 组合逻辑。
根本问题：种子数据和权限模型的 mismatch。需要做以下修复：
- (a) permissions 表加 description 列 → 不改现有 schema，而是同时兼容。最简单是在查询时不 select description。
- (b) roles.py 应改为：SELECT 时不再引用 p.description；或 models.PermissionResponse 的 description 用空字符串。
- (c) 需要在种子数据里补上被 roles.py 引用的权限码，或者在 roles.py 中使用已存在的 role:manage。
决定修复 strategy：
- roles.py 中移除所有 p.description 引用。
- models.py 中 PermissionResponse 的 description 给默认值空。
- 修复种子数据：role:manage → 拆分为 roles.py 所需的具体权限 role:create, role:update, role:delete, role:assign_permission, user:assign_role
先修复 roles.py：直接用 grep 找所有 p.description 引用然后批量修复。可能存在缩进差异。阅读实际行内容：现在清楚了，三处 p.description 问题在行 29, 67, 283。还有 description=r["description"] 引用了 permissions 的 alias，但 permissions 表现在有 resource/action 而非 description。我可以：
- 修改 database.py 的 permissions 表定义，改为同时有 description 列 → 最安全
- 修改 queries 不再选 p.description → 但是 description=r["description"] 就出问题了
最优方案：升级 permissions 表 schema，添加 description 同时保留 resource/action 的已有列。这样 codespace 无需额外映射。这时候种子数据查询也不会出问题，数据库会自己 handle。但旧的
Send a message to queue next

2、总体架构图


前端消息列表
先追加用户消息
POST SSE请求
FastAPI接口
SSE消息队列
用户输入
Vue前端
Worker线程
Agent运行
Agent事件
text_delta / todo_delta /
done
SSE流式响应
页面追加渲染



代码块

```
E:\my_project\LX_AICoding
├─ agent/                                  Agent 后端核心，负责 FastAPI、DeepAgents、工具、存储、运行调度
│  ├─ app.py                               FastAPI 服务入口：加载环境、配置日志、注册 API 路由和 CORS
│  ├─ server.py                            当前新版 DeepAgent 工厂：绑定 backend、permissions、middleware、skills、subagent、checkpoint
│  ├─ prompt.py                            系统提示词：中文输出、Gitee 限制、虚拟路径规则、任务类型规则
│  ├─ env_utils.py                         环境变量加载：复用 open-swe .env，再用本项目 .env 覆盖
│  ├─ reviewer.py                          Reviewer 查询入口：读取 findings 并格式化审查评论
│  ├─ reviewer_diff.py                     获取本地 Git diff，供 reviewer 分析
│  ├─ reviewer_findings.py                 ReviewFinding 数据结构和评论格式化逻辑
│  ├─ reviewer_publish.py                  将 reviewer findings 发布到 Gitee PR 评论
│  ├─ __init__.py                          agent Python 包标识
│  │
│  ├─ api/                                 FastAPI API 层
│  │  ├─ __init__.py                       导出普通 API router 和 dashboard router
│  │  ├─ routes.py                         普通 REST API：health、任务创建、任务查询、日志读取
```

```
16  │   ├── dashboard_routes.py                    open-swe 前端适配 API：threads、
    messages、approve、SSE、payload 转换
17  │   └── schemas.py                             Pydantic 请求模型，目前主要是
    TaskCreateRequest
18  │
19  ├── core/                                      Agent 运行编排核心
20  │   ├── __init__.py                            core Python 包标识
21  │   ├── runtime.py                             任务总调度：分类、方案生成、确认实
    施、pull-only、workspace listing
22  │   ├── streaming_runtime.py                   消费 DeepAgents v3 事件流，解析正
    文、todo、subagent、运行限制
23  │   ├── graph.py                               兼容旧入口；提供 store、
    checkpointer，build_agent 转调 server.get_agent
24  │   ├── model.py                               构建 deepseek-v4-pro ChatOpenAI
    模型
25  │   ├── settings.py                            路径配置：data、logs、workspace、
    projects、skills
26  │   ├── task_intent.py                         本地关键词任务分类：coding、
    planning、analysis、qa、sync、inspect
27  │   ├── events.py                              写 run_events 的轻量封装，供
    runtime/tools/middleware 使用
28  │   ├── background.py                          FastAPI 后台任务入口，调用
    runtime 并记录失败
29  │   ├── logging_config.py                      日志配置和日志读取：backend 日志、
    agent-runs 日志
30  │   ├── memory.py                              加载长期记忆 workspace.md
31  │   ├── persistence.py                         创建 LangGraph SQLite
    checkpointer
32  │   ├── repo_mapping.py                        Gitee 仓库 URL 与本地 projects 目
    录映射、发现、保存、验证
33  │   ├── state.py                               Agent task state 类型定义
34  │   └── middleware/                            DeepAgents/LangChain Agent
    middleware
35  │       ├── __init__.py                        middleware 导出入口
36  │       ├── tool_sanitize.py                   工具入参清洗：路径、Gitee URL、
    offset/limit、.secrets、..
37  │       ├── tool_error.py                      工具异常兜底：异常转 ToolMessage，
    让 Agent 可恢复
38  │       └── run_limits.py                      运行保护：工具调用次数和总运行时间限
    制
39  │
40  ├── backends/                                  本地 Windows sandbox/backend
41  │   ├── __init__.py                            backends Python 包标识
42  │   ├── local_shell.py                         DeepAgents 本地 backend：
    ls/read/write/edit/glob/grep/execute、Git askpass、虚拟目录
43  │   ├── workspace.py                           工作区路径解析，保证路径不越出
    WORKSPACE_ROOT
```

```
44  │   │   └── permissions.py                          路径越界异常、命令白名单、Git 分支和 commit message 安全校验
45  │   │
46  │   ├── tools/                                      自定义工具层；文件/命令工具主要由 DeepAgents backend 原生提供
47  │   │   ├── __init__.py                            导出当前自定义工具
48  │   │   ├── gitee_api.py                           Gitee URL 解析、token 脱敏、创建 PR、发布 PR 评论
49  │   │   ├── gitee_tools.py                         Agent 可调用 Gitee 工具：open_gitee_pull_request、publish_gitee_pr_comment
50  │   │   ├── web_search.py                          智谱 web_search 工具，按 thread 写运行事件
51  │   │   ├── fetch_url_tools.py                     fetch_url 工具：读取公开 URL，HTML 转 Markdown/文本
52  │   │   ├── safe_http.py                           安全 HTTP 层：防 SSRF、内网访问、DNS rebinding、重定向校验
53  │   │   ├── runtime_context.py                     工具运行时读取 thread_id、task_kind、只读状态
54  │   │   └── reviewer_tools.py                      Reviewer 工具：添加 finding、列出 findings
55  │   │
56  │   ├── store/                                      业务 SQLite Store
57  │   │   ├── __init__.py                            导出 LocalSqliteStore
58  │   │   └── sqlite_store.py                        管理 threads、runs、run_events、thread_messages、review_findings、repo mappings
59  │   │
60  │   ├── skills/                                     DeepAgents skills
61  │   │   ├── repo-bootstrap-analysis/
62  │   │   │   └── SKILL.md                           首次分析 Gitee 仓库的方法论 skill
63  │   │   └── code-review/
64  │   │       └── SKILL.md                           Reviewer Agent 最小审查规范
65  │   │
66  │   └── memory/                                     长期记忆
67  │       └── workspace.md                           工作区目录语义、项目边界和安全说明
68  │
69  ├── scripts/                                        启动、停止、验证、调试脚本
70  │   ├── start_all.py                                同时启动 FastAPI 后端和 Vite 前端
71  │   ├── stop_all.py                                 停止后端/前端端口进程
72  │   ├── start_backend.cmd                           单独启动后端
73  │   ├── start_ui.cmd                                单独启动前端
74  │   ├── inspect_deepagents_stream.py                检查 DeepAgents 事件流形态
75  │   └── verify_*.py                                 各能力验证脚本：backend、streaming、middleware、Gitee、plan workflow 等
76  │
77  ├── data/                                           SQLite、WAL/SHM、历史方案文件
78  │   ├── store.sqlite*                               业务 Store：任务、运行、消息、事件、PR、repo mapping
```

```
79  │   ├── checkpoints.sqlite*                LangGraph checkpoint: Agent
    thread state
80  │   └── plans/                              历史技术方案 Markdown 归档
81  │
82  ├── logs/                                   后端和 Agent 运行日志
83  │   ├── backend-YYYY-MM-DD.log              FastAPI、接口、配置、异常日志
84  │   └── agent-runs-YYYY-MM-DD.log           Agent 运行、工具、Git、Gitee 相关
    日志
85  │
86  ├── ui/                                     React + Vite 前端，
87  │
88  ├── pyproject.toml                          Python 项目依赖：deepagents、
    fastapi、langchain-openai、requests 等
89  ├── README.md                               项目启动、接口、日志、验证说明
90  ├── .env                                    本项目环境变量覆盖配置
91  └── .gitignore                              Git 忽略规则
```

3、前置知识和项目解决的问题

大模型应用工程师需要先理解三个概念：

概念	简单解释
Agent	一个模型在循环中调用工具，直到任务完成
Harness	包在模型外面的工程外壳：prompt、tools、middleware、memory、filesystem、streaming 等
Runtime	真正驱动 Agent 执行、保存状态、输出事件的运行层

LangChain 官方把 Agent 描述为“模型在循环中调用工具直到任务完成”。DeepAgents 则在普通 Agent Loop 之上内置了文件系统、task planning、subagent、memory、skills 等能力。**（可参考之前的Harness课程项目）**

当前项目解决什么问题

LX-AICODING 是一个基于服务器部署的Gitee Coding Agent 教学项目。

它要解决的问题是：

问题	项目中的解决方式
Agent 如何安全操作代码仓库	LocalShellBackend 限定 E:\ai_workspace 虚拟工作区
Agent 如何理解任务类型	task_intent.py 本地关键词分类
Agent 如何先方案后编码	runtime.py 规划阶段和确认阶段
Agent 如何读写文件和执行命令	DeepAgents 原生 filesystem tools + 自定义 backend
Agent 如何创建 Gitee PR	gitee_tools.py 和 gitee_api.py
Agent 如何处理工具错误	ToolErrorMiddleware 转成可恢复 ToolMessage
Agent 如何防止危险路径	SanitizeToolInputsMiddleware 清洗工具入参
Agent 如何展示实时过程	streaming_runtime.py 写入 run_events，Dashboard SSE 轮询
Agent 如何保存运行状态	store.sqlite 保存业务状态，checkpoints.sqlite 保存 LangGraph 状

第二章、创建项目和全局配置

1、项目的依赖

代码块

```
Python==3.14
dependencies = [
  "deepagents>=0.6.11",
  "fastapi>=0.138.0",
  "uvicorn>=0.49.0",
  "langchain-openai>=1.3.2",
  "langgraph-checkpoint-sqlite>=3.1.0",
  "python-dotenv>=1.2.2",
  "requests>=2.32.0",
  "zhipuai>=2.1.5",
]
git命令
```

2、环境变量配置

项目的跟目录下

代码块

```
# 如果本项目 .env 中也配置了同名变量，本项目 .env 优先。
DEEPSEEK_API_KEY="sk-xxxxxxxx"
DEEPSEEK_BASE_URL="https://api.deepseek.com"
MAIN_MODEL=deepseek-v4-pro
```

```
GITEE_TOKEN="xxxxxx"
GITEE_API_BASE_URL="https://gitee.com/api/v5"
GITEE_WEB_BASE_URL="https://gitee.com"

# Agent 只允许操作这个工作区下的仓库文件。
AI_WORKSPACE_ROOT=E:\ai_workspace

# SQLite 文件固定放在课程项目 data 目录，便于课堂演示和排查。
LX_AICODING_DATA_DIR=E:\my_project\LX_AICoding\data
CHECKPOINT_DB_PATH=E:\my_project\LX_AICoding\data\checkpoints.sqlite
STORE_DB_PATH=E:\my_project\LX_AICoding\data\store.sqlite

# 日志文件固定放在课程项目 logs 目录。
LX_AICODING_LOG_DIR=E:\my_project\LX_AICoding\logs

ZHIPU_API_KEY="xxxxx.xxxxx"
ZHIPU_BASE_URL="https://open.bigmodel.cn/api/paas/v4/"

DEFAULT_REPO_PROVIDER="gitee"
LANGCHAIN_TRACING_V2=
LANGSMITH_TRACING=false
DASHBOARD_BASE_URL=http://127.0.0.1:3000
DASHBOARD_API_BASE_URL=http://127.0.0.1:2024
DASHBOARD_ALLOWED_ORIGINS=http://127.0.0.1:3000,http://localhost:3000
DASHBOARD_DEV_LOGIN=admin
DASHBOARD_DEV_EMAIL=admin@example.com
CONFIGURED_ADMINS=admin@example.com
DASHBOARD_JWT_SECRET=1b3f0add152896994274f429cd18fe8a57a4f149712ed24f185895fe26
a6d63e
SANDBOX_TYPE=local_shell
LOCAL_SHELL_WORKSPACE=E:\ai_workspace
LOCAL_SHELL_PROJECTS_DIR=projects
LOCAL_SHELL_SHARED_PYTHON_VENV=runtimes\python\default\.venv
LOCAL_SHELL_ENABLE_COMMAND_GUARD=true
LOCAL_SHELL_ALLOW_NETWORK=true
# Logging rotation. Current logs are backend.log / agent-runs.log; rotated
logs use .YYYY-MM-DD suffix.
LX_AICODING_LOG_LEVEL=INFO
LX_AICODING_LOG_RETENTION_DAYS=14
LX_AICODING_LOG_WHEN=midnight
LX_AICODING_LOG_INTERVAL=1

# Agent 运行保护阈值。复杂 coding 任务需要给代码修改、测试和 Git/PR 收尾预留足够工具调
用额度。
AGENT_MAX_TOOL_CALLS=240
AGENT_MAX_SECONDS=1800
AGENT_CODING_MAX_TOOL_CALLS=300
```

```
AGENT_CODING_MAX_SECONDS=1800
```

3、项目全局配置和入口

说明	文件	作用
Web 服务入口	agent/app.py	创建 FastAPI app
环境加载	agent/env_utils.py	加载 DeepSeek、Gitee、搜索等密钥
路径配置	agent/core/settings.py	配置 data、logs、workspace
日志配置	agent/core/logging_config.py	按日期写日志

当前项目关键配置

配置	默认值	作用
PROJECT_ROOT	E:\my_project\LX_AICoding	当前AICoding项目源码目录
DATA_DIR	data/	SQLite 数据目录
STORE_DB_PATH	data/store.sqlite	业务 Store
CHECKPOINT_DB_PATH	data/checkpoints.sqlite	LangGraph checkpoint
LOG_DIR	logs/	日志目录
WORKSPACE_ROOT	E:\ai_workspace	Agent 真正操作用户仓库的工作区
PROJECTS_DIR	E:\ai_workspace\projects	Gitee 仓库目录
SKILLS_DIR	E:\ai_workspace\skills	DeepAgents skills 目录
LX_AICODING_LOG_LEVEL	INFO	日志级别
LX_AICODING_LOG_RETENTION_DAYS	14	日志保留天数
LX_AICODING_LOG_WHEN	midnight	日志轮转周期
LX_AICODING_LOG_INTERVAL	1	日志轮转间隔，配合 LX_AICODING_LOG_WHEN 使用

其中日志相关配置都在 `agent/core/settings.py` 中集中定义：

日志文件分为两类：

日志文件	内容范围	用途
logs/backend.log	全量后端日志	排查 FastAPI、任务创建、API 请求、Store、工具异常题
logs/agent-runs.log	只记录 agent.run.* logger	专门观察 Agent 执行链路、工具调用、中间件、流式

当前项目里典型分类：

logger name	所在文件	写入文件
agent.api	agent/api/routes.py	backend.log
agent.memory	agent/core/memory.py	backend.log
agent.run.runtime	agent/core/runtime.py	backend.log + agent-runs.log
agent.run.streaming	agent/core/streaming_runtime.py	backend.log + agent-runs.log
agent.run.shell	agent/backends/local_shell.py	backend.log + agent-runs.log
agent.run.gitee	agent/tools/gitee_tools.py	backend.log + agent-runs.log
agent.run.middleware.tool_error	tool_error.py	backend.log + agent-runs.log

函数或常量	作用
LOG_DIR	当前项目日志目录，默认是 E:\my_project\LX_AICoding\logs
LOG_LEVEL	日志级别，非法配置会在 logging_config.py 中回退到 INFO
LOG_ROTATION_WHEN	轮转时间点，默认 midnight
LOG_ROTATION_INTERVAL	轮转间隔，默认 1
LOG_RETENTION_DAYS	历史日志保留数量，默认 14
backend_log_path()	返回 backend.log 或历史 backend.log.YYYY-MM-DD
agent_log_path()	返回 agent-runs.log 或历史 agent-runs.log.YYYY-MM-DD

第三章、设计本地工作区和文件后端

1、前置知识点

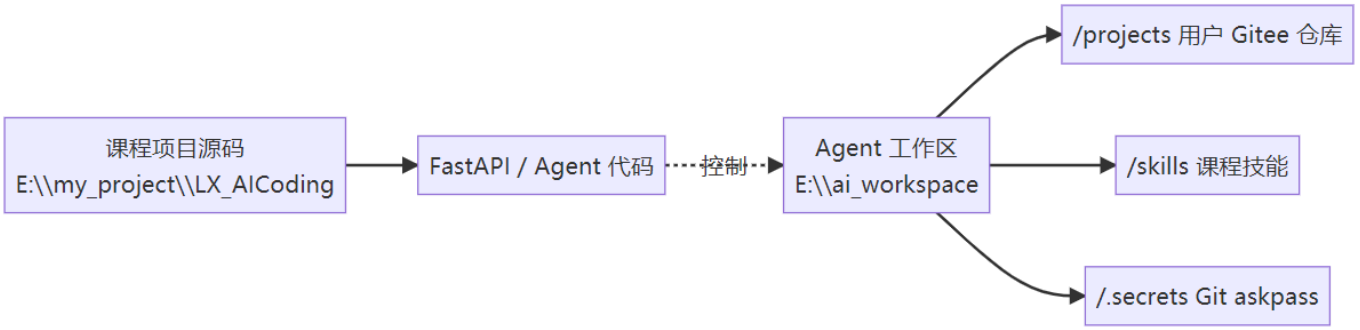
DeepAgents 提供虚拟文件系统能力。Agent 可以通过 `ls`、`read_file`、`write_file`、`edit_file`、`glob`、`grep` 访问文件；如果 backend 支持 shell，还会出现 `execute`。

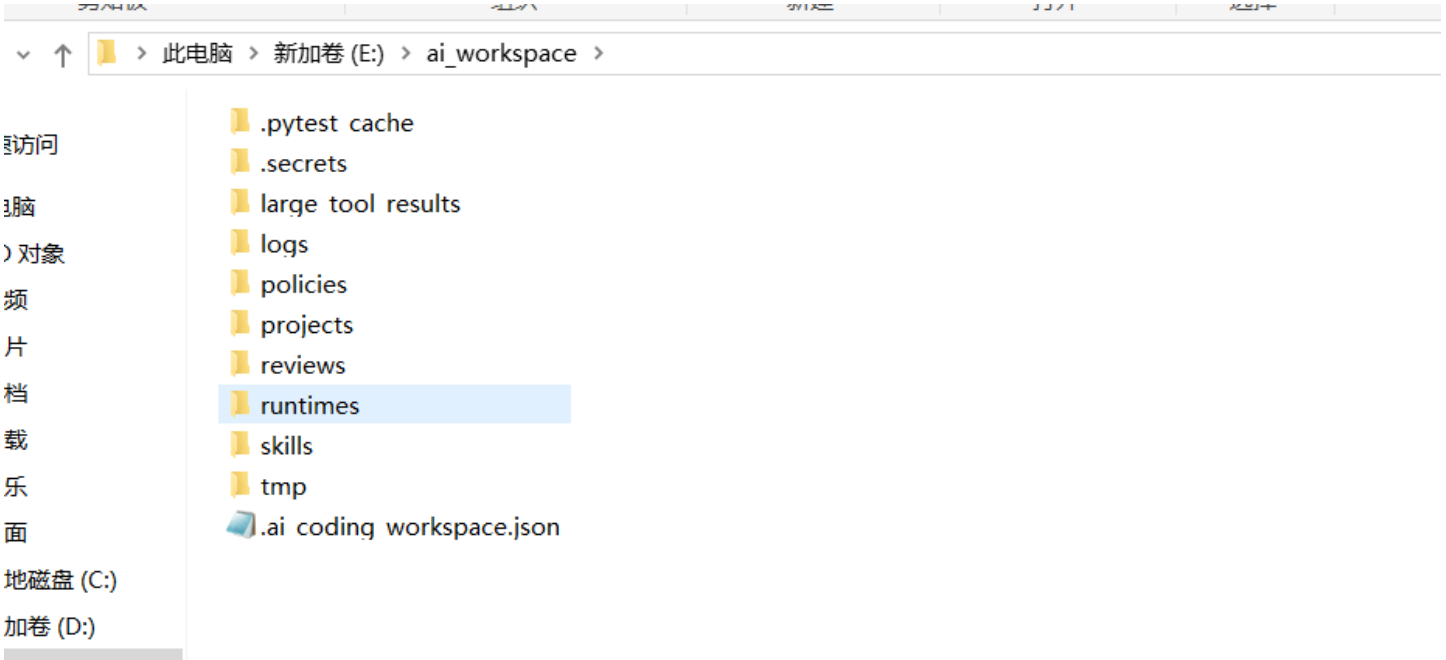
对于Agent工程师，先记住：

概念	作用
Backend	文件和命令真正执行的位置
Virtual filesystem	Agent 看到的是 /projects/a.py 这种虚拟路径
Permission	限制 Agent 可以读写哪些虚拟路径
Sandbox	给 execute 提供受控执行环境

2、为什么项目源码和 Agent 工作区要分开

`E:\my_project\LX_AICoding` 是Agent项目本身；`E:\ai_workspace` 是 Agent 操作用户代码的工作区。两者分开，能降低 Agent 误改课程项目的风险。





这套目录不是随意设计的。它把 Agent 能处理的内容分成几类：

类型	目录	工程意义
业务源码	/projects	Agent 真正开发、测试、提交的地方
能力资料	/skills	给 Agent 看的专业流程，默认只读
规则资料	/policies	工作区、Git、安全规则，默认只读
输出资料	/reviews、/tmp	允许 Agent 写审查结果和临时文件
运行环境	/runtimes	Python/Node 等共享环境，不允许模型随意改
敏感信息	/.secrets	只给 backend 使用，不给模型读取

3、为什么本项目要自定义 LocalShellBackend

DeepAgents 官方的 `LocalShellBackend` 直接操作宿主机，官方文档也提示它只适合受控开发环境。LX-AICODING 面向课堂和本地 Windows，因此自定义了 agent/backends/local_shell.py：

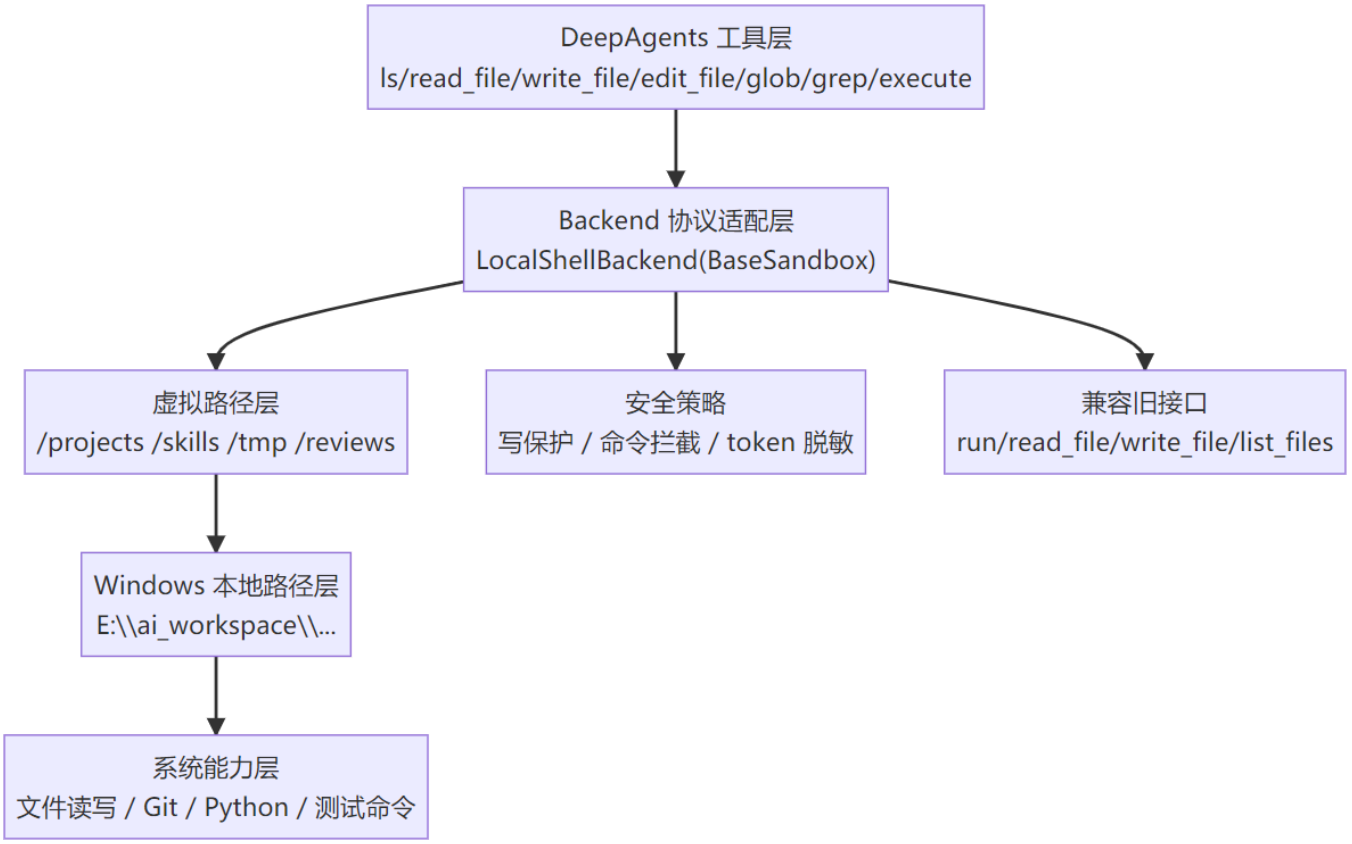
需求	项目实现
Windows 路径兼容	把虚拟路径映射到 E:\ai_workspace
DeepAgents 文件协议	实现 ls/read/write/edit/glob/grep
命令执行	实现 execute 和旧兼容 run
Gitee 认证	自动生成 askpass 文件，不把 token 写入命令
安全目录	/skills、/policies、/runtimes、/logs 拒绝写入
旧代码兼容	保留 read_file/write_file/list_files/run

`LocalShellBackend` 是本项目的工程底座。它不是一个简单的 `subprocess.run()` 包装器，而是把 DeepAgents 文件后端、Windows 本地执行、Gitee 认证、安全边界和旧代码兼容封装到一个统一对象里。

虚拟路径	本地路径	写入策略	说明
/projects	E:\ai_workspace\projects	可写	用户 Gitee 仓库源码
/skills	E:\ai_workspace\skills	只读	DeepAgents skills
/policies	E:\ai_workspace\policies	只读	工作区策略
/reviews	E:\ai_workspace\reviews	可写	审查资料和输出
/runtimes	E:\ai_workspace\runtimes	只读	共享运行环境
/tmp	E:\ai_workspace\tmp	可写	临时文件
/logs	E:\ai_workspace\logs	只读	工作区日志
/.secrets	E:\ai_workspace\.secrets	禁止读写展示	Git askpass 和敏感凭据

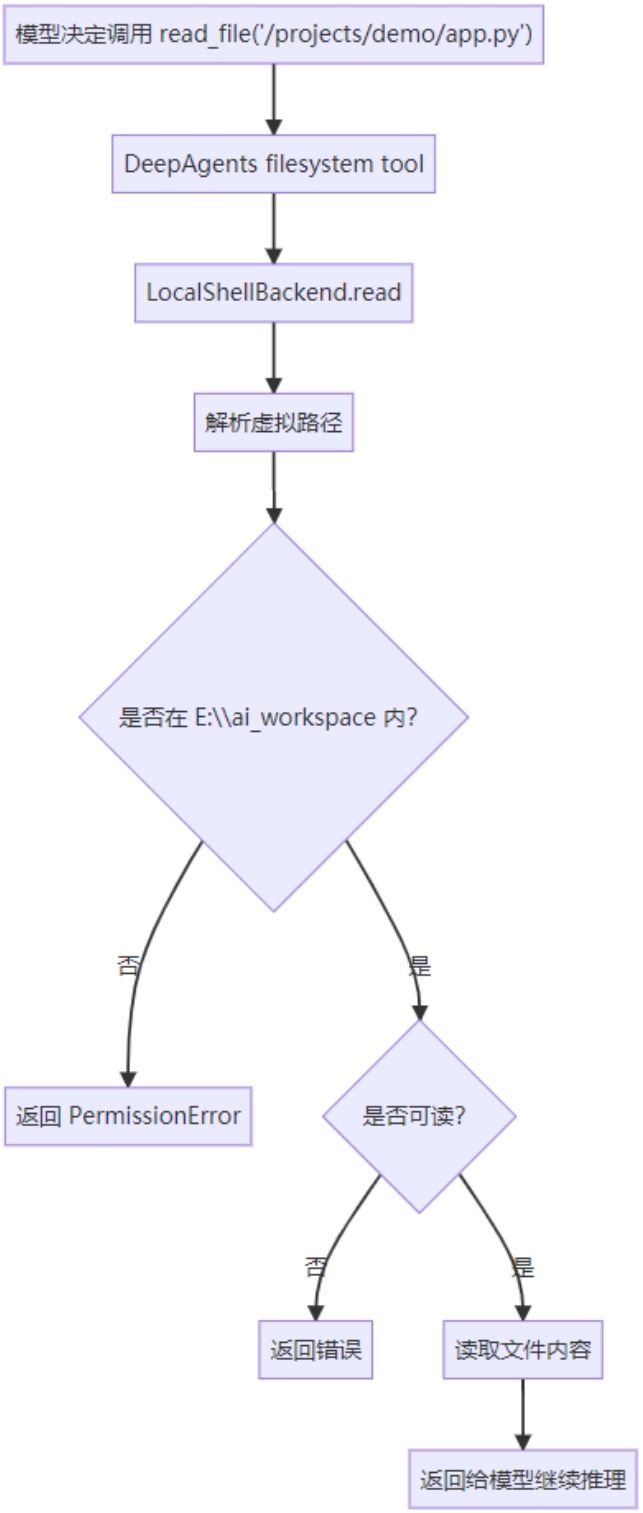
4、自定义LocalShellBackend

`LocalShellBackend` 可以拆成四层理解：



亮点	解决的问题	对应代码位置
DeepAgents 原生 backend 协议	让 ls/read_file/write_file/edit_file/glob/grep/execute 这些原生工具可用	LocalShellBacke
虚拟路径映射	模型只看到 /projects/demo，不需要知道 Windows 真实盘符	_resolve_virtual
Windows 本地兼容	兼容 E:\ai_workspace、PowerShell、Git、Python venv、路径分隔符	__init__()、_no
工作区初始化	自动创建 projects/skills/policies/reviews/runtimes/tmp/logs/.secrets	_ensure_layout(
默认策略文件	自动生成 workspace/git/security 策略说明	_ensure_policy_
Gitee askpass	Git 操作自动认证，不把 token 拼进 clone URL 或命令	_ensure_gitee_a _execution_env
命令安全防护	拦截危险命令、工作区外绝对路径、路径穿越	_deny_reason()-
虚拟路径命令替换	命令里写 /projects/repo 时自动替换成本地真实路径	_virtual_comma
token 脱敏	日志和工具返回中隐藏 Gitee token	_mask_token()
旧接口兼容	保留旧版 runtime/pull-only 分支使用的 run/list_files/read_file/write_file	兼容方法区域

文件后端调用关系



第四章、设计自定义工具

DeepAgents 已经提供文件和命令工具，

工具分类：

能力类型	来源
文件读写、目录查看、命令执行	DeepAgents 原生工具 + LocalShellBackend
联网搜索、网页读取、Gitee PR、审查发现	本项目自定义工具

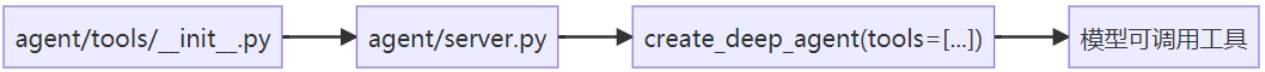
当前工具目录结构

文件	类型	作用
__init__.py	工具导出入口	控制哪些工具暴露给 Agent
gitee_tools.py	Agent tool	创建 PR、发布 PR 评论
gitee_api.py	支撑模块	Gitee URL 解析、token 读取、API 请求、token 脱敏
web_search.py	Agent tool	智谱联网搜索
fetch_url_tools.py	Agent tool	读取网页并转为 Markdown/文本
safe_http.py	支撑模块	SSRF 防护、DNS pin、安全重定向
reviewer_tools.py	Agent tool	写入和读取代码审查发现
runtime_context.py	支撑模块	从 LangGraph config 读取 thread_id、task_kind

暴露给 主Agent 的工具

文件	工具	作用
gitee_tools.py	open_gitee_pull_request	创建或复用 Gitee PR
gitee_tools.py	publish_gitee_pr_comment	发布 Gitee PR 评论
web_search.py	web_search	智谱联网搜索
fetch_url_tools.py	fetch_url	读取公开网页并转文本
reviewer_tools.py	add_review_finding	记录审查问题
reviewer_tools.py	list_review_findings	列出审查问题

这些工具在 `agent/tools/init.py` 中统一导出，并在 `agent/server.py` 的 `create_deep_agent(tools=[...])` 中注册。



1、自定义工具如何知道当前Agent运行环境

自定义工具不能靠全局变量猜当前上下文，而是通过 LangGraph 的 `get_config()` 读取当前运行配置。工具层要能拿到当前运行上下文，否则就无法把工具结果写回正确线程，也无法知道当前任务是否只读。

函数	返回	用途
get_runtime_configurable()	configurable 字典	读取当前工具调用上下文
get_runtime_thread_id()	当前 thread_id 或 None	写 run event、写 SQLite Store
get_runtime_task_kind()	当前 task_kind	判断任务类型
runtime_is_read_only_task()	True/False	工具内部二次权限判断

`2、gitee_api.py`：Gitee API 支撑层

这个文件不是直接暴露给 Agent 的工具，但它是 Gitee 工具的底层实现。

函数	作用
parse_gitee_repo_url(repo_url)	解析 Gitee 仓库地址，得到 owner、repo、clone_url
get_gitee_token()	从 GITEE_TOKEN 或 SCM_GITEE_TOKEN 读取 token
mask_token(text)	日志和错误信息中脱敏 token
_existing_pr_from_error(text)	识别“相同 head/base 的 PR 已存在”错误
create_pull_request(...)	调 Gitee API 创建 PR，或复用已有 PR
post_pr_comment(...)	调 Gitee API 发布 PR 评论

设计亮点：

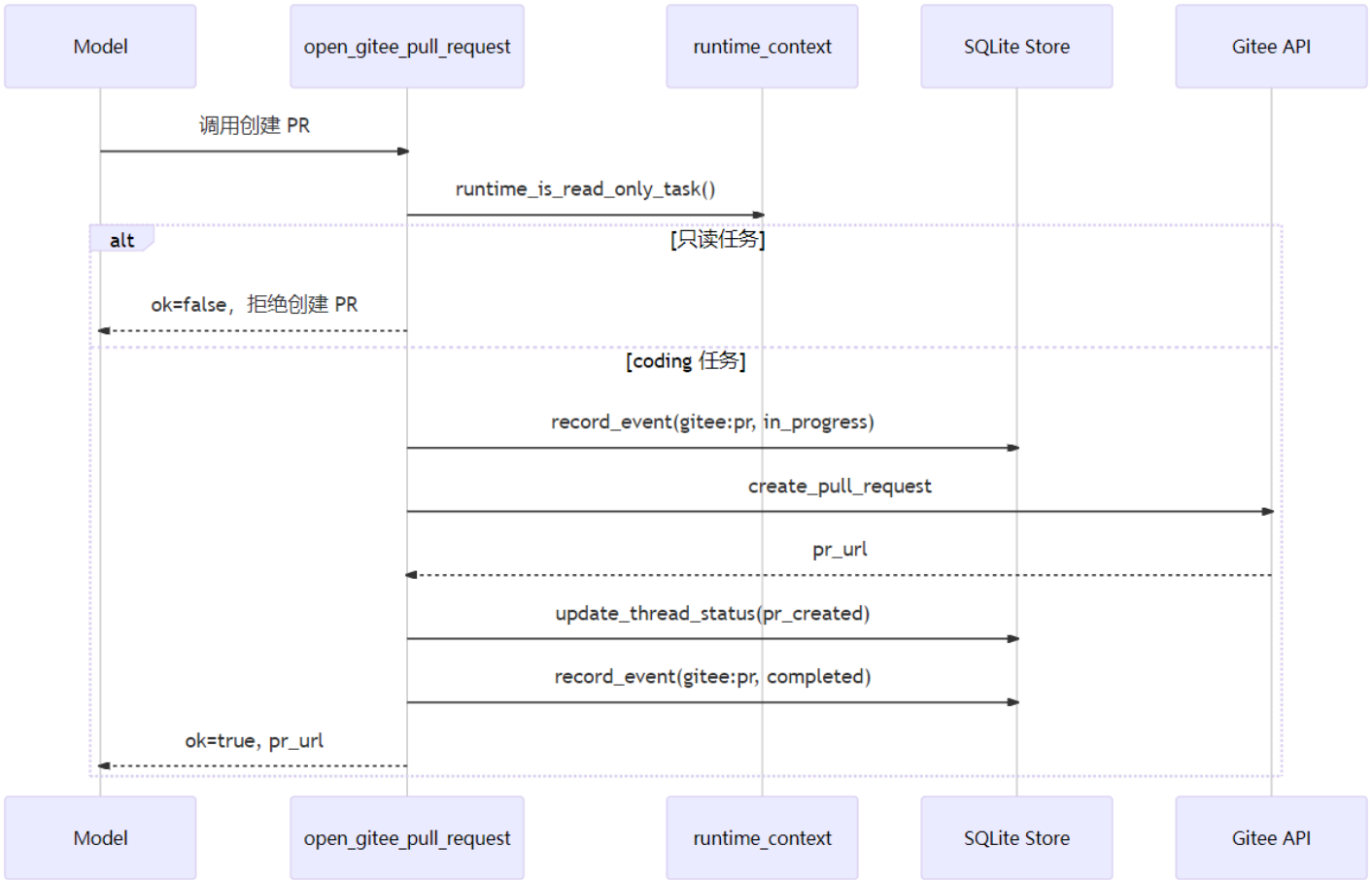
设计	价值
clone URL 不带 token	避免 token 写入命令、日志、.git/config
mask_token 统一脱敏	防止异常、日志、前端事件泄露密钥
重复 PR 复用	同一分支重复演示时不把成功任务误判为失败

`3、gitee_tools.py`：Gitee PR 工具设计

`open_gitee_pull_request(...)` 参数：

参数	说明
owner	Gitee 仓库 owner
repo	Gitee 仓库名
head	源分支
base	目标分支，默认 master
title	PR 标题
body	PR 描述

执行流程：



`publish_gitee_pr_comment(...)` 参数：

参数	说明
owner	Gitee 仓库 owner
repo	Gitee 仓库名
number	PR 编号
body	评论正文

4、`fetch_url_tools.py`：网页读取工具

`fetch_url` 不是简单 `requests.get()`，而是通过 `safe_http.py` 做安全请求。只要 Agent 能访问 URL，就必须考虑 SSRF。尤其是企业内网环境，`fetch_url` 这类工具不能裸用 `requests.get()`。

代码块

```
SSRF 是 Server-Side Request Forgery 的缩写，中文名为服务器端请求伪造.服务器为了防止SSRF攻击：
防御SSRF通常采用多层防护的“纵深防御”策略：
网络层防护：这是第一道防线，通过防火墙、安全组等策略，严格限制服务器的对外访问能力，只允许其访问必要的外部服务。
应用层防护：在代码层面进行严格的校验。
输入验证：对用户输入的URL进行解析和验证。
使用“白名单”：只允许访问预先批准的域名或 IP 地址，拒绝所有不在名单上的请求。
限制协议：只允许 http 和 https 协议，禁止 file://、dict:// 等可能造成危害的协议。
拒绝内网地址：在应用层面拦截指向内网IP（如 127.0.0.1、192.168.x.x 等）的请求。
```

核心函数：

函数	作用
_resolve_and_validate(url)	校验 URL 协议、hostname 和解析后的 IP
_pin_dns(hostname, addr_infos)	当前线程内固定 DNS 解析结果
_pinned_create_connection(...)	让 urllib3 连接使用已校验 IP
blocked_response(url, reason)	生成统一阻断结果
request_with_safe_redirects(...)	每次请求和重定向前都重新做安全校验

工具签名：

fetch_url(url: str, timeout: int = 30) -> dict[str, Any]

适用场景：

场景	示例
用户给了官方文档链接	读取文档内容辅助分析
用户给了错误页面	抽取错误说明
需要查接口说明	把 HTML 转成模型更容易读的文本

返回结构：

字段	说明
ok	是否成功
url	最终 URL
status_code	HTTP 状态码
content_type	响应内容类型
markdown_content	截断后的 Markdown/文本，最多约 20000 字符
content_length	原始转换后文本长度
error	失败原因

HTML 转文本策略：

1. 优先尝试 `markdownify`。
2. 如果依赖不可用，使用标准库 `HTMLParser` 提取正文文本。
3. 跳过 `script`、`style`、`noscript`。
4. 对段落、标题、列表、表格行插入换行。

5、web_search.py：联网搜索工具

`web_search` 用智谱搜索，适合：

- 最新第三方文档。
- 用户提到的外部资料。
- 错误信息背景。

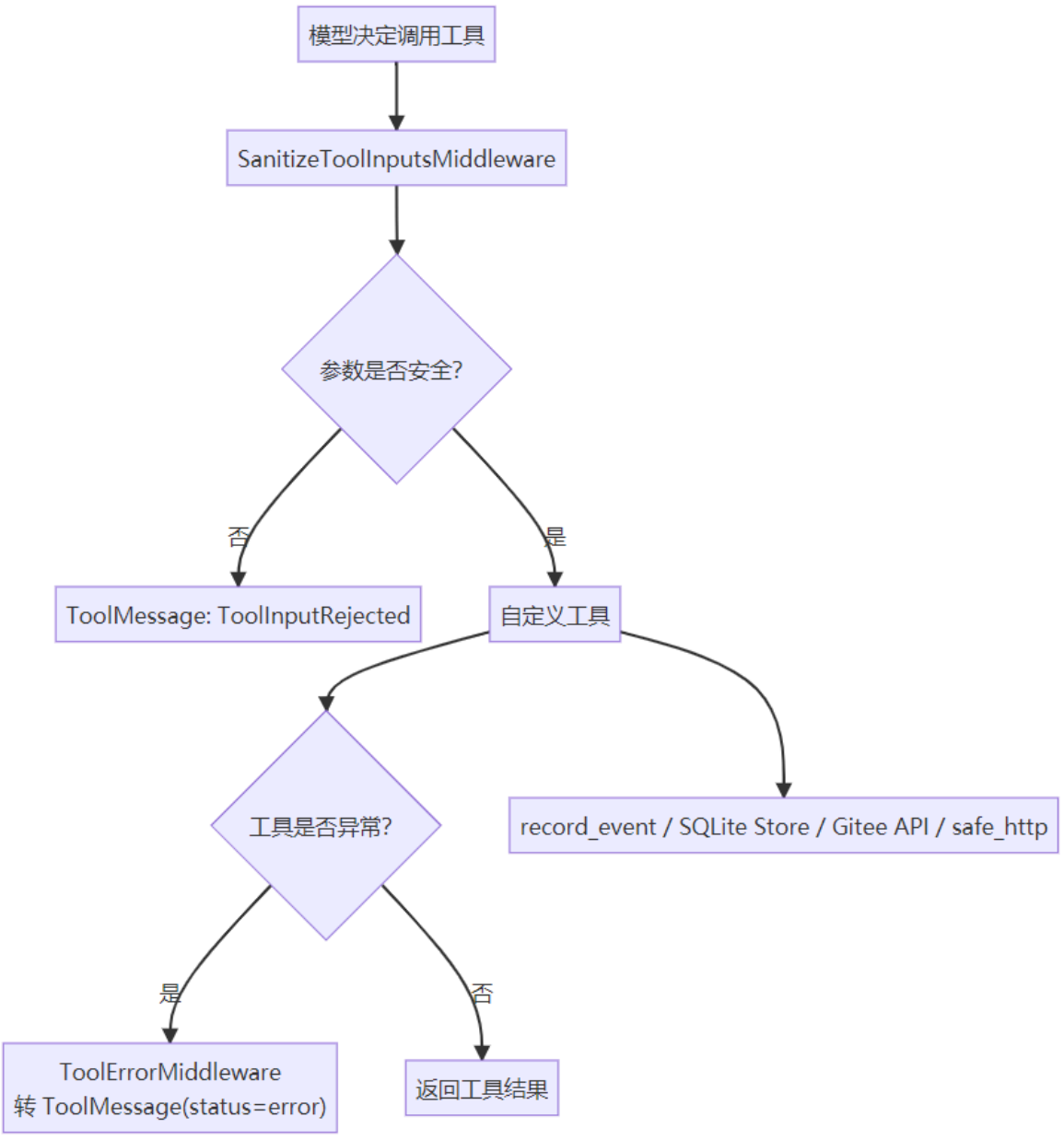
工具签名：

web_search(query: str) -> str

核心实现点：

设计	说明
@lru_cache(maxsize=1)	智谱客户端懒加载并复用
require_env("ZHIPU_API_KEY")	调用时才强制要求密钥
SDK 兼容	优先尝试 zai.ZhipuAiClient，再尝试 zhipuai.ZhipuAI
搜索数量	count=3
搜索引擎	search_pro
事件记录	写入 web_search:{query} 的 in_progress/completed/error
token 脱敏	异常通过 mask_token 处理

6、工具和中间件的配合



第五章、设计中间件

LangChain 官方文档强调 middleware 可以控制 Agent 运行过程中的多个阶段：日志、prompt 变换、工具选择、重试、fallback、rate limit、guardrails 等。

在 LX-AICODING 中，中间件主要解决三个问题：

1. 工具调用前，参数是否安全。
2. 工具执行失败后，是否能恢复。
3. Agent 运行过程中，是否超过工具调用次数或运行时间上限。

当前中间件总览

文件	类	作用
tool_sanitize.py	SanitizeToolInputsMiddleware	工具入参清洗和拒绝
tool_error.py	ToolErrorMiddleware	工具异常转可恢复 ToolMessage
run_limits.py	AgentRunLimitTracker	raw event 消费时检查运行上限
LangChain 内置	ModelCallLimitMiddleware	限制模型调用轮数

注意：`SanitizeToolInputsMiddleware`、`ToolErrorMiddleware` 会直接传入 `create_deep_agent(..., middleware=[...])`。`AgentRunLimitTracker` 不在 DeepAgents middleware 列表里，而是在 `streaming_runtime.py` 消费 raw event 时工作。

代码块

```
middleware=[
    SanitizeToolInputsMiddleware(backend=backend),
    ModelCallLimitMiddleware(run_limit=MODEL_CALL_RECURSION_LIMIT, exit_behavior="end"),
    ToolErrorMiddleware(backend=backend),
]
```

1、`SanitizeToolInputsMiddleware` 中间件详解

这个中间件运行在工具真正执行之前，处理模型最容易生成错误参数的地方。

组成	说明
PATH_ARGUMENTS	统一识别路径类参数：path、cwd、repo_dir、project_dir、file_pat new_path
GITEE_URL_ARGUMENTS	识别 Gitee 仓库地址参数：repo_url
READ_FILE_INT_ARGUMENTS	识别读取文件时的数字参数：offset、limit
sanitize_workspace_path(...)	清洗路径并拒绝危险路径
sanitize_tool_kwargs(...)	按参数名批量清洗工具入参
wrap_tool_call(...)	同步工具调用拦截点
awrap_tool_call(...)	异步工具调用拦截点

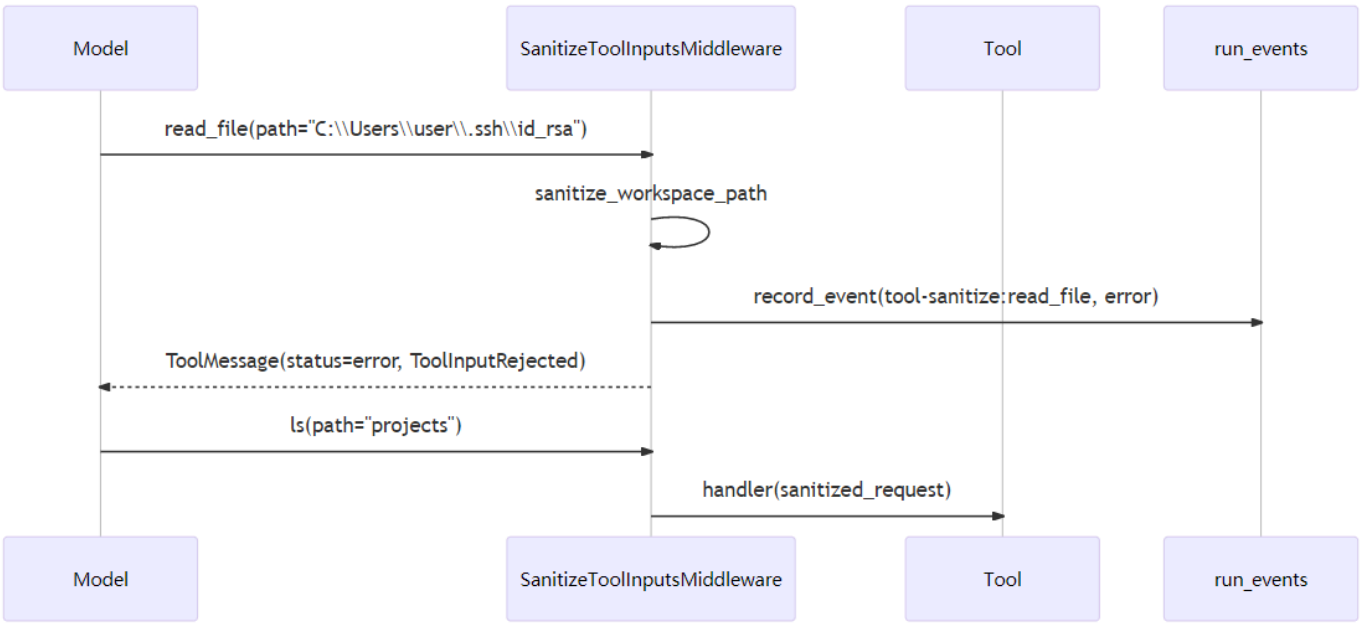
调用前后示例：

代码块

```
模型参数：E:\ai_workspace\projects\demo\README.md
清洗结果：projects/demo/README.md

模型参数：C:\Users\user\.ssh\id_rsa
清洗结果：拒绝，返回 ToolInputRejected
```

参数拒绝不是系统崩溃，而是一次可恢复的工具观察结果。



返回给模型的结构里包含：

字段	作用
ok=false	明确这是失败结果
tool	告诉模型哪个工具被拦截
error_type=ToolInputRejected	表示是参数问题，不是系统异常
workspace	告诉模型当前允许的工作区根目录
hint	引导模型改用相对路径、标准 Gitee 地址

2、ToolErrorMiddleware中间件详解

`ToolErrorMiddleware` 是当前项目的运行状态、前端事件、错误提示、脱敏和恢复策略上的业务实现。捕获工具异常，把异常转成模型可以继续理解的观察结果。没有这层时，工具异常可能直接让任务失败。加入后，Agent 可以看到错误原因，再决定下一步是修复代码、换命令，还是向用户说明阻塞原因。

核心函数：

函数	作用
tool_error_result(...)	把异常转换为中文结构化错误
_error_tool_message(...)	把异常包装成 ToolMessage(status="error")
_record_original_tool_error(...)	把原工具事件从 in_progress 回写成 error
wrap_tool_call(...)	同步工具异常兜底
awrap_tool_call(...)	异步工具异常兜底

`ToolErrorMiddleware` 的核心价值：

没有它	有它
工具抛异常，整轮 Agent failed	异常转 ToolMessage，模型可继续修正
前端步骤卡在 in_progress	回写原事件为 error
用户只看到 500	用户能看到具体工具失败原因

还负责：

- 把异常转换成模型可读的结构化 `ToolMessage`

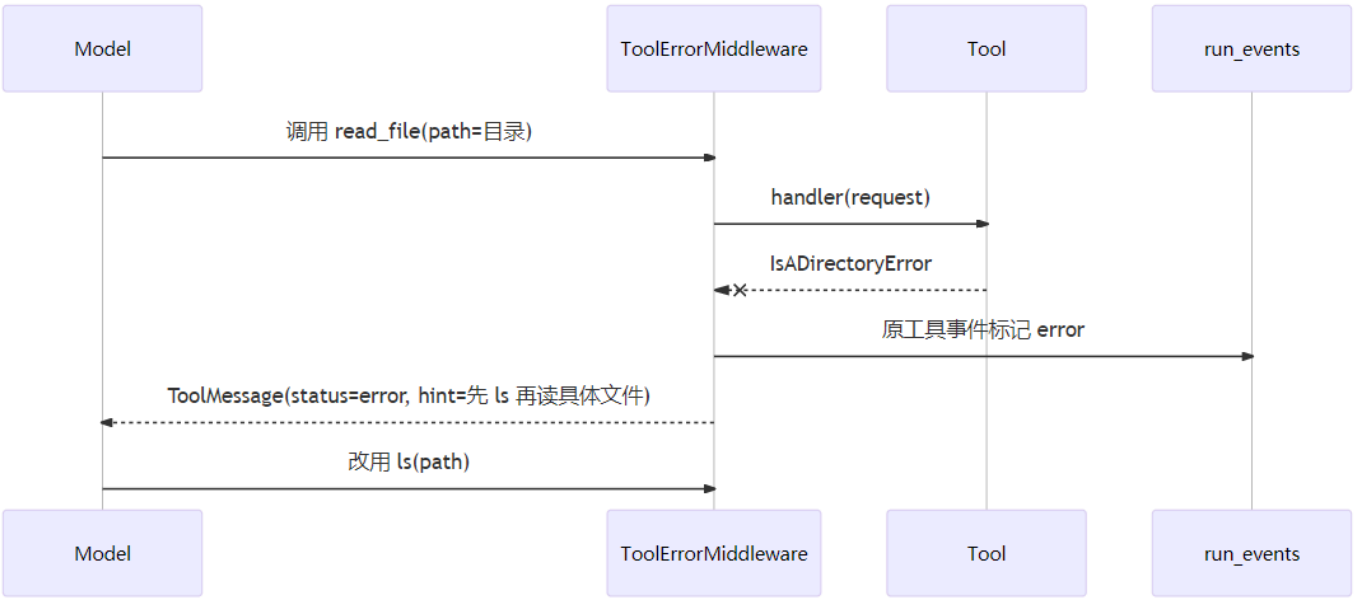
- 给模型明确的中文修正建议
- 对错误文本做 token 脱敏
- 把失败状态写入 `run_events`
- 修正前端步骤状态，避免一直停留在 `in_progress`
- 同时覆盖同步工具和异步工具调用

异常类型与提示语

`tool_error_result(...)` 会根据异常类型生成不同 hint：

异常类型	典型场景	给模型的恢复建议
WorkspacePermissionError	后端拒绝访问工作区外路径	使用 /projects 或 /projects/仓库名
IsADirectoryError	把目录当文件读取	先 ls，再读具体文件
FileNotFoundError	文件不存在	先确认真实路径
PermissionError	文件被占用或系统拒绝	改用工作区内具体文件路径
TimeoutError	命令或操作超时	缩小任务范围
其他异常	工具内部普通失败	根据 error 字段调整参数后重试

工具失败恢复流程



原工具事件为什么要回写

很多工具在执行前会写入一条运行事件，例如：

工具	事件 key
ls / list_files	list:{path}
read_file	read:{path}
write_file	write:{path}
edit_file	write:{file_path}
execute	cmd:{command}
run_command	cmd:{command}:{cwd}
open_gitee_pull_request	gitee:pr

如果工具异常后只新增一条 `tool-error:*` 事件，前端原来的步骤可能一直显示运行中。所以 `_record_original_tool_error(...)` 会用相同 key 把原事件更新成 `error`。

这是 Agent 产品里很实用的细节：**错误恢复不仅要让模型继续，也要让用户界面状态正确结束。**

3、Agent runtime 的运行保护器

AgentRunLimitTracker 的作用是：**限制单轮 Agent 任务的运行规模，防止 Agent 长时间循环或无限调用工具。**

它主要做两类保护：

1. **限制运行时间**
   - 记录本轮 Agent 开始时间。
   - 每收到一个 LangGraph 事件，就检查是否超过最大运行秒数。
   - 超过后抛出 `AgentRunLimitExceeded`，让 runtime 停止任务。
2. **限制工具调用次数**
   - 监听 LangGraph 流式事件。
   - 如果事件表示工具开始调用，就累计 `tool_calls`。
   - 超过最大工具调用次数后抛出 `AgentRunLimitExceeded`。

它还支持按任务类型设置不同阈值，例如：

代码块

```
TASK_KIND_DEFAULT_LIMITS = {
    "qa": (60, 600),
    "inspect": (30, 300),
    "sync": (40, 300),
    "analysis": (120, 900),
    "planning": (120, 900),
    "coding": (300, 1800),
```

```
8   }
```

含义是：

```
代码块
1   "coding": (300, 1800)
```

表示 coding 类型任务默认最多允许：

- 300 次工具调用
- 1800 秒运行时间

为什么需要AgentRunLimitTracker

需要 AgentRunLimitTracker 的原因不是“框架没有限制能力”，而是它解决的是**项目运行层面的兜底保护**，和 DeepAgents / LangChain 自带 middleware 的职责层级不完全一样。

框架自带 middleware 保护的是 Agent 内部循环；AgentRunLimitTracker 保护的是当前项目的一整轮后端任务运行。

比如 coding 任务里可能发生这些情况：

1. 模型没有无限调用 LLM，但工具一直在执行、重试、读文件。
2. 子 Agent、DeepAgents 原生工具、自定义工具都在事件流里产生调用。
3. 前端已经显示某个步骤 `in_progress`，但后端任务迟迟不结束。
4. 某些事件不是标准模型调用事件，用模型调用次数不一定能准确反映任务规模。
5. 不同任务类型需要不同保护阈值，`qa` 应该短，`coding` 可以长。

所以 AgentRunLimitTracker 是一层**外部保险丝**。

它的设计重点是：

```
代码块
1   tracker.observe_event(event)
```

runtime 每消费一个 LangGraph raw event，就调用它观察一次：

- 先检查总耗时是否超过 `max_seconds`
- 再识别工具调用事件
- 工具调用次数超过 `max_tool_calls` 就抛异常

它不替代 LangChain / DeepAgents 自带限制，而是补充一层项目级控制。框架限制用于控制 Agent loop，项目自定义 AgentRunLimitTracker 用于控制服务端一次任务的资源边界、任务超时边界和前端状态收敛边界。

AgentRunLimitTracker的流程解析

第一步：先确定本轮任务的限制值

通过 `AgentRunLimits.from_env(task_kind)` 创建限制配置。

它会按任务类型读取阈值，例如：

```
代码块
1   coding -> 最多 300 次工具调用，最多 1800 秒
2   qa     -> 最多 60 次工具调用，最多 600 秒
```

第二步：创建运行追踪器

runtime 开始执行一轮 Agent 时，会创建：

```
代码块
1   tracker = AgentRunLimitTracker(task_kind=task_kind)
```

它会记录：

```
代码块
1   self.started_at = time.monotonic()
2   self.tool_calls = 0
```

也就是：

- 本轮任务什么时候开始
- 已经调用了多少次工具

第三步：每收到一个 LangGraph 事件就检查一次

Agent 运行时会不断产生 raw event。runtime 每消费一个事件，就调用：

```
代码块
1   tracker.observe_event(event)
```

这个方法先检查时间：

```
代码块
1   self._check_time()
```

如果当前时间 - 开始时间超过 `max_seconds`，就抛出：

```
代码块
1   AgentRunLimitExceeded
```

然后它会判断这个事件是不是工具调用事件：

```
代码块
1   method = event.get("method")
2   if method in {"tool_calls", "tools"}:
3       ...
```

如果识别到工具开始调用，就：

```
代码块
1   self.tool_calls += 1
2   self._check_tool_calls()
```

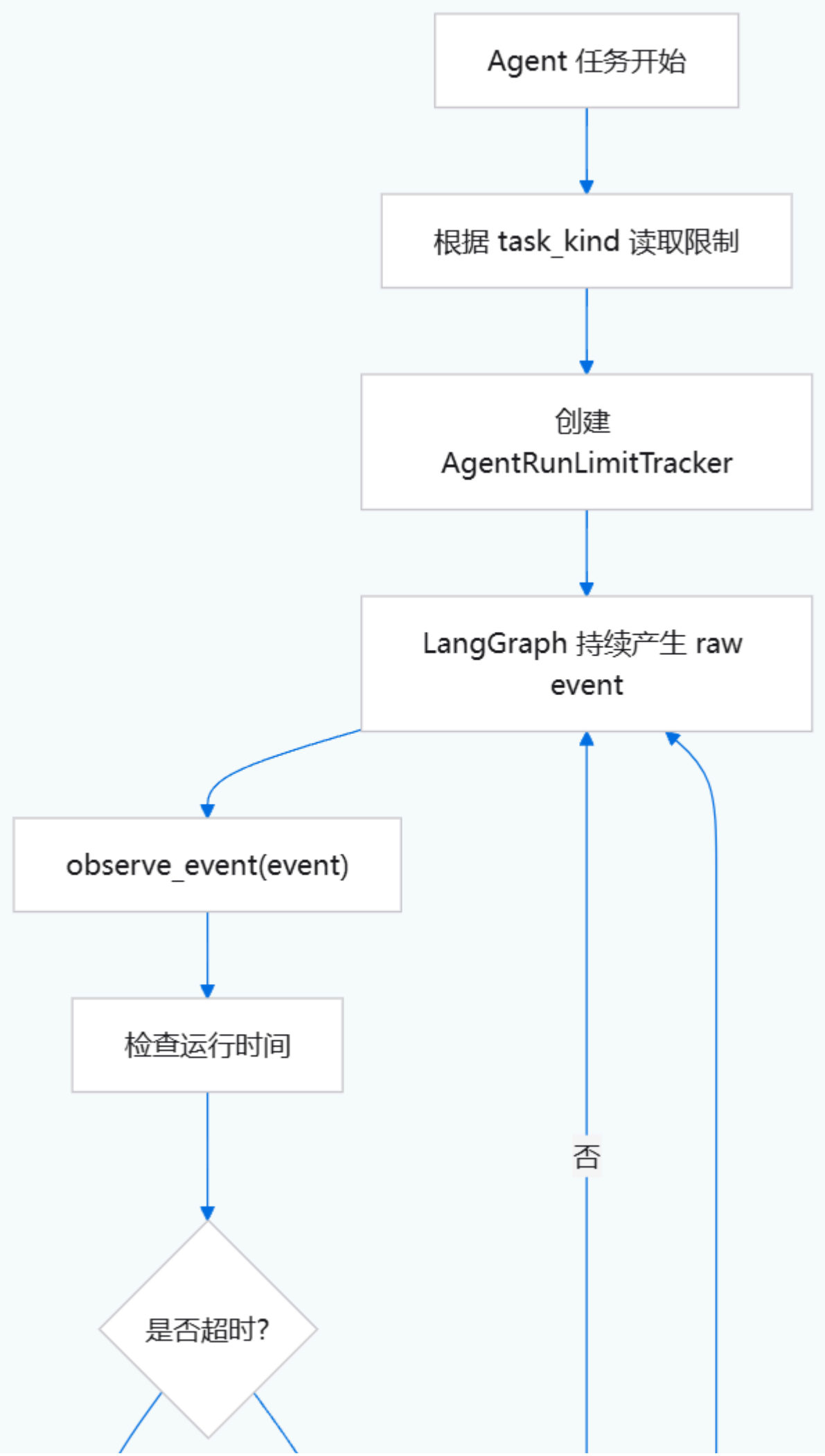
第四步：超限后抛异常，中断本轮任务

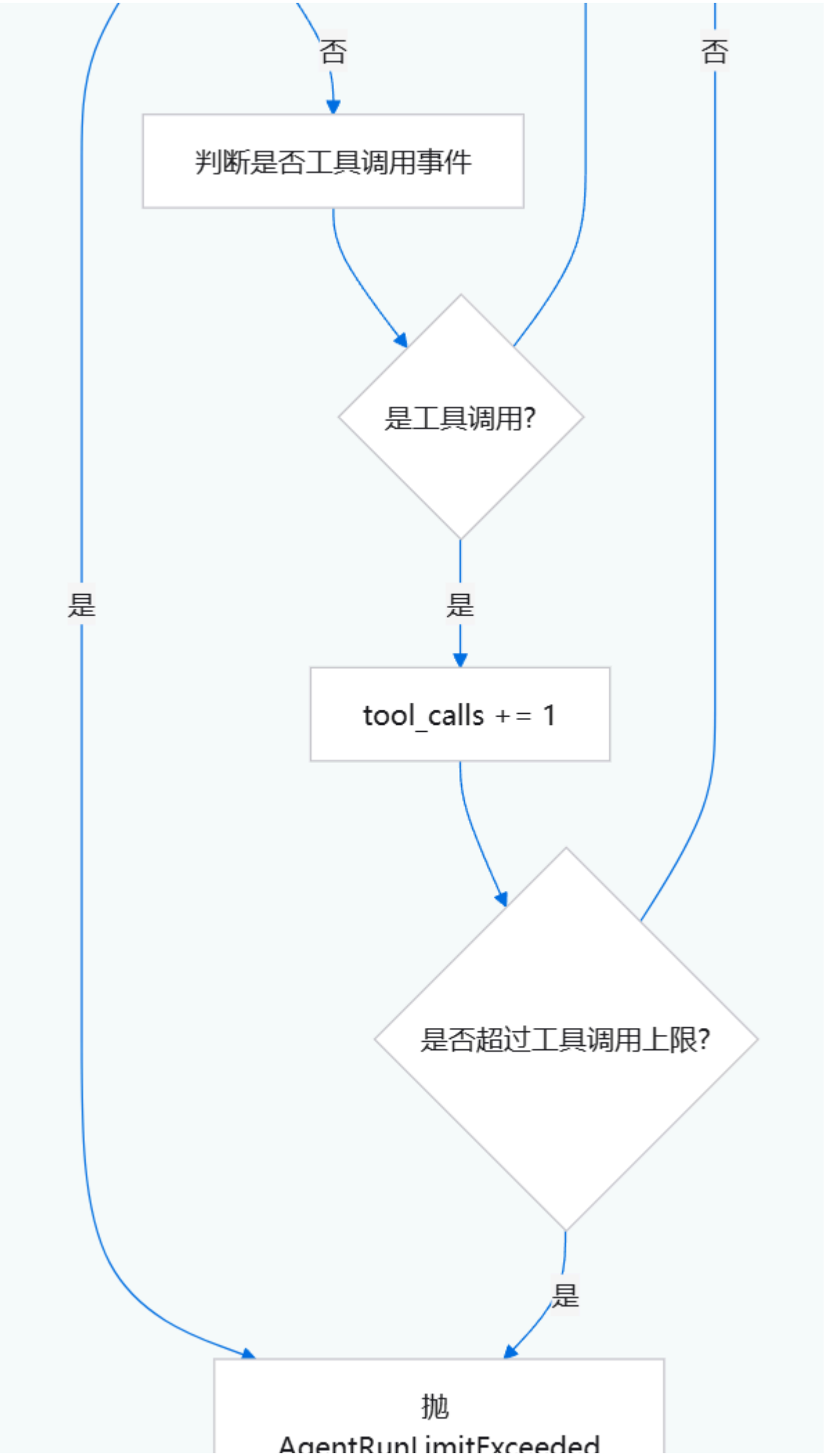
如果工具调用次数超过限制：

```
代码块
1   if self.tool_calls > self.limits.max_tool_calls:
2       raise AgentRunLimitExceeded(...)
```

runtime 捕获这个异常后，就可以停止继续执行，并把错误返回给前端或用户。

所以它的核心流程是：


Agent 任务开始
根据 task_kind 读取限制
创建 AgentRunLimitTracker
LangGraph 持续产生 raw event
observe_event(event)
检查运行时间
是否超时?
否


否
否
判断是否工具调用事件
是工具调用?
是
是
tool_calls += 1
是否超过工具调用上限?
是
抛
AgentRunLimitExceeded

Raw Event 和run_event的相关概念

1. **Raw Event 是什么**

Raw event 指的是 DeepAgents / LangGraph 原生事件流，来源在：

[streaming_runtime.py](E:/my_project/LX_AICoding/agent/core/streaming_runtime.py)

核心调用是：

代码块

```
stream = agent.stream_events(
    {"messages": [{"role": "user", "content": content}]},
    version="v3",
    config={"configurable": {"thread_id": thread_id}},
)
```

这里的 `stream` 会持续产出 LangGraph 原生事件，比如：

代码块

```
for event in stream:
    ...
```

这些 `event` 才是 raw event。

它们可能表示：

- 模型正在输出文本
- 工具调用开始
- 工具调用参数增量
- 工具调用完成
- 子 Agent 生命周期
- message chunk
- tool chunk

项目里消费 raw event 的核心函数是：

代码块

```
_consume_raw_event_stream(...)
```

它做的事情是：

```
代码块
1   for event in stream:
2       limit_tracker.observe_event(event)
3       delta = _text_delta_from_event(event)
4       tool_chunk = _tool_chunk_from_message_event(event)
5       tool_call = _tool_call_from_event(event)
6       tool_event = _tool_event_from_raw(event)
7       subagent = _subagent_from_event(event)
```

也就是说，raw event 是 DeepAgents 运行时吐出来的“原始运行信号”。

2. **run_events/ record_event() 是什么**

`record_event()` 在：[events.py](E:/my_project/LX_AICoding/agent/core/events.py)

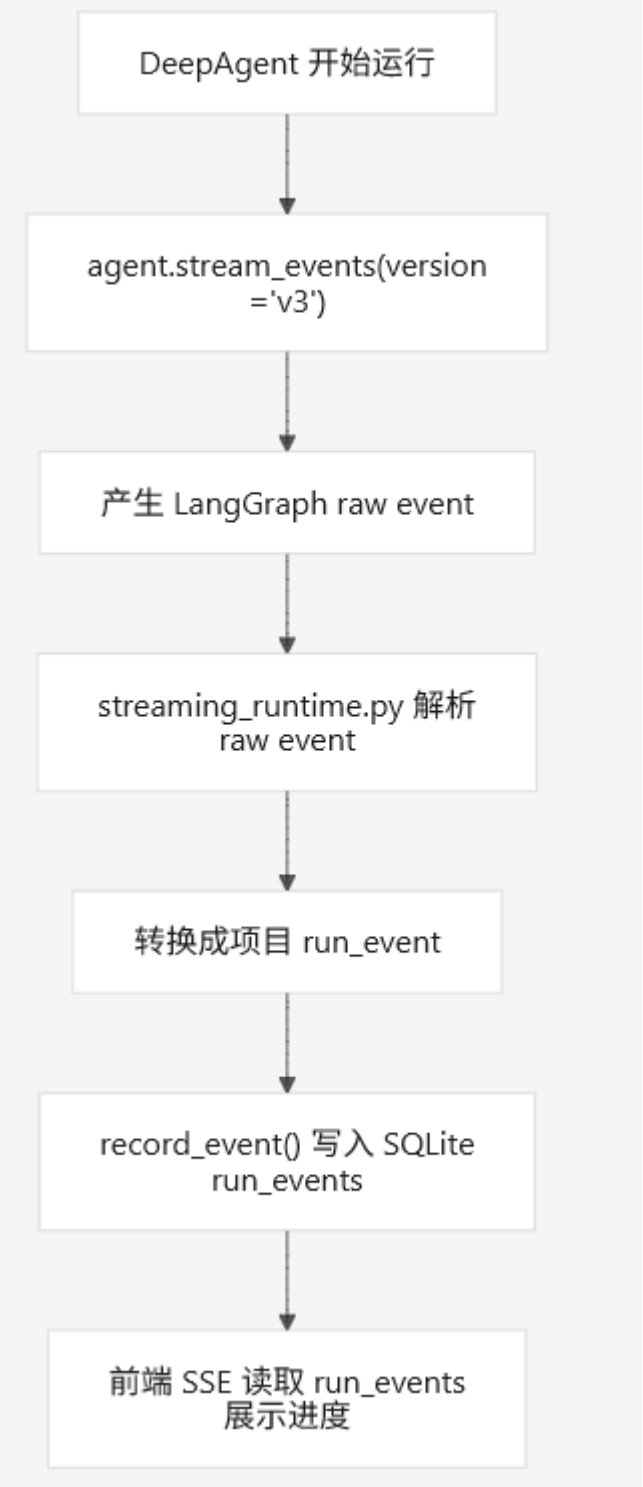
它的核心代码是：

```
代码块
1   _get_event_store().add_run_event(
2       event_id=f"{thread_id}:{key}",
3       thread_id=thread_id,
4       kind=kind,
5       title=title,
6       status=status,
7       detail=detail,
8   )
```

它只是把一条项目业务事件写入 SQLite。写入的表是：run_events；表结构在：sqlite_store.py中；所以 `record_event()` 创建的是：项目自己的 run_event，不是：DeepAgents raw event

3. **两者关系**

可以这样理解：



4. 为什么需要 record_event()

因为前端不直接消费 DeepAgents raw event。

项目采用的是：

代码块

```
DeepAgents raw event
        ↓
后端解析、清洗、合并
        ↓
record_event 写入 run_events
        ↓
```

```
7   前端 SSE 轮询 / 推送 thread 快照
```

这样做有几个好处：

- 前端不用绑定 LangGraph 原生事件协议。
- raw event 字段变化时，只需要改后端解析层。
- 可以把模型输出、工具调用、Gitee PR、URL 抓取、错误信息统一成一种前端步骤格式。
- 可以把 `in_progress`、`completed`、`error` 状态持久化，刷新页面后还能看到最近一次任务过程。
- 工具内部也可以主动调用 `record_event()`，不一定都依赖 raw event 解析。

一句话总结

`raw event` 是 DeepAgents / LangGraph 原生运行事件，由 `agent.stream_events()` 产生。

`record_event()` 是项目自己的事件落库函数，把“后端希望前端展示的步骤”写入 `run_events` 表。

4、ContextInjectionMiddleware

在 Agent 运行前读取仓库级记忆，并注入到系统上下文。Agent 不需要每轮都从零分析仓库

5、MemoryUpdateMiddleware

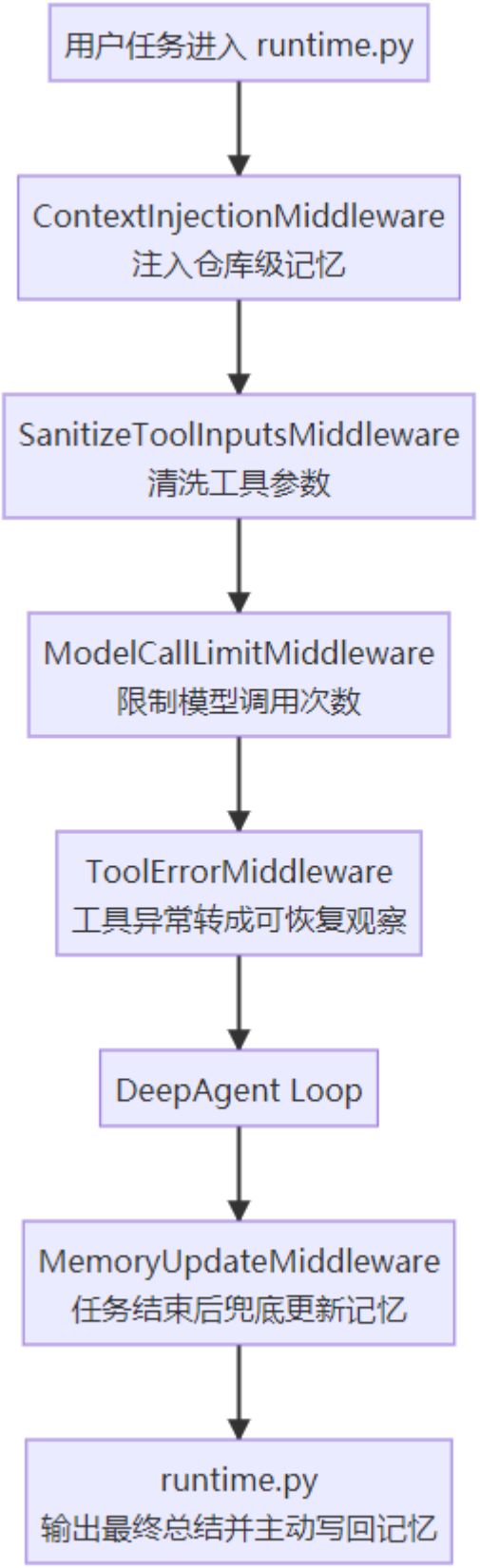
Agent 完成后兜底提取本轮结论，写回仓库级记忆，当前主要写回路径在 `runtime.py`，该中间件是兜底机制。

为什么不只靠中间件？

因为不同任务类型结束位置不同。方案任务、编码任务、问答任务的最终回答组织方式不同，`runtime.py` 更容易拿到“最终对用户说了什么”。因此当前项目采用：

1. `runtime.py` 在任务成功结束后主动调用 `repo_memory_update.py`。
2. `MemoryUpdateMiddleware` 在 Agent 完成后兜底更新。
3. 写入前由 `repo_memory_update.py` 清洗和压缩内容，避免污染长期记忆。

6、自定义中间件的调用顺序



说明：`AgentRunLimitTracker` 不属于 LangChain middleware，它位于流式事件解析层，用来限制整轮工具调用和模型调用次数，防止前端一直等待。

这里的顺序体现了一个工程原则：

1. 先尽量把错误参数纠正或拒绝。
2. 再让真实工具执行。

3. 如果工具仍失败，把异常变成可恢复反馈。
4. 在流式层持续观察运行规模，避免 Agent 长时间失控。

第六章、项目提示词和任务分类规则

Prompt 在 Agent 项目里不是“写一段好听的话”，而是运行规约。

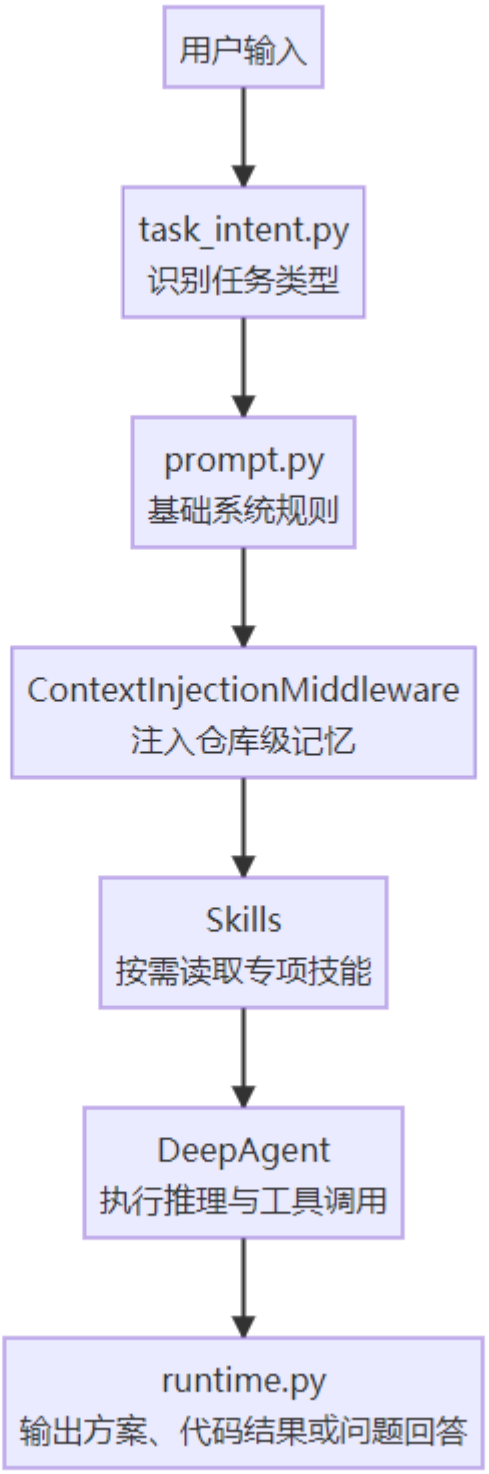
它至少要回答：

问题	Prompt 中怎么约束
支持什么平台	当前只支持 Gitee
输出语言	面向用户必须中文
文件路径怎么写	使用 /projects/<repo> 虚拟路径
使用哪些工具	DeepAgents 原生工具和自定义工具
编码任务怎么做	读取、修改、验证、提交、PR
只读任务怎么做	禁止修改、提交、push、PR
何时使用 skill	首次分析仓库或生成方案时使用 repo-bootstrap-analysis

1、当前 Prompt 结构

`prompt.py` 负责“稳定规则”，中间件负责“运行时上下文”，skills 负责“专项流程”，用户输入负责“本轮目标”。这四类信息不要混在一个超长提示词里。

文件	作用
agent/prompt.py	定义主 Agent 的系统提示词和任务规则
agent/core/task_intent.py	判断用户指令属于编码、方案、问答、仓库分析等哪类任
agent/core/runtime.py	根据任务类型组织运行流程，决定是否进入方案生成或任施
agent/core/middleware/context_injection.py	运行前把仓库级记忆注入 Agent 上下文
agent/server.py	FastAPI 服务入口，接收前端请求并触发 Agent 运行
E:\ai_workspace\skills	DeepAgents 原生 skills 目录，存放可被 Agent 读取的技



基础系统规则应该包含什么

规则类别	讲解重点
平台规则	第一版只考虑 Gitee 仓库，不考虑 GitHub、GitLab 等其他平台
工作区规则	Agent 看到的是 E:\ai_workspace，仓库应位于 /projects/<repo_name>
中文输出规则	面向中文教学场景，过程解释、最终回答、错误说明都必须使用中文
安全规则	不能读取或输出 .env、.secrets、token、私钥等敏感信息
方案优先规则	用户提出较复杂代码需求时，先给技术方案，确认后再实施
工具规则	能通过文件后端完成的文件操作，优先走 DeepAgents backend 暴露的工具
测试规则	代码修改后要运行相关测试，失败时继续分析并尝试修复
Gitee 规则	分支、提交、推送、Pull Request 都以 Gitee 为目标平台

当前推荐的任务类型

如果所有用户输入都按照“写代码”处理，会出现三个问题：

问题	表现
简单问答被过度执行	用户只是问目录结构，Agent 却开始创建分支、写代码、提交 PR
技术方案被直接实施	用户想先讨论方案，Agent 直接改文件，教学时不可控
工具调用次数暴涨	Agent 反复分析仓库、运行测试、读同一批文件，容易触发运行保护上限

因此项目里通过 `task_intent.py` 先做任务分类。课堂上可以把它理解成 Agent 的“入口分流器”。

类型	典型用户输入	推荐行为
qa	“当前项目有哪些目录？”	读取必要上下文后直接回答，不改代码，不创建 PR
planning	“给我一个改造 SQLite 的方案”	读取仓库，输出完整方案，最后询问是否确认实施
coding	“确认实施” “帮我写用户模块”	创建或复用分支，修改代码，运行测试，提交并创建 PI
analysis	“帮我分析这个仓库结构”	读取文件结构，整理结论，不做代码修改
review	“帮我 review 这个 PR”	读取 diff 和上下文，输出审查意见

技术方案任务的提示词要求

方案任务和编码任务最大的区别是：它必须输出可读、完整、可确认的方案，而不是直接改代码。

部分	内容
需求理解	用自己的话复述用户想做什么
当前结构	列出已经读取到的关键文件和模块关系
改造目标	明确要新增、修改、保持不变的内容
数据结构	如果涉及数据存储，说明表结构、字段、约束或文件格式
接口设计	如果涉及后端，说明路由、请求、响应、鉴权方式
页面影响	如果涉及前端，只说明对接点，不展开前端源码
测试策略	说明应该新增哪些测试、回归哪些测试
风险点	说明兼容性、数据迁移、权限、并发等风险
确认语	结尾询问：是否确认实施该方案？

编码任务的提示词要求

编码任务应当更强调执行闭环：

1. 定位仓库和分支。
2. 读取仓库级记忆，避免重复分析。
3. 根据用户确认的方案制定任务清单。
4. 只修改必要文件。
5. 运行最小相关测试。
6. 如果测试失败，分析原因并继续修复。
7. 全部通过后提交、推送、创建或复用 Gitee Pull Request。
8. 最后总结修改清单、测试结果、分支和 PR 地址。

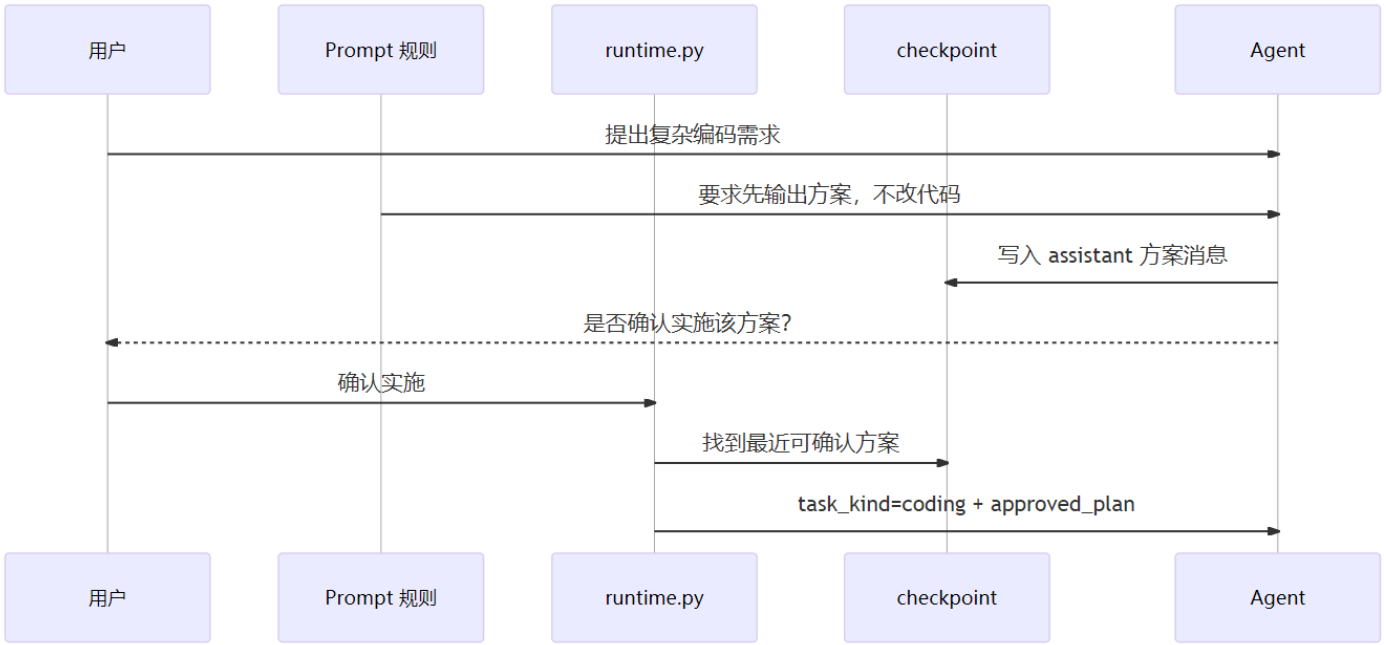
注意：这个过程就是一个典型的 Coding Agent Loop。它不是一次模型调用解决问题，而是“计划、工具调用、观察、修复、验证、总结”的循环。

方案任务和编码任务的关键差异

对比项	方案任务	编码任务
是否改代码	不改	改
是否提交 PR	不提交	成功后提交
是否必须测试	只提出测试策略	必须尽量运行测试
输出重点	技术设计、风险、步骤	修改清单、测试结果、分支和 PR
结束语	是否确认实施该方案?	总结本轮完成情况

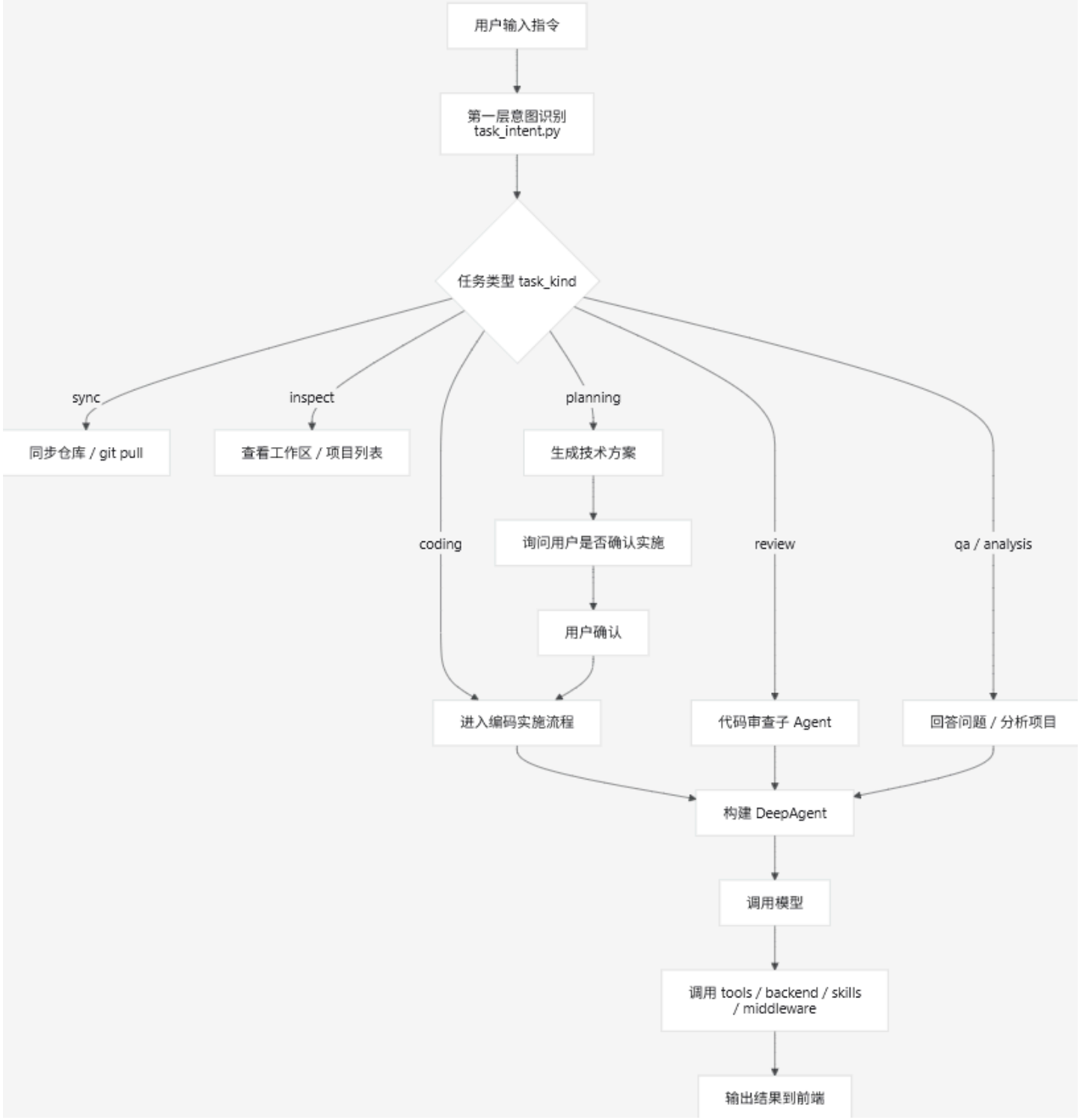
这也是当前项目“人类确认”的核心。学生要理解：在真实项目中，AI 不应该看到一个复杂需求就直接动手改代码。先生成方案可以降低误改风险，也方便程序员通过输入交互内容修改技术方案和审核技术方案。

两种任务的转换流程



2、`task_intent.py` 多层用户意图识别

`task_intent.py` 就是当前 Agent 项目的第一层用户意图识别。



这里的 `task_intent.py` 不负责真正解决问题，它主要负责：

职责	说明
粗粒度分类	判断是 coding、planning、review、qa、analysis、sync、inspect
权限前置判断	哪些任务只读，哪些任务允许写代码
降低 Agent 误操作风险	避免用户只是问问题时，Agent 却开始改代码
帮 runtime 做分流	不同任务类型走不同 prompt、tools、middleware、运行保护阈值

所以，它更像是 **Agent 运行前的交通分流系统**。简化之后，逻辑类似这样：

代码块

```
def classify_task_kind(prompt: str) -> TaskKind:
    text = normalize(prompt)

    if is_pull_only_task(text):
        return "sync"

    if is_workspace_listing_task(text):
        return "inspect"

    if contains_review_keywords(text):
        return "review"

    if contains_plan_keywords(text) and not contains_coding_keywords(text):
        return "planning"

    if contains_analysis_keywords(text) and not contains_coding_keywords(text):
        return "analysis"

    if contains_qa_keywords(text) and not contains_coding_keywords(text):
        return "qa"

    if contains_coding_keywords(text):
        return "coding"

    return "qa"
```

任务类型	是否只读	典型行为
qa	是	回答用户问题
analysis	是	分析项目结构、解释代码
planning	是	输出技术方案，不改代码
review	是	审查 PR / diff，不改代码
inspect	是	查看工作区、列项目
sync	特殊	只同步远程代码
coding	否	允许写文件、测试、提交、push、PR

关键词规则 vs 小模型意图识别

企业 Agent 项目里，第一层意图识别通常有两类方案。

方案 A：关键词 / 规则分类

维度	规则分类
速度	非常快
成本	几乎为零
稳定性	很高
可解释性	很强，命中哪个关键词很清楚
维护方式	不断补关键词和规则
上下文理解	较弱
模糊表达处理	较弱
安全性	高，适合权限边界判断
适合场景	高风险操作入口、教学项目、任务类型较少的系统

典型问题：

用户输入	规则分类可能的问题
“可以，按刚刚那个来”	当前句子没有“修改、实现、编码”等关键词，
“重新改一下刚才的方案”	可能误判成 coding，也可能判成 planning
“不要改代码，只给我实现方案”	同时有“改代码”和“方案”，需要否定词判断
“帮我 review 这个 PR，然后根据问题给修复方案”	既像 review，又像 planning
“这个可以，试试”	必须知道“这个”指的是上一轮技术方案

方案 B：小规模 LLM 分类（8B-qwen3.5的模型-BF8）

维度	小模型意图识别
速度	慢于规则
成本	有额外模型成本
稳定性	需要约束，否则有波动
可解释性	中等，可以让模型返回 reason
维护方式	维护 prompt 和 schema
上下文理解	强
模糊表达处理	强
安全性	不能完全依赖，需要规则兜底
适合场景	多轮对话、复杂业务、自然语言表达多样的企业 Agent

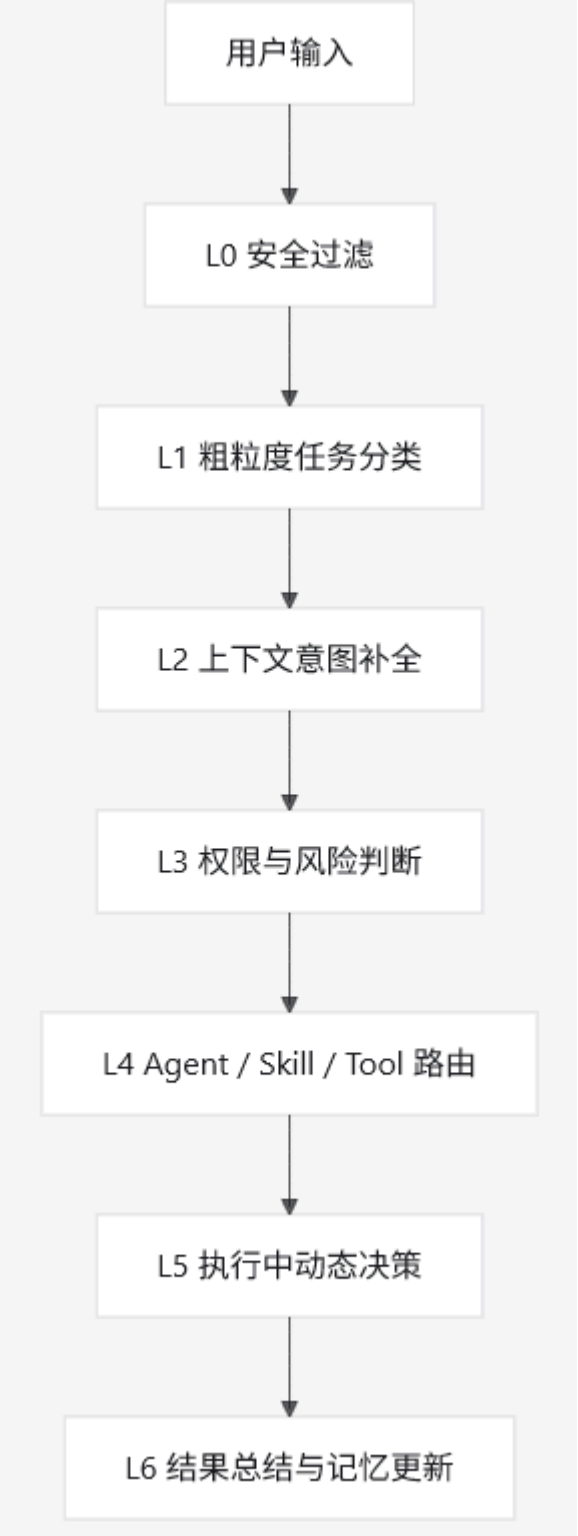
小模型分类可以输出结构化结果：

代码块

```
{
  "task_kind": "planning",
  "confidence": 0.86,
  "is_read_only": true,
  "is_approval": false,
  "is_plan_revision": true,
  "requires_confirmation": true,
  "reason": "用户要求修改上一轮技术方案，没有要求直接改代码"
}
```

3、企业 Agent 项目的多层意图识别

企业级 Agent 通常不会只做一层分类，而是多层判断。


用户输入
L0 安全过滤
L1 粗粒度任务分类
L2 上下文意图补全
L3 权限与风险判断
L4 Agent / Skill / Tool 路由
L5 执行中动态决策
L6 结果总结与记忆更新

对应到企业 Agent：

层级	名称	作用	当前项目
L0	安全过滤	判断是否危险、越权、注入	部分在 prompt 和工具权
L1	粗分类	判断是问答、方案、编码、审查、同步	task_intent.py
L2	上下文补全	理解“确认”“按刚刚的”“修改上一版”	当前 runtime 有部分逻辑
L3	权限判断	是否允许写文件、提交、push	is_read_only_task + ru
L4	路由	选择主 Agent、子 Agent、skills、tools	graph.py / server.py / r
L5	执行决策	Agent Loop 内部工具调用和推理	DeepAgent
L6	记忆更新	写入 repo memory、任务总结	memory middleware

所以，`task_intent.py` 只是其中的 L1。

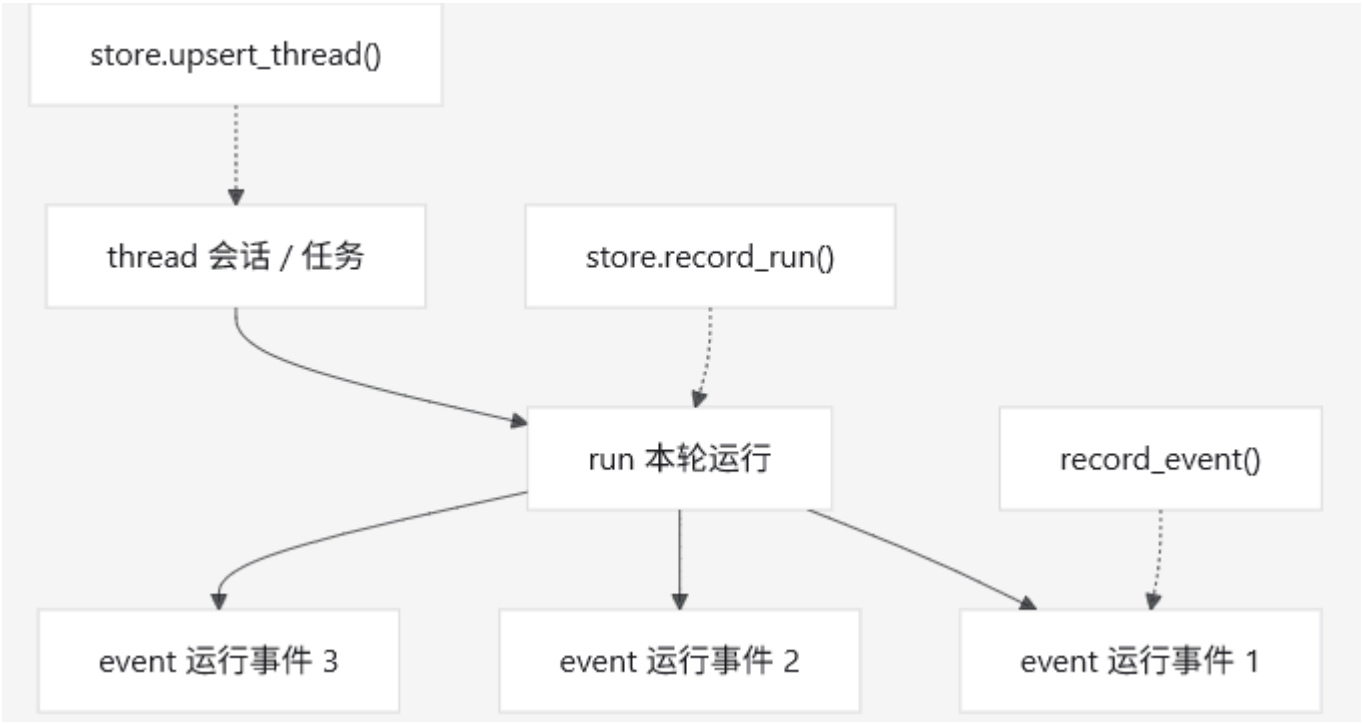
4、store中的业务数据的存储和记录：提取初探

这三个函数都在“记录任务运行状态”，但层级不同。可以把它们理解成：

> `store.upsert_thread` 记录“这个会话/任务是谁”。
> `store.record_run` 记录“这一次运行是什么状态”。
> `record_event` 记录“运行过程中发生了哪些步骤”。

三者定位对比

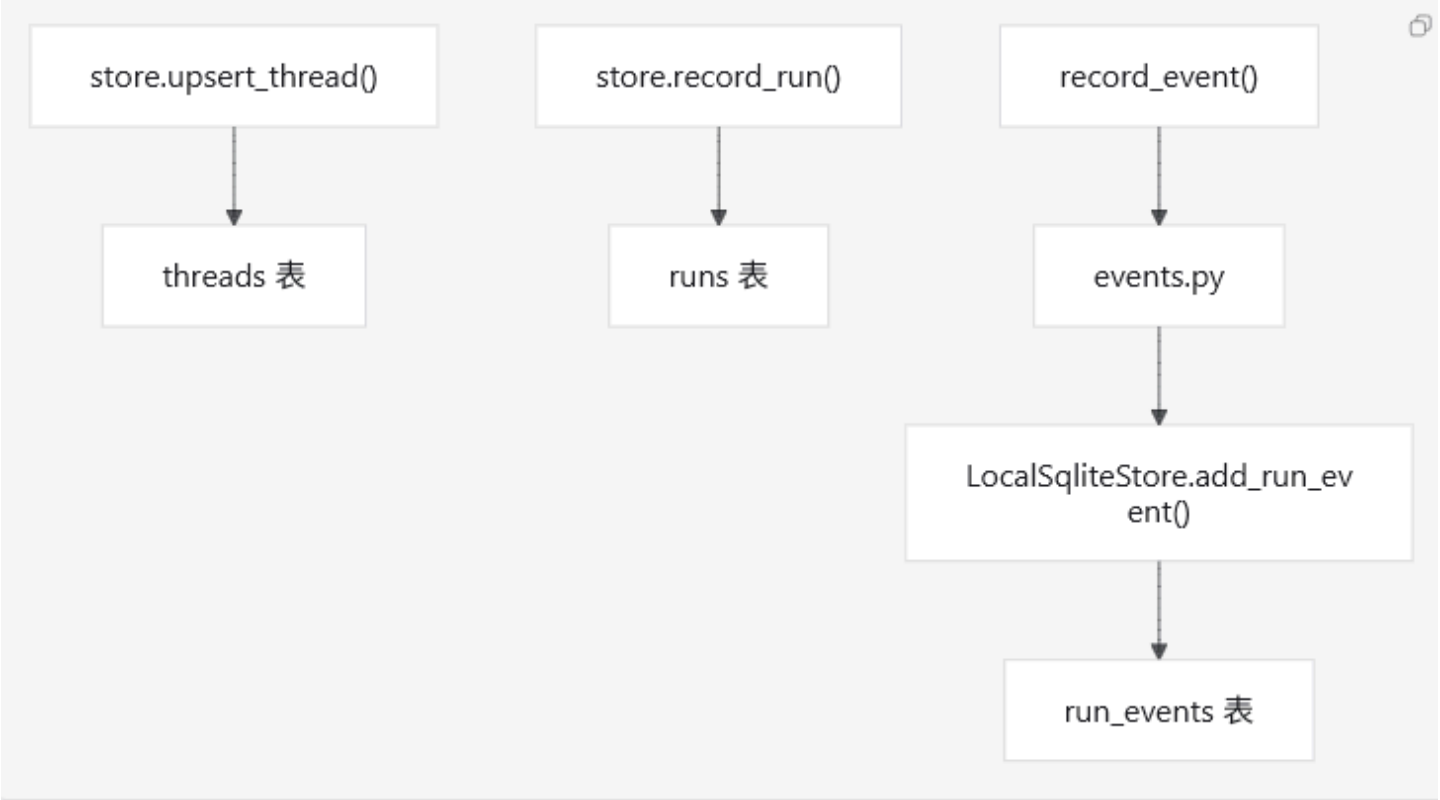
函数	数据粒度	主要回答的问题	
store.upsert_thread()	会话 / 任务级别	这是哪个任务？属于哪个仓库？当前总体状态是什么？	左侧任务列表、任
store.record_run()	单次运行级别	这个 thread 本轮执行是否成功？什么时候开始/结束？错误是什么？	当前运行状态、这
record_event()	步骤 / 事件级别	Agent 执行过程中发生了哪些动作？	页面中的步骤流、化



对应关系如下。

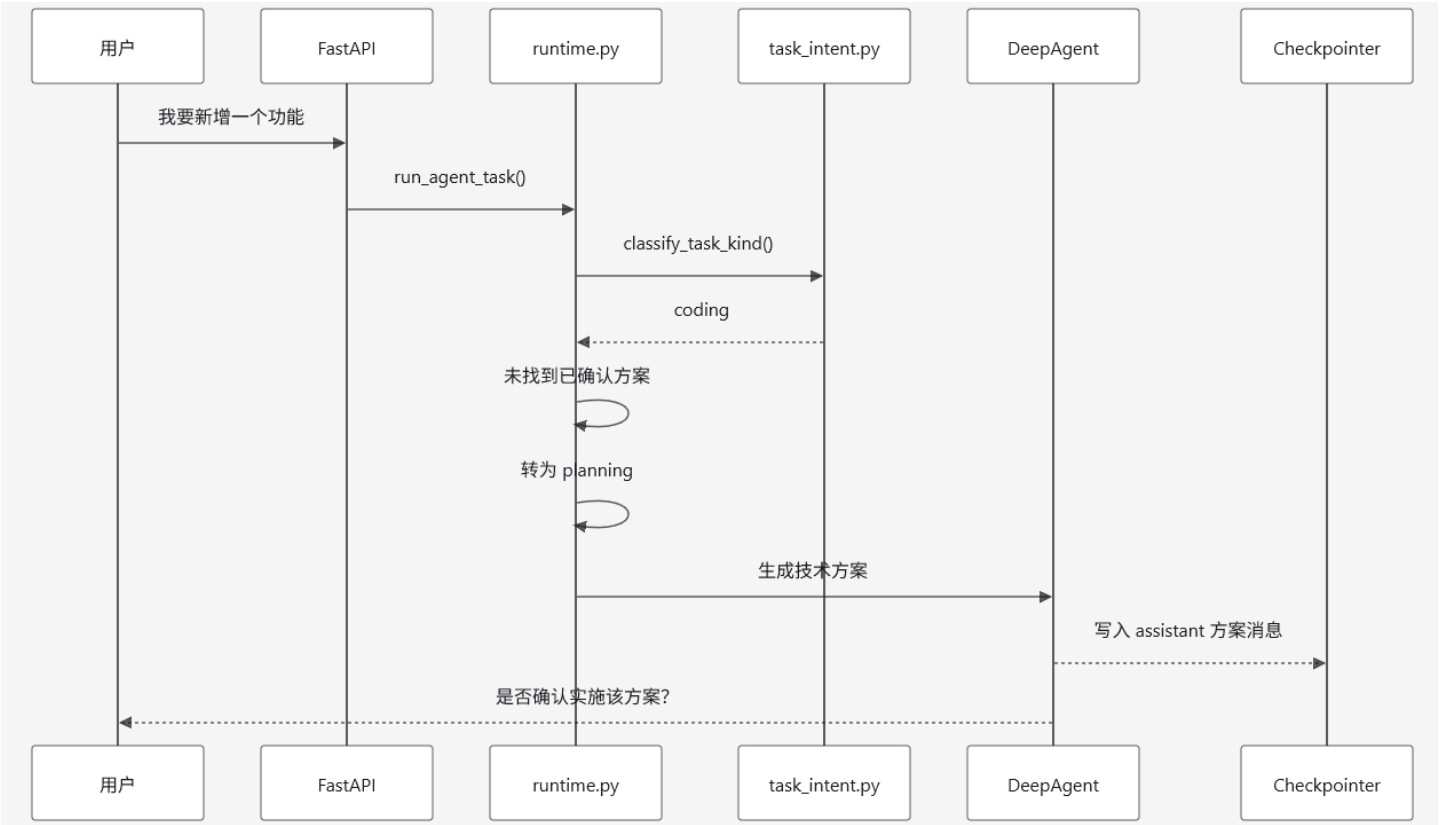
函数	最终写入的表	写入方式
store.upsert_thread()	threads	INSERT ... ON CONFLICT(thread_id) DO UPDATE
store.record_run()	runs	INSERT ... ON CONFLICT(run_id) DO UPDATE
record_event()	run_events	内部调用 store.add_run_event()，然后 INSERT ... ON CONFLI

具体链路是：



5、runtime.py：运行时编排层”

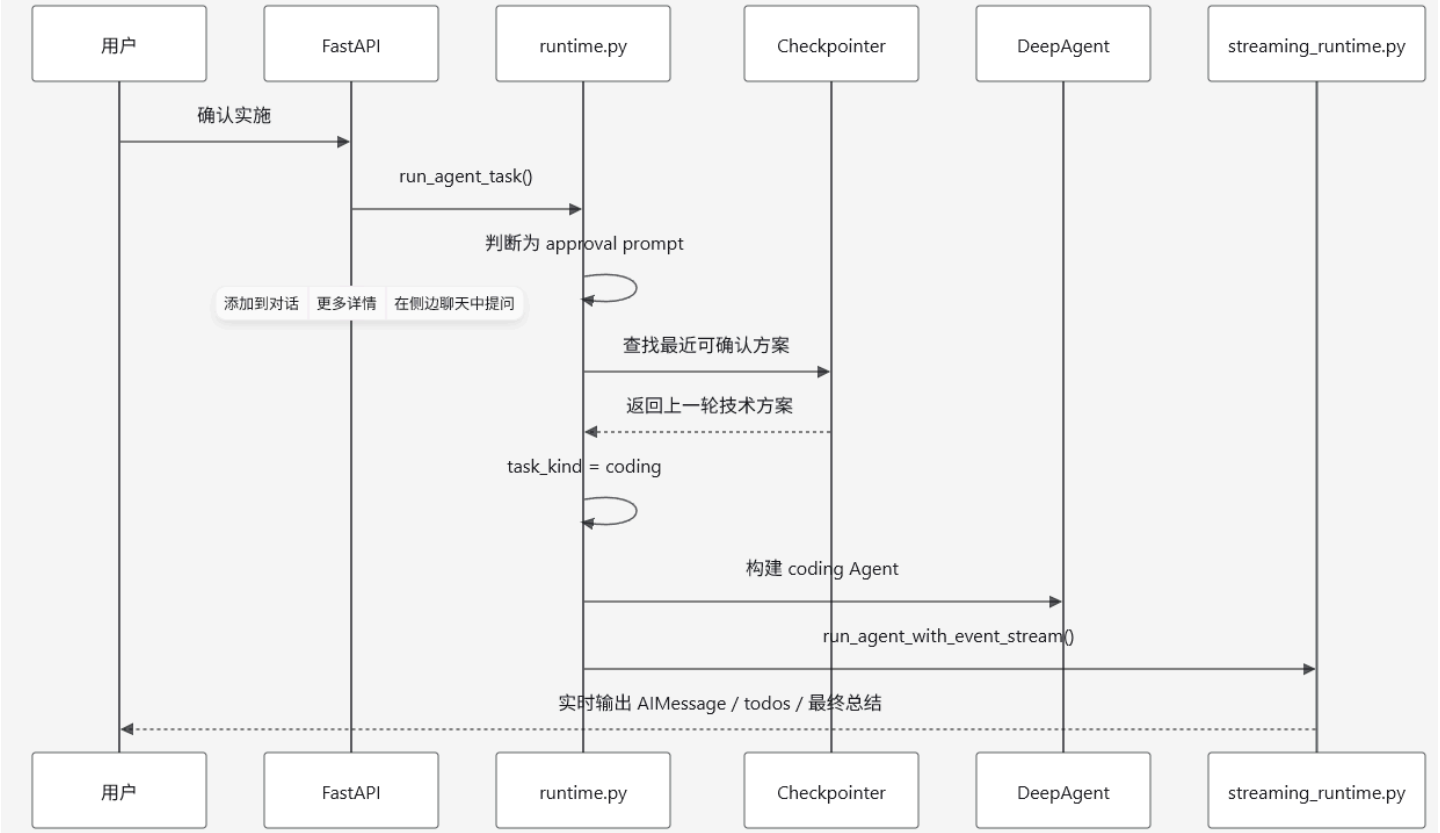
用户首次提出编码需求



这里重点是：

> 用户虽然提出了 coding 类需求，但 runtime 不会立刻写代码，而是强制先进入 `planning`。

用户确认实施



这里 runtime 做了一个非常重要的转换：

用户当前输入	真正传给 coding Agent 的任务
“确认实施”	上一轮完整技术方案 + 原始用户需求

否则 Agent 会把“确认实施”四个字当成开发需求。

它可以简化成下面的伪代码：

代码块

```
def run_agent_task(repo_url, prompt, thread_id=None):
    # 1. 特殊任务：不需要模型
    if is_workspace_listing_task(prompt):
        return run_workspace_listing_task(...)

    if is_pull_only_task(prompt):
        return run_pull_only_task(...)

    # 2. 第一层任务分类
    task_kind = classify_task_kind(prompt)

    # 3. 判断当前是不是已有会话
    thread_id = thread_id or new_uuid()
    existing_thread = store.get_thread(thread_id)

    # 4. 如果用户说“确认实施”
```

```
    if existing_thread and is_approval_prompt(prompt):
        plan = latest_confirmable_plan_from_checkpoint(thread_id)
        if plan:
            task_kind = "coding"
            approved_plan_text = plan.content
            coding_prompt = restore_original_user_requirement(plan)

    # 5. 如果用户不是确认，而是修改上一轮方案
    elif existing_thread and is_plan_revision_prompt(prompt):
        plan = latest_confirmable_plan_from_checkpoint(thread_id)
        if plan:
            return run_plan_response_task(previous_plan=plan,
revision_prompt=prompt)

    # 6. 如果是 coding，但没有已确认方案，先转 planning
    if task_kind == "coding" and approved_plan_text is None:
        return run_plan_response_task(...)

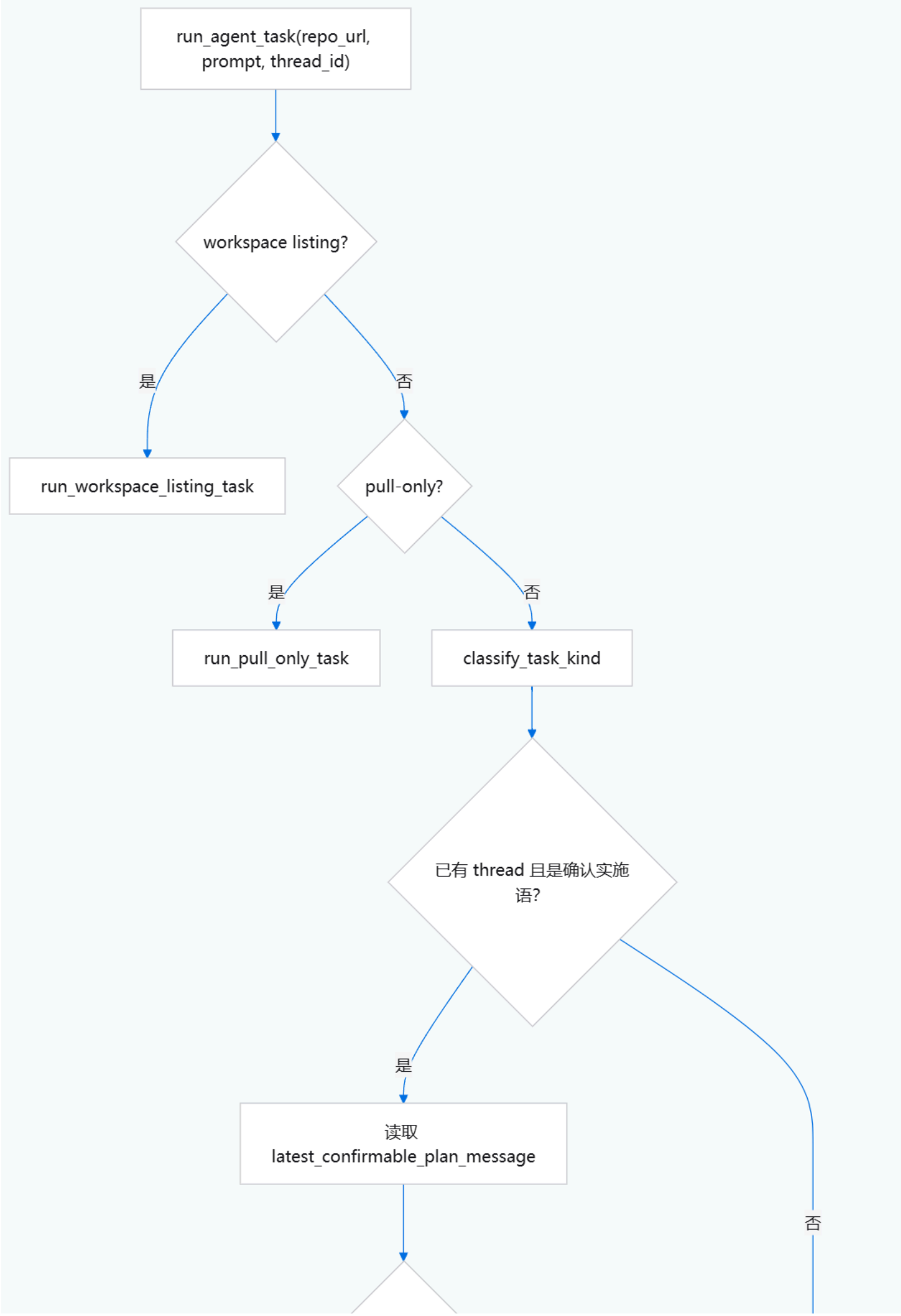
    # 7. 真正创建 Agent 并运行
    agent = build_agent_for_runtime(thread_id, task_kind, repo_url)

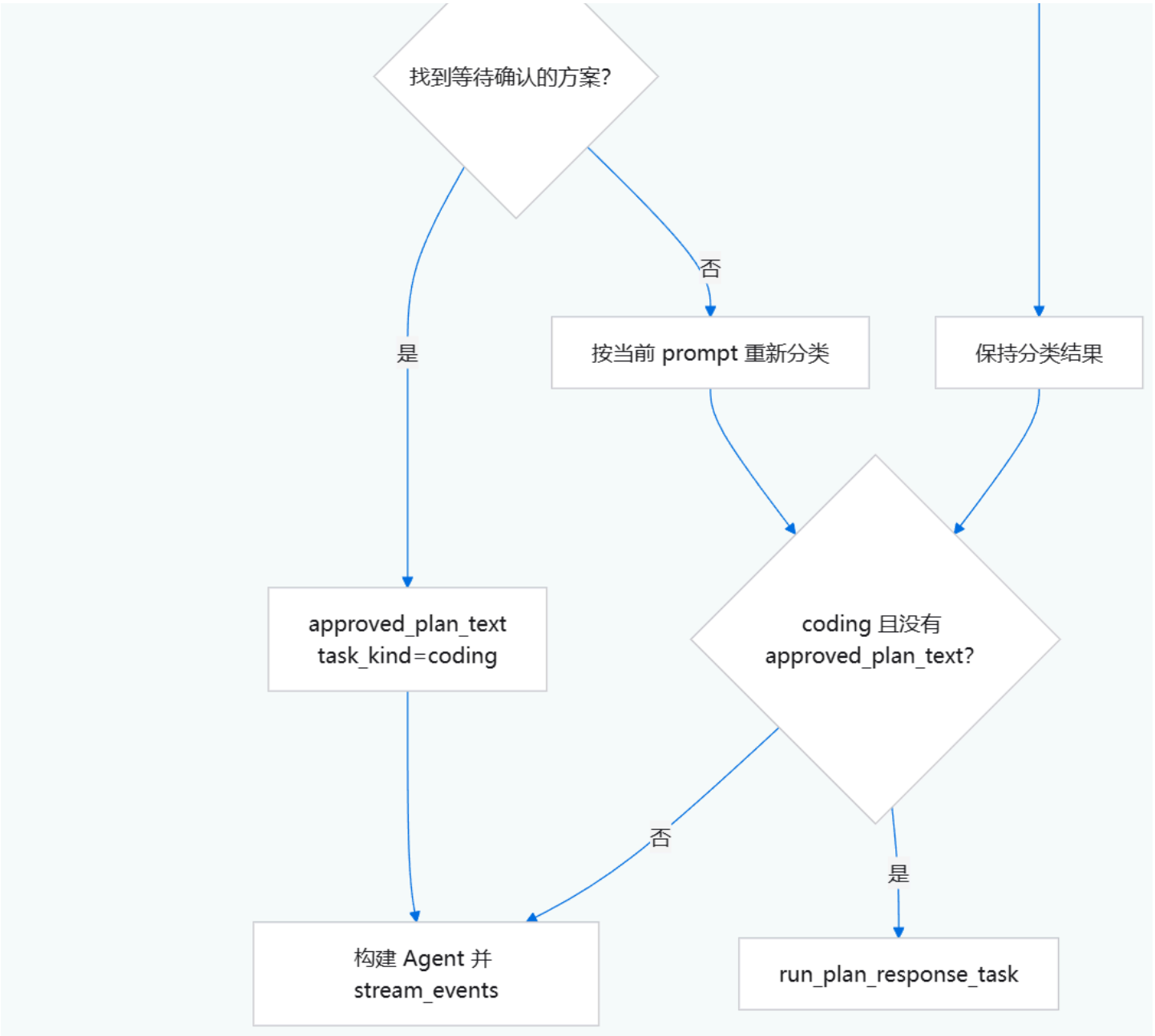
    result = run_agent_with_event_stream(
        agent=agent,
        content=build_agent_user_content(...),
        task_kind=task_kind,
    )

    # 8. 任务完成，更新状态和仓库记忆
    update_status()
    update_repo_memory_from_final_answer()

    return result
```

runtime.py 里的主要函数拆解


run_agent_task(repo_url, prompt, thread_id)
workspace listing?
是
否
run_workspace_listing_task
pull-only?
是
否
run_pull_only_task
classify_task_kind
已有 thread 且是确认实施语?
是
否
读取 latest_confirmable_plan_message


找到等待确认的方案?
否
是
按当前 prompt 重新分类
保持分类结果
approved_plan_text
task_kind=coding
coding 且没有
approved_plan_text?
否
是
构建 Agent 并
stream_events
run_plan_response_task

函数	作用
_build_agent_for_runtime()	构造 Agent 运行配置，调用 agent.server.get_agent() 创建 D
_ensure_repo_memory_for_mapping()	根据仓库映射初始化仓库级长期记忆
_build_agent_user_content()	组装发给 coding / qa / review Agent 的用户内容
_build_plan_user_content()	组装“只生成技术方案”的用户内容
_is_approval_prompt()	判断用户是否在确认实施方案
_is_plan_revision_prompt()	判断用户是否在修改上一轮方案
_latest_confirmable_plan_message()	从 checkpoint 中找最近可确认的技术方案
_latest_non_approval_user_prompt()	找到上一轮真正的用户需求，避免把“确认”当需求
initialize_task_record()	初始化任务记录，创建 thread 状态
run_workspace_listing_task()	不调用模型，直接列出 workspace 项目
run_pull_only_task()	不调用模型，直接执行 clone / fetch / pull
run_plan_response_task()	专门生成技术方案
run_agent_task()	主入口，所有常规任务都从这里进入
get_task()	给前端读取单个任务详情
list_tasks()	给前端读取任务列表
delete_task()	删除 Store 业务数据和 checkpoint 会话状态

---

runtime.py 的几类任务分支

分支	是否调用模型	入口函数	说明
查看工作区项目	否	run_workspace_listing_task()	直接用 LocalShellBackend.list_files()
git pull	否	run_pull_only_task()	直接执行 Git 命令
技术方案	是	run_plan_response_task()	只读，生成方案，最后询问确认
普通问答 / 分析	是	run_agent_task()	task_kind 为 qa 或 analysis
代码审查	是	run_agent_task()	task_kind 为 review，走 reviewer 能力
编码实施	是	run_agent_task()	必须先有确认过的方案

一、`run_workspace_listing_task`：本地项目列表分支

这个分支不调用模型，适合用户问：

当前 workspace 有哪些项目？

执行步骤：

1. 调用 `initialize_task_record`。
2. 清理旧 `run_events`。
3. 创建 `LocalShellBackend`。
4. 调用 `backend.list_files("projects")`。
5. 写入 agent 消息，metadata 为 `{"task_kind": "inspect"}`。
6. 更新 run 和 thread 状态为 completed。

二、`run_pull_only_task`：只同步远程代码分支

这个分支也不调用模型，适合用户明确说：

先把远程代码 pull 一下

执行步骤：

步骤	说明
解析 Gitee URL	parse_gitee_repo_url(repo_url)
定位本地目录	discover_repo_mapping(...)
已存在仓库	git remote set-url、git checkout master、git fetch --all、git pull origin master --ff only
不存在仓库	git clone 到 projects
保存映射	save_clone_mapping(...)
写回状态	run/thread completed 或 failed

特殊处理：

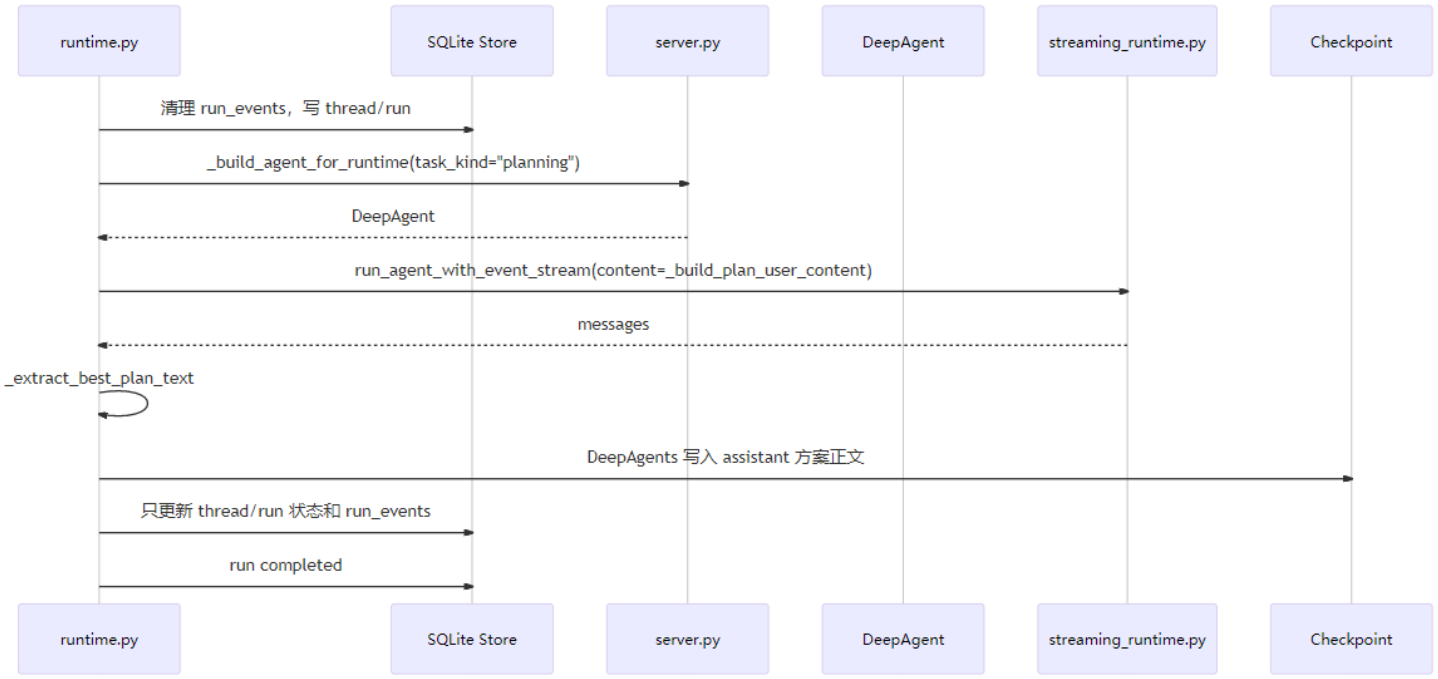
_run_git_with_fetch_head_retry

它用于处理 Windows 下偶发的 `.git/FETCH_HEAD` 权限异常：如果检测到该错误，会尝试删除 `FETCH_HEAD` 后重试一次。

三、`run_plan_response_task`：生成技术方案

它的职责是为 coding 需求生成一个只读技术方案。

执行流程：



两个问题：1、创建Agent的工厂函数是什么样的？ 2、一个Agent如何流式输出并执行的

第七章、设计业务 Store 和 Checkpoint

Agent 项目通常有两类状态：

状态类型	谁使用	保存内容
Agent runtime state	LangGraph / DeepAgents	messages、工具调用状态、图恢复信息
Business state	应用后端和前端	任务列表、运行状态、事件、PR URL

不要把两者混在一起。

如果只记住一句话，就是：

> Checkpoint 服务 Agent 恢复，Store 服务产品展示和业务查询。

1、本项目三个 SQLite

当前项目中至少会涉及几类 SQLite：

SQLite	作用
store.sqlite	业务数据，例如会话、任务、事件、仓库映射
checkpoints.sqlite	LangGraph checkpoint
langgraph_store.sqlite	DeepAgents StoreBackend 记忆数据

它们不应该随意合并。

对比项	checkpoints.sqlite	store.sqlite
谁写	LangGraph SqliteSaver	项目代码 LocalSqlit
谁读	DeepAgents / LangGraph、checkpoint_history.py	FastAPI、Dashboar
数据形态	图状态、messages、checkpoint 元数据	threads、runs、ru mappings
是否面向前端	前端历史正文的事实来源，但通过后端 API 转换后输出	只提供业务状态和当
是否建议手写 SQL 修改	不建议	可以按业务表结构查

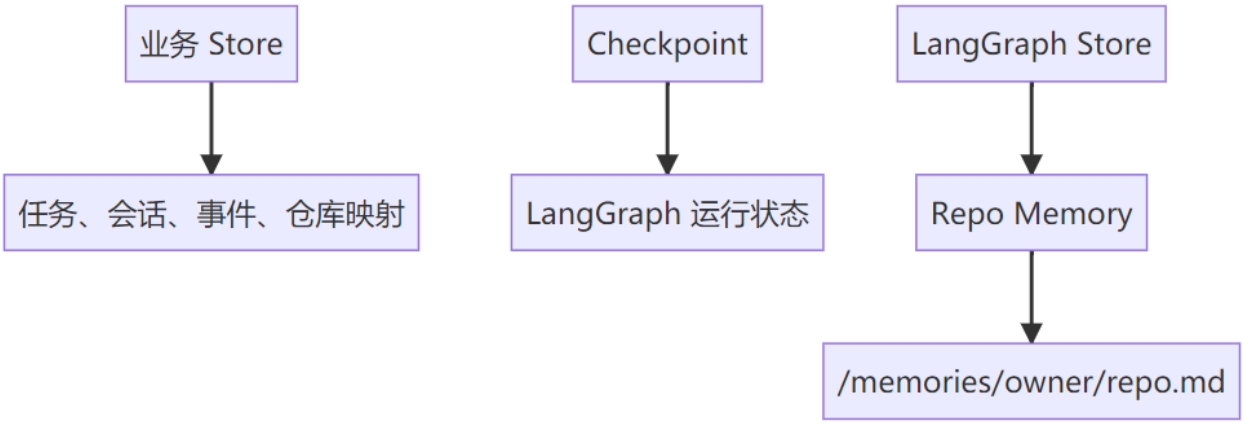
很多学员会问：既然 LangGraph 已经有 checkpoint，为什么还要业务 Store？原因是二者保存的数据类型不同。

对比项	Checkpoint	业务 Store
保存内容	LangGraph 运行状态、用户消息、assistant 消息	任务记录、会话列表、运行
面向对象	框架运行	产品业务
谁主要使用	LangGraph / DeepAgents / 后端历史转换逻辑	FastAPI / 前端状态接口 / ru
是否适合直接给前端查	不适合前端直连，但后端会把它转换成可见历史	适合展示业务状态和运行事 源

一句话：Checkpoint 是“让 Agent 和对话历史能恢复”，业务 Store 是“让应用能展示任务状态、运行过程和业务结果”。两者都服务产品，但数据边界不同。

仓库记忆与业务 Store 的关系

仓库记忆使用 LangGraph Store，并通过 `StoreBackend` 暴露为虚拟文件。它不属于业务 Store。



数据归属原则：谁产生，谁负责恢复

当前项目最终采用下面的归属规则：

数据	产生者	恢复来源	是否给前端正文
用户消息	LangGraph / DeepAgents 会话	checkpoint	是
assistant 正文	LangGraph / DeepAgents 会话	checkpoint	是
当前运行中的 token	DeepAgents v3 event	SSE 当前流	是，仅当前 run
任务计划	write_todos	当前 run_events，经 SSE 推送	是，作为任务计
PR 链接	Gitee 工具 / runtime	Store.threads	是，作为状态和
reviewer findings	reviewer 工具	Store.review_findings	不直接当聊天正用
仓库长期记忆	repo memory 更新逻辑	LangGraph StoreBackend	不直接给前端历

一句话：聊天正文只认 checkpoint；运行中过程看 SSE；业务数据留在 Store。

2、`graph.py`：Store 和 Checkpoint 的统一入口

核心函数：

函数	返回	作用
get_store()	LocalSqliteStore(STORE_DB_PATH)	获取业务 Store 单例
get_checkpointer()	make_checkpointer(CHECKPOINT_DB_PATH)	获取 LangGraph SQ
build_agent(thread_id, task_kind)	DeepAgent	兼容入口，内部调用 agent.server.get_a

为什么这里要提供统一入口?

1. 避免各个模块自己创建数据库路径。
2. 确保业务 Store 和 checkpoint 是两套清晰的生命周期。
3. 让 `server.py` 创建 DeepAgent 时可以稳定拿到同一个 checkpointer。

两个 Store 的本质区别

函数	返回对象	数据库文件	谁定义表结构	
get_store()	项目自定义 LocalSqliteStore	store.sqlite	当前项目自己定义	业务数据: 射
get_langgraph_store()	LangGraph SqliteStore	langgraph_store.sqlite	LangGraph 框架定义	DeepAgen

所以它们名字都叫 Store，但不是同一个层级。



为什么不能只用 get_langgraph_store()?

不合适。

LangGraph Store 是通用 KV / document store，不适合做你现在这些业务查询:

- 最近 50 个任务;
- 某个任务的运行状态;
- 某个任务的 PR 地址;
- 某个任务的 reviewer findings;
- 某个 Gitee 仓库对应哪个本地目录;
- 前端 SSE 如何读取 run_events。

这些都是明确关系型业务数据，用你自己的 `LocalSqliteStore` 更清晰。

3、`persistence.py`：真正创建Checkpoint 和Store

核心函数：

代码块

```
def make_checkpointer(db_path: Path) -> SqliteSaver:
    db_path.parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(str(db_path), check_same_thread=False)
    saver = SqliteSaver(conn)
    saver.setup()
    return saver
```

关键点：

代码	说明
sqlite3.connect(...)	创建 SQLite 连接
check_same_thread=False	允许 checkpointer 被不同线程调用
SqliteSaver(conn)	使用 LangGraph 官方 SQLite checkpointer
saver.setup()	初始化 LangGraph checkpoint 所需表结构

课堂提醒：

> `checkpoints.sqlite` 的表结构由 LangGraph 管理。我们只需要知道它负责 Agent thread state 恢复。

4、`LocalSqliteStore` 业务数据管理

方法	作用
upsert_thread(...)	创建或更新任务摘要
get_thread(thread_id)	读取单个任务
list_threads(limit)	读取任务列表
update_thread_status(...)	更新任务状态、PR URL、分支名
record_run(...)	创建或更新 run 记录
add_run_event(...)	写入或更新运行事件
list_run_events(thread_id)	读取前端运行步骤
clear_run_events(thread_id)	新一轮运行前清理临时事件
finish_open_run_events(...)	任务结束时关闭残留 in_progress 事件
add_thread_message(...)	历史兼容写入方法，当前主流程尽量不依赖
list_thread_messages(thread_id)	历史兼容读取方法，不作为前端历史正文来源
delete_thread(thread_id)	删除业务 Store 中的 thread 及附属记录
add_finding(...) / list_findings(...)	写入和读取 reviewer findings

这里的一个重要设计是：很多写入都是“可重复写”的。

方法	幂等设计
upsert_thread	ON CONFLICT(thread_id) DO UPDATE
record_run	ON CONFLICT(run_id) DO UPDATE
add_run_event	ON CONFLICT(id) DO UPDATE
add_thread_message	历史兼容表仍保留幂等写法，但当前主流程不再依赖它恢复正文

这样做的原因是 Agent 运行过程中同一个步骤会先写 `in_progress`，完成后再用同一个 key 更新成 `completed` 或 `error`。

5、events.py：运行事件单独封装

位置：

agent/core/events.py

核心函数：

record_event(thread_id, key, title, kind, status, detail)

它会把事件 ID 拼成：

{thread_id}:{key}

然后调用：

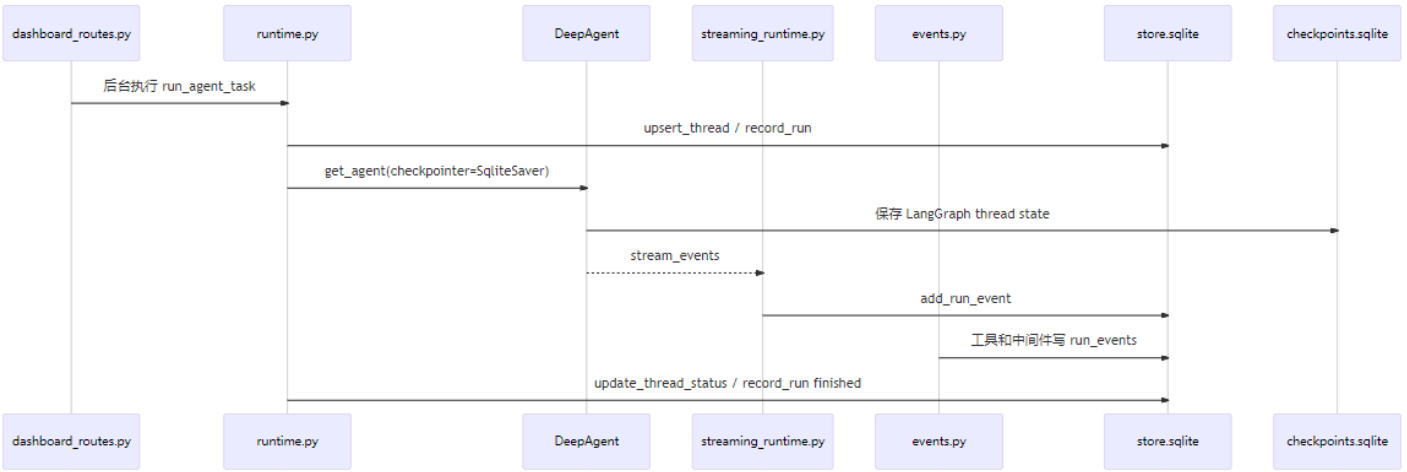
LocalSqliteStore.add_run_event(...)

为什么不直接在工具里导入 `graph.get_store()`？

> 因为 tools、middleware、runtime、graph 之间容易形成循环导入。`events.py` 用独立的 `_event_store` 延迟创建 Store，专门服务事件写入，降低模块耦合。

事件写入失败时，`record_event` 会捕获异常并记录日志，不中断真实 Agent 任务。原因是：

事件失败	**真实任务**
前端少一个展示步骤	不应该导致代码修改、搜索、PR 创建失败
属于可观察性问题	属于业务执行问题



第八章、引入长期记忆和Skills

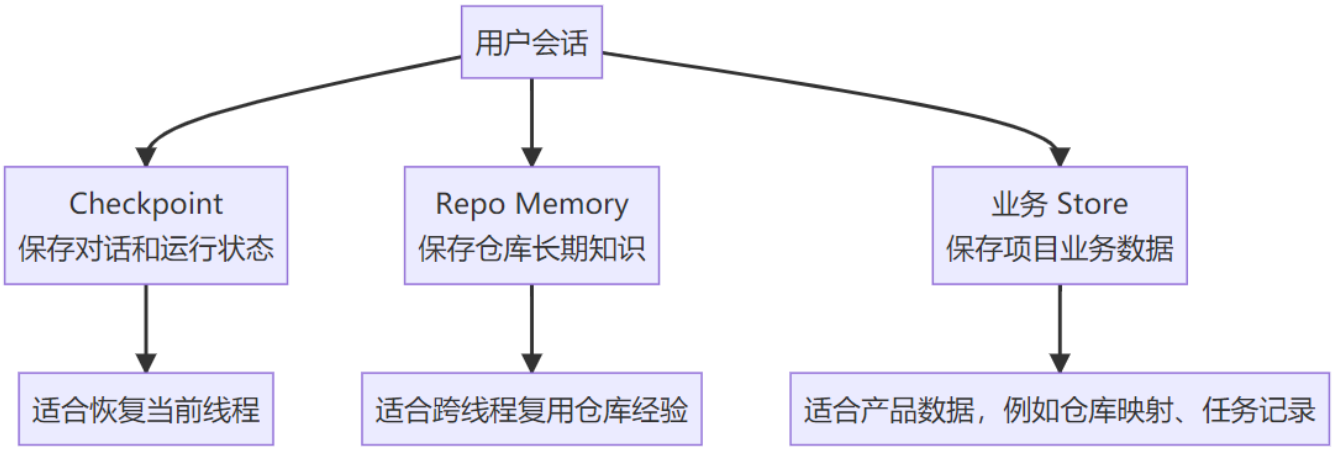
1 为什么需要长期记忆

Coding Agent 最浪费工具调用的地方，通常不是写代码本身，而是反复确认这些信息：

反复确认的信息	如果没有记忆会怎样
仓库目录结构	每轮都重新 list_files、读取 README、扫描模板目录
启动命令	每轮都猜 uvicorn、pytest、npm、yarn 怎么运行
测试命令	复杂项目里可能反复试错，消耗大量模型调用
关键模块	不知道路由、数据库、鉴权、模板分别在哪些文件里
已完成的分支和 PR	重复创建分支或重复创建 Pull Request
最近一次失败原因	测试失败后下一轮又走同样错误路径

仓库级记忆的目标是：让 Agent 下一轮进入同一个仓库时，先拿到“这个仓库怎么工作”的摘要。

2 当前项目的记忆分层



类型	当前用途	生命周期
Checkpoint	保存 LangGraph 对话状态、工具消息、运行过程	跟会话线程绑定
Repo Memory	保存某个 Gitee 仓库的长期开发知识	跟仓库绑定，可跨会话复用
业务 Store	保存 UI、任务、仓库映射等应用数据	跟系统业务绑定

注意：仓库级记忆不是把所有聊天记录复制进去，而是提炼稳定结论。

当前项目使用 DeepAgents 官方推荐的 `StoreBackend` 思路，让记忆以虚拟文件方式暴露给 Agent。

项	当前设计
Agent 可见路径	/memories/{owner}/{repo}.md
Store namespace	("lx-aicoding", "repo-memory", owner, repo)
Store key	repo.md
物理数据库	data/langgraph_store.sqlite
初始化时机	项目创建或首次解析仓库时初始化
更新时机	coding / planning / analysis / review 任务结束后总结并写回

这里不再额外维护一张“记忆索引表”。原因是当前项目已经有“Gitee 仓库地址 -> 本地项目目录”的映射逻辑，仓库名本身就能作为唯一 namespace 的一部分。

3、仓库记忆应该写什么

适合写入	示例
项目技术栈	FastAPI、SQLite、pytest、Jinja2 模板
关键入口	main.py 是后端入口，templates/ 存放页面模板
启动方式	python -m uvicorn main:app --reload
测试方式	python -m pytest 或项目实际测试命令
数据存储	使用 db.py 封装 SQLite，数据库文件位置
重要约束	只允许 Gitee，用户输出必须中文
最近结论	已经存在用户模块、部门模块、权限模块
PR 信息	最近分支和 Pull Request 地址

不适合写入：

不适合写入	原因
.env 内容	敏感信息，不能持久化到记忆
token、私钥、密码	安全风险
大段源码	会污染上下文，占用 token
一次性推理过程	下轮通常不需要复用
错误的中间输出	会误导后续任务

4、仓库记忆的流程

概念	当前项目中的值	作用
Agent 可见路径	/memories/{owner}/{repo}.md	Agent 看到和读写的虚拟记忆文
Store 内部 key	/{owner}/{repo}.md	LangGraph Store 真实保存的 k
namespace	("lx-aicoding", "repo-memory", owner, repo)	每个仓库独立隔离记忆
backend route	/memories/ -> StoreBackend	让 Agent 通过文件工具访问 Sto

为什么 Agent 可见路径和 Store key 不一样?

因为当前 `CompositeBackend` 配置是:

代码块

```
routes={
    "/memories/": StoreBackend(...)
}
```

DeepAgents 会把 `/memories/` 前缀剥离后再交给 `StoreBackend`。
所以:

代码块

```
Agent 读写: /memories/msb-goldbin/abc-test.md
Store 实际 key: /msb-goldbin/abc-test.md
```

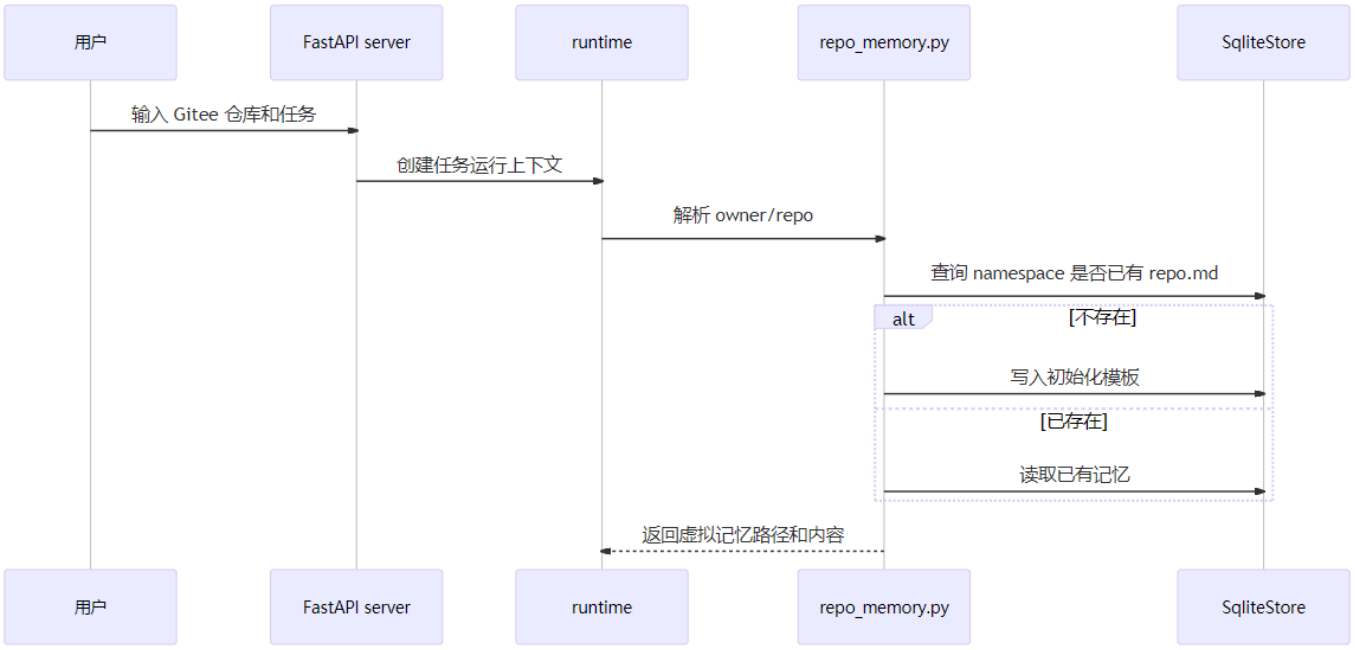
初始化流程

初始化发生在 `runtime.py` 准备仓库上下文时:

代码块

```
用户给仓库地址
-> 解析 owner/repo
-> 找到 projects 下对应目录
-> 调用 ensure_repo_memory_initialized()
-> 如果 Store 里没有记忆文件，就创建初始模板
```

初始化只发生一次。已有记忆不会被覆盖。



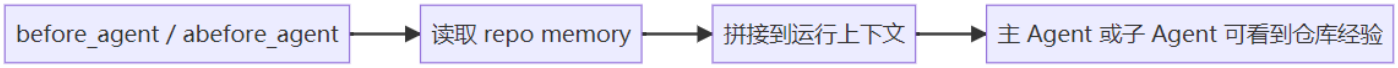
初始化内容应该是“空模板 + 结构化标题”，而不是伪造项目事实。

当前 `store` 表里的数据，主要都是仓库记忆：

prefix	key	说明
lx-aicoding.repo-memory.msb-goldbin.abc-test	/msb-goldbin/abc-test.md	新版 abc-test 仓库记忆
lx-aicoding.repo-memory.msb-goldbin.ai_coding	/msb-goldbin/ai_coding.md	新版 ai_coding 仓库记忆
lx-aicoding.repo-memory.msb-goldbin.abc-test	/memories/repo.md	更早版本旧 key
lx-aicoding.repo-memory.msb-goldbin.ai_coding	/memories/repo.md	更早版本旧 key

仓库记忆如何注入 Agent

当前项目通过 `ContextInjectionMiddleware` 在 Agent 执行前读取仓库级记忆，并把内容加入系统上下文。



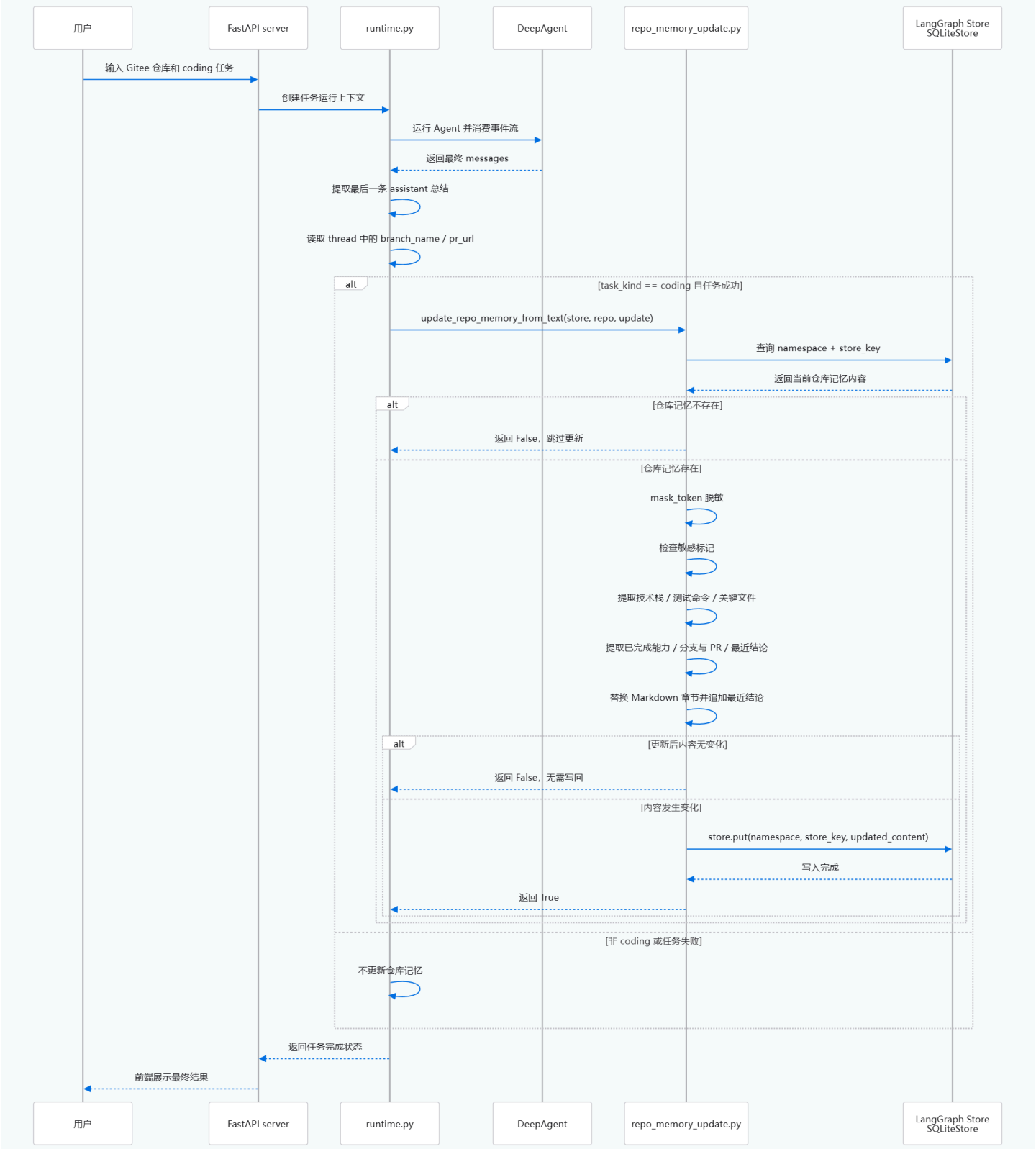
仓库记忆什么时候更新

只有 coding 任务成功后更新仓库记忆。

更新位置	作用
runtime.py	在任务成功结束后，根据最终回答和上下文主动更新记忆，这是主要路径的
MemoryUpdateMiddleware	在 Agent 完成后兜底更新，避免未来运行路径调整后忘记写回。写了代码

coding 成功后，`runtime.py` 会：

```
1. 提取最终 assistant 总结
2. 读取当前 thread 的 branch_name / pr_url
3. 调用 update_repo_memory_from_text()
4. repo_memory_update.py 提炼稳定事实
5. 写回 Store 中的 /owner/repo.md
```



repo_memory.py 和 repo_memory_update.py 的分工

文件	职责
repo_memory.py	负责路径、namespace、backend、初始化模板
repo_memory_update.py	负责从最终回答中提炼技术栈、测试命令、关键文件、最近结论
runtime.py	决定什么时候初始化、什么时候更新
server.py	创建 Agent 时挂载记忆 backend
context_injection.py	把记忆内容注入 Agent 上下文

5、skills技能

Skills 是 DeepAgents 原生机制，用来告诉 Agent 某类任务应该怎么做。它不是代码工具，而是“任务操作手册”。

Skills 不是 Python 函数，也不是工具。它们更像“给 Agent 看的操作手册”。例如：

- 首次分析仓库时，先看 README，再看目录，再找测试命令。
- 编码实施时，先确认方案，再小步修改，再跑测试。
- 代码审查时，先看 diff，再看风险，再给出结论。

这类流程如果全部写进主 Prompt，会让 Prompt 越来越大。放进 skill 后，Agent 可以按需读取。

当前项目的 skills 目录分两层：

目录	作用
agent/skills	项目源码中的技能模板，方便版本管理
E:\ai_workspace\skills	Agent 运行时真正读取的 skills 目录

每次修改源码里的 skill 后，都要同步到 `E:\ai_workspace\skills`，否则运行中的 Agent 读取不到最新版本。

当前可用 Skills

Skill	作用
repo-bootstrap-analysis	仓库首次分析、目录识别、Gitee 仓库准备
ai-coding-implementation	编码实施流程
code-review	Gitee PR 和本地 diff 审查流程

第九章、创建DeepAgent工厂

1、前置知识点

普通 LangChain Agent 的核心组件是：

Agent = Model + Tools + System Prompt + Middleware

DeepAgents 在这个基础上增加了：

Filesystem Backend + Permissions + Skills + Memory + Subagents + 中间件，Tools+ 模型

这一节要先区分三个概念：

概念	说明	当前项目中的承载位置
Agent 实例	create_deep_agent(...) 返回的可运行图对象	agent/server.py:get_agent 每次按配置构建
运行状态	对话、图执行状态、checkpoint	agent/core/graph.py:get_checkpointer()
工作区资源	本地文件系统、命令执行、Gitee 凭据注入	agent/backends/local_shell.py，按 thread 缓存

注意：**非单例 Agent 并不等于状态丢失**。状态应该由 checkpointer、业务 Store、文件后端承担，而不是隐式藏在一个长期存活的 Python 对象里。

2、create_deep_agent 的组成

参数	当前项目传入	作用
model	make_main_model()	主 Agent 模型
tools	web_search、fetch_url、Gitee PR、reviewer tools	自定义外部能力
system_prompt	get_system_prompt(task_kind)	任务规则
subagents	general-purpose subagent	委派分析任务
backend	backend_factory	DeepAgents 文件和命令后端
permissions	_agent_filesystem_permissions()	文件读写权限
middleware	sanitize、model call limit、tool error	安全、限制、恢复
skills	["/skills/"]	技能目录
checkpointer	get_checkpointer()	LangGraph 状态恢复

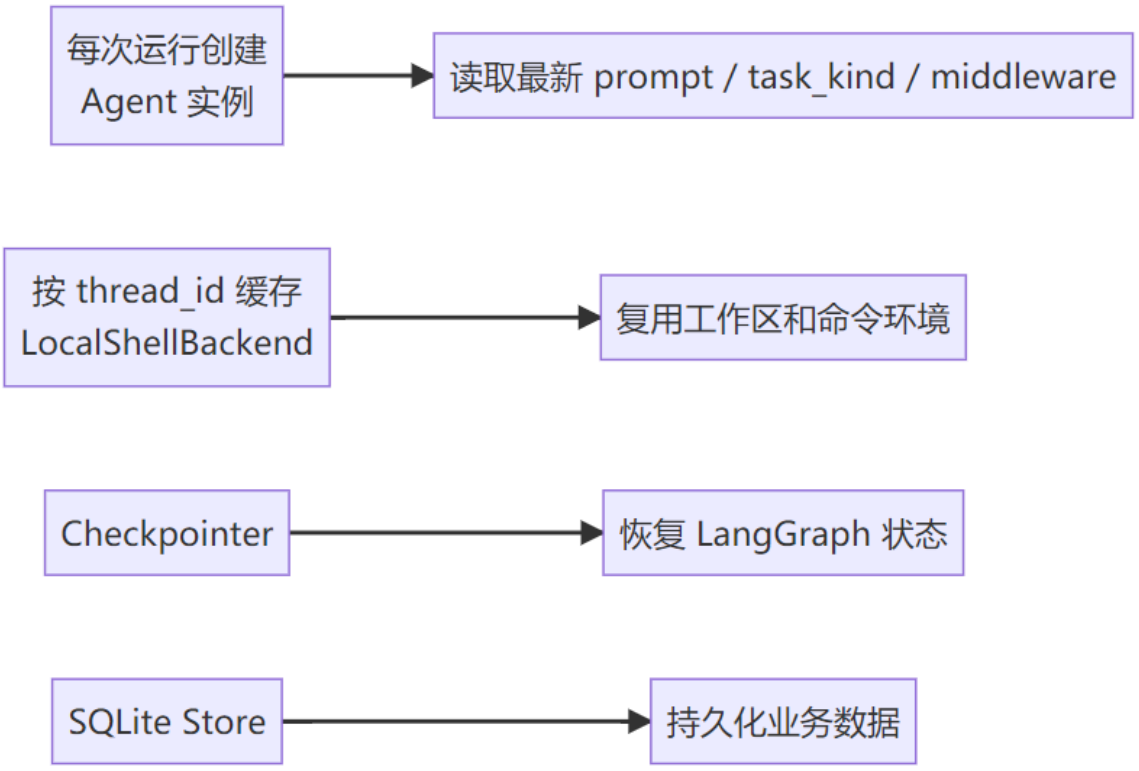
3、为什么 Agent 实例采用非单例

当前项目不是把 `create_deep_agent(...)` 的结果做成全局单例，而是在 `get_agent(config)` 中按运行配置创建 Agent 实例。

核心原因如下：

原因	说明
任务类型不同	coding、planning、analysis、qa 会拿到不同的 system_prompt
会话上下文不同	每个 thread_id 对应不同工作区、checkpoint 和业务记录
权限可能变化	不同任务或未来不同用户可以使用不同 permissions
长期记忆要新鲜	ContextInjectionMiddleware 会读取当前仓库级记忆，重建 Agent 可以拿到最新_文
避免跨会话污染	单例对象容易把工具状态、middleware 状态、config 状态混在一起
方便调试和热更新	**修改 Prompt、Skills、权限后，新运行可以直接使用新配置（Agent平台）**

这套设计的关键不是“所有东西都重新创建”，而是把生命周期拆开：



因此，Agent 实例是轻生命周期；backend、checkpoint、store 才是承载长期状态的部分。

4、企业项目中 Agent 实例的常见管理方案

`create_deep_agent(...)` 执行后会得到一个 Agent 实例。企业项目里通常有几种管理方式：

方案	做法	优势	风险或代价
全局单例 Agent	服务启动时创建一个 Agent，全局复用	实现简单，创建开销低	配置容易陈旧，权 户上下文容易串线
按会话级别缓存 Agent	thread_id -> Agent 缓存	会话隔离更好，复用成本较低	缓存淘汰复杂，Pr Tools 更新后可能不
按请求创建 Agent	每次运行重新 create_deep_agent	配置最新，隔离清晰，调试简单	每次有一定构建开
工厂 + 资源池	Agent 按请求创建，模型、backend、store、checkpoint 由容器管理	工程边界清晰，利于扩展和观测	需要更完整的生命 理
远程图服务托管	由 LangGraph Server 或平台托管 graph 生命周期	服务化能力强，便于水平扩容	运维和平台依赖更

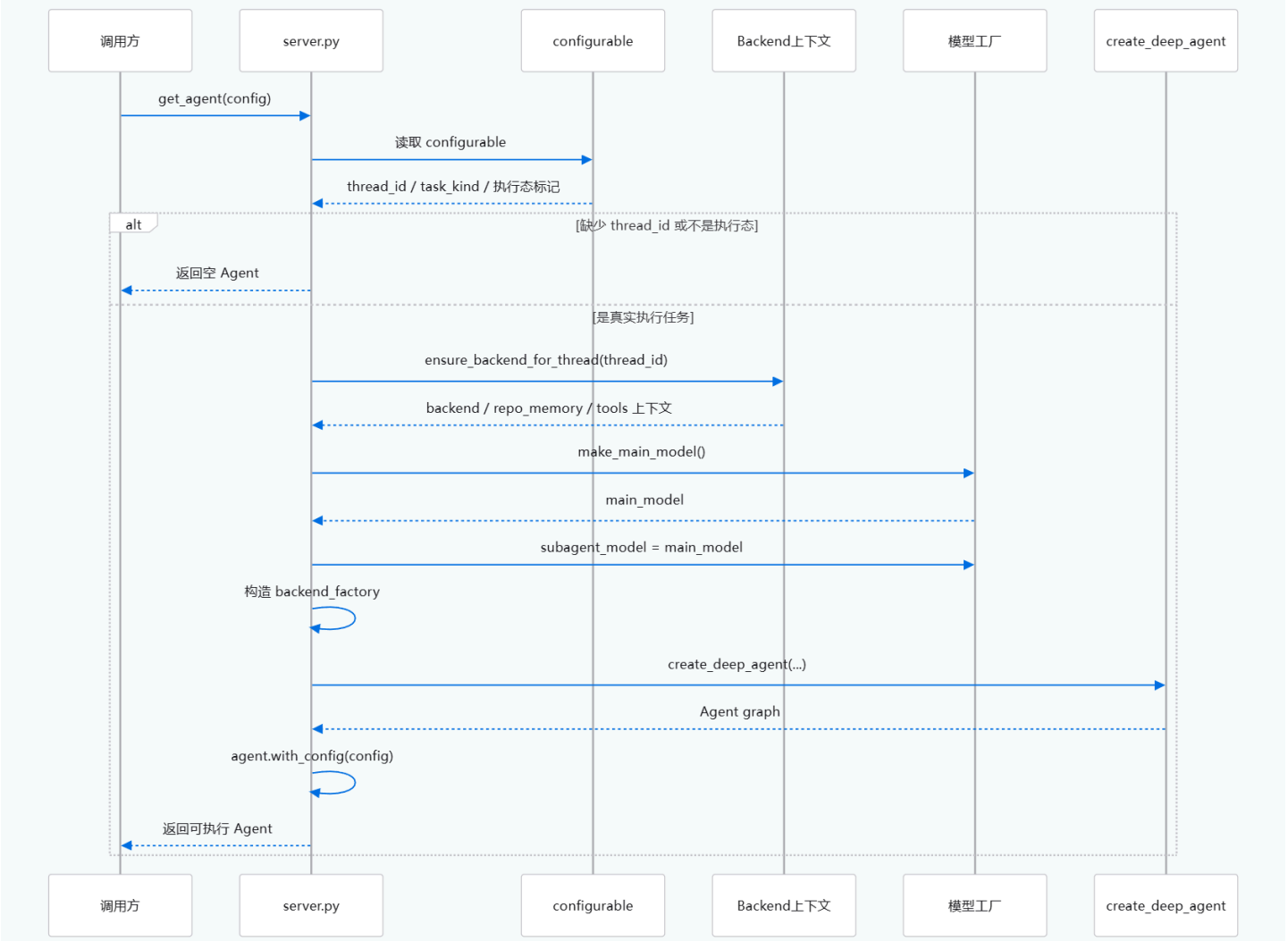
当前项目采用的是接近“按请求创建 Agent + 会话级 backend 缓存 + 持久化 checkpoint/store”的混合方案。

当前项目设计	具体体现	好处
Agent 不做全局单例	get_agent(config) 中调用 create_deep_agent(...)	每次运行拿到最新
backend 按线程复用	_BACKENDS: dict[str, LocalShellBackend]	同一会话复用本地 射
checkpoint 独立持久	checkpointer=get_checkpointer()	Agent 对象销毁后
业务数据进入 SQLite	agent/store/sqlite_store.py	run、message、
非执行态返回空 Agent	没有 thread_id 或 __is_for_execution__ 时返回轻量 Agent	避免图探测阶段误

总结：

> 当前项目不依赖“长寿命 Agent 对象”保存状态，而是让 Agent 成为一次运行的装配结果，让后端、checkpoint 和数据库承担可恢复状态。这比单例更适合企业项目中的多线程、多任务、多权限场景。

5、get_agent 的构建步骤



对应代码要点：

代码位置	作用
graph_loaded_for_execution(config)	判断当前是不是“真实执行态”
ensure_backend_for_thread(thread_id)	为当前会话创建或复用 LocalShellBackend
_task_kind_from_config(configurable)	从配置读取任务类型
_agent_filesystem_permissions()	生成主 Agent 文件权限
_general_purpose_subagent(subagent_model)	生成通用分析子 Agent
create_deep_agent(...)	组装模型、工具、提示词、后端、权限、中间件、ski checkpoint

recursion_limit 是：LangGraph 图的最大执行步数 / super-step。

可能的图执行过程：

step	节点	做了什么
1	agent	模型读取任务，决定查看文件
2	tools	执行 ls
3	agent	模型看到目录，决定读 main.py
4	tools	执行 read_file
5	agent	模型决定修改代码
6	tools	执行 edit_file
7	agent	模型决定运行测试
8	tools	执行 execute pytest
9	agent	模型看到测试失败，决定修复
10	tools	执行 edit_file
11	agent	模型再次运行测试
12	tools	执行 execute pytest
13	agent	模型总结完成
14	END	图结束

这里大概用了 14 个图步数。

如果 `recursion_limit=10`，任务可能还没完成就被 LangGraph 中断。

6、SubAgent的作用

当前项目在 `agent/server.py` 中定义了一个通用子 Agent：

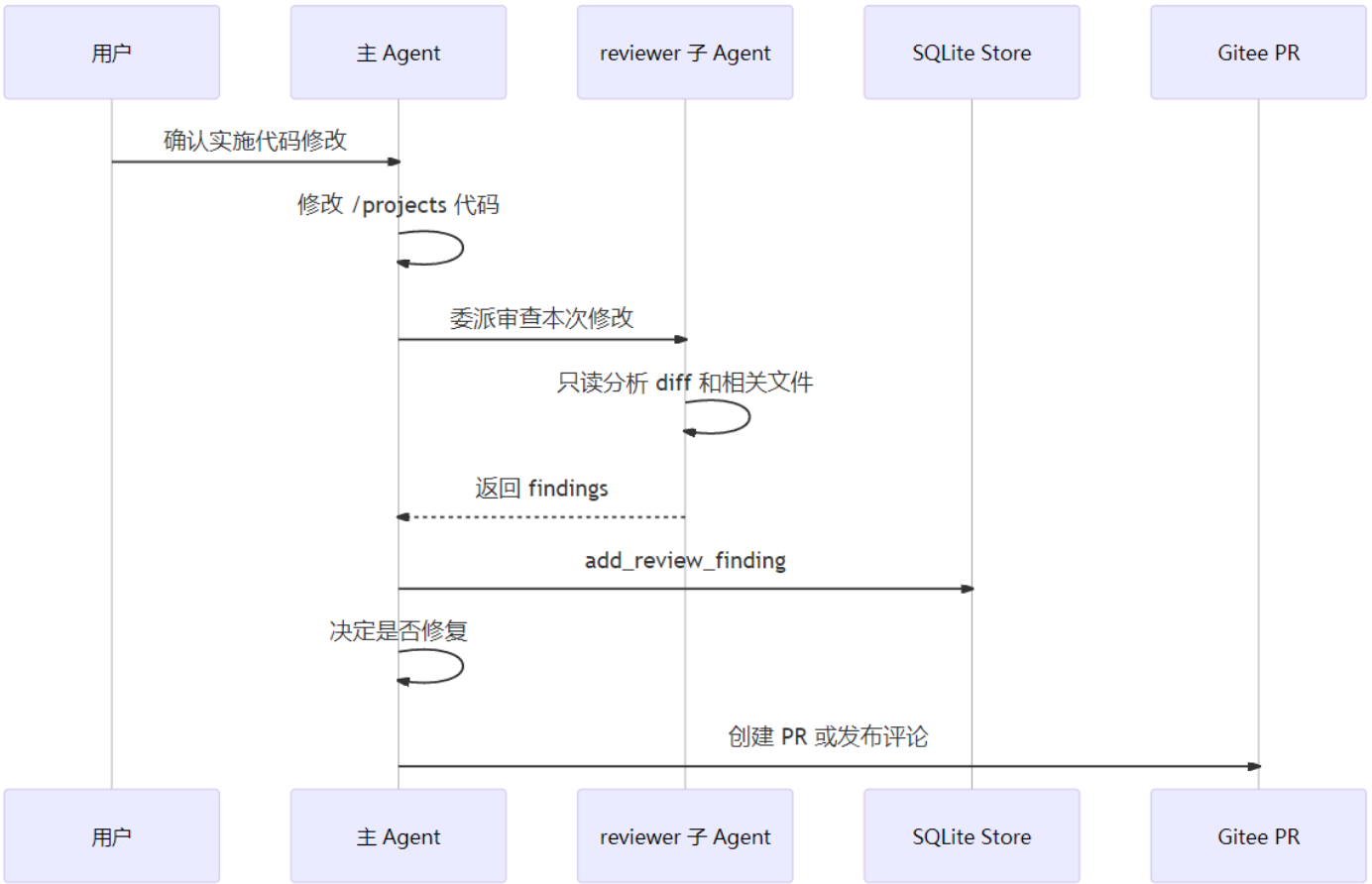
当前 `_general_purpose_subagent()` 是“显式覆盖默认 general-purpose 子 Agent”，目的是加权限控制。

它的定位不是“另一个可以随意写代码的 Agent”，而是主 Agent 的分析助手。

维度	主 Agent	通用子 Agent
职责	最终理解用户需求、修改代码、调用 PR 工具、输出结论	阅读、分析、总结、提出建议
写源码权限	可以写 /projects/**	不允许写 /projects/**
适合任务	实施方案、修复代码、创建 PR	代码结构分析、大范围检索、对比
责任边界	对最终行为负责	给主 Agent 提供辅助结论

为什么要引入子 Agent：

1. **降低主 Agent 上下文压力**：大仓库分析可以委派给子 Agent，主 Agent 聚焦决策和执行。
2. **隔离权限风险**：子 Agent 默认不能改源码，适合作为只读分析角色。
3. **形成专业分工**：未来可以继续增加 reviewer、test-planner、security-auditor 等专用子 Agent。
4. **让复杂任务更可控**：主 Agent 负责调度，子 Agent 负责局部问题，边界更清晰。



7、DeepAgents permissions

DeepAgents permissions 是传给 `create_deep_agent(..., permissions=[...])` 的声明式文件权限规则。它主要控制 DeepAgents 原生文件工具能对哪些虚拟路径执行读写操作。

官方概念可以简化为：

字段	含义
operations	控制 read、write 或两者
paths	虚拟路径 glob，例如 /projects/**
mode	allow 或 deny
匹配顺序	按规则顺序匹配，先匹配先生效

重点：

> permissions 管的是 DeepAgents 文件工具的可见能力；backend 管的是最终本地路径和命令执行安全。两者不能互相替代。

当前项目的主 Agent permissions

当前主 Agent 权限在 agent/server.py 的 `_agent_filesystem_permissions()` 中定义。

规则顺序	operations	paths	mode
1	read	/projects/**、/skills/**、/policies/**、/reviews/**、/runtimes/**、/logs/**、/tmp/**	allow
2	write	/projects/**、/reviews/**、/tmp/**	allow
3	write	/skills/**、/policies/**、/runtimes/**、/logs/**	deny
4	read/write	/**	deny

permissions 与 backend 的边界

能力	DeepAgents permissions	LocalShellBackend
控制 read_file/write_file/edit_file 能否读写某路径	主要负责	最终兜底
控制虚拟路径映射到哪个 Windows 目录	不负责	负责
控制 execute 命令能否访问系统路径	不足以负责	必须负责
防止写 .secrets	可以通过 deny 辅助	必须硬防护
Gitee token 注入和脱敏	不负责	负责
旧工具兼容	不负责	负责

第十章、Reviewer子Agent

所有的主和子Agent共享哪些： 文件后端，checkpointer（对象），store（对象），config

哪些单独的：工具，中间件，权限、上下文（state），skills、提示词、模型，

`code_reviewer` 是 LX-AICODING 中专门负责代码审查的 SubAgent。

它的核心职责不是写代码，而是围绕 Gitee Pull Request 或本地分支 diff 做专业审查：

- 读取 Gitee PR 上下文；
- 读取审查规则；
- 读取本地 diff；
- 识别真实风险；
- 校验 finding 文件和行号；
- 记录结构化 review findings；
- 输出中文审查报告；
- 在用户确认后，可由主 Agent 将审查报告发布到 Gitee PR 评论区。

它和主 Agent 的职责边界如下：

角色	主要职责	是否修改代码
主 Agent	需求理解、技术方案、代码实现、测试、提交、PR、修复闭环	是
code_reviewer 子 Agent	PR / diff 审查、记录 findings、输出审查报告	否

这个设计遵循一个原则：

实现代码和审查代码分离

也就是说，主 Agent 可以写代码，但代码审查要委派给一个只读的专业子 Agent，避免“自己写、自己审、自己放过”的职责混杂。

如果都由主 Agent 完成，会出现几个问题：

问题	影响
职责混杂	主 Agent 容易边审查边改代码，违背用户“只 review”的要求
审查不客观	模型容易解释自己刚生成的代码，而不是严格发现风险
权限过大	review 阶段本应只读，却可能误触发写文件、commit、push
结果不可复用	自然语言报告如果不结构化，后续修复方案难以精确引用
规则不稳定	主 Agent 的开发 prompt 和 reviewer 的审查 prompt 容易互相干扰

使用 SubAgent 后，可以把能力拆开：

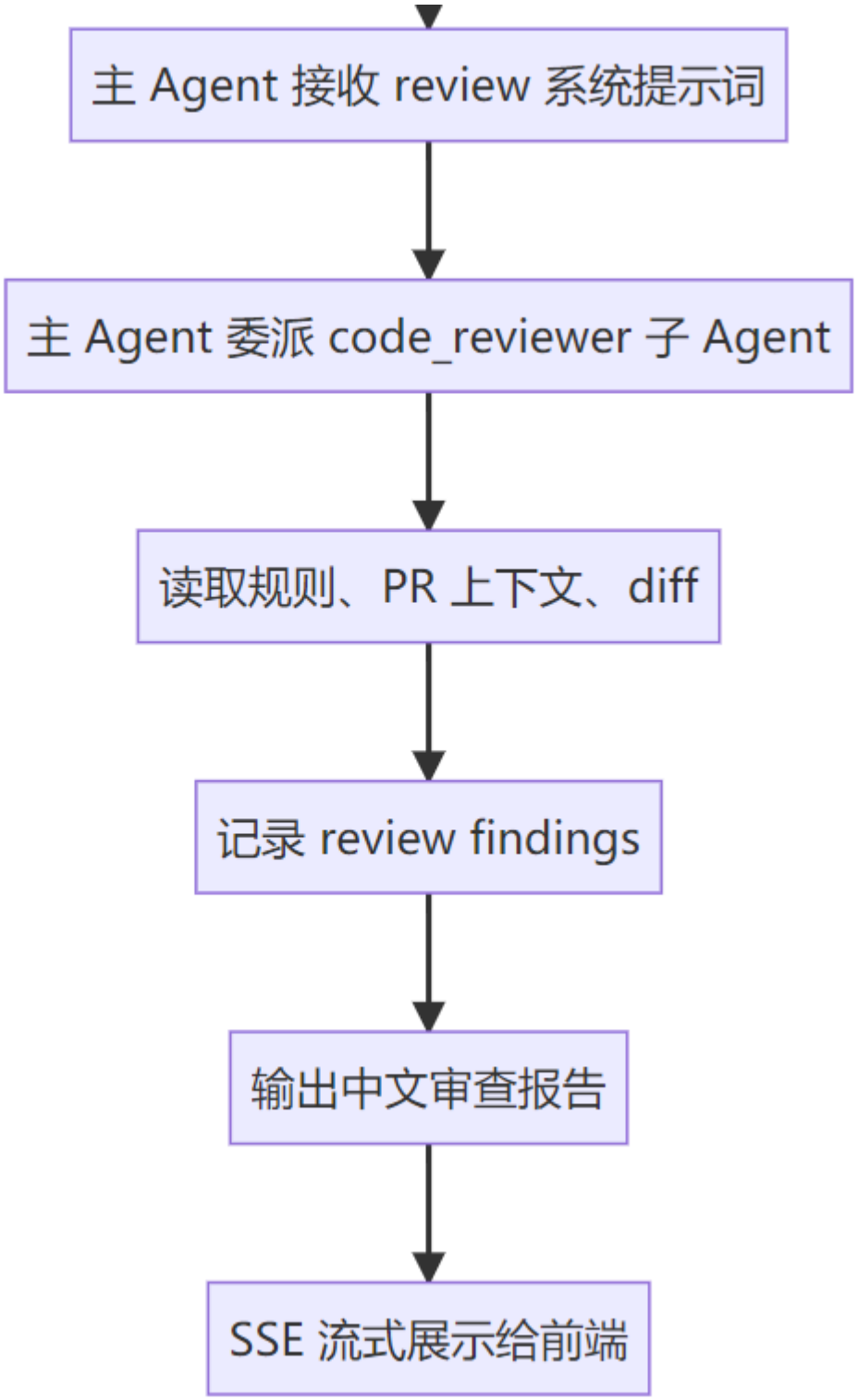
主 Agent：流程编排、任务分类、用户确认、代码修复

Reviewer 子 Agent：只读审查、风险判断、findings 记录、审查报告

这样更符合企业中的角色分工：

开发者 != Reviewer

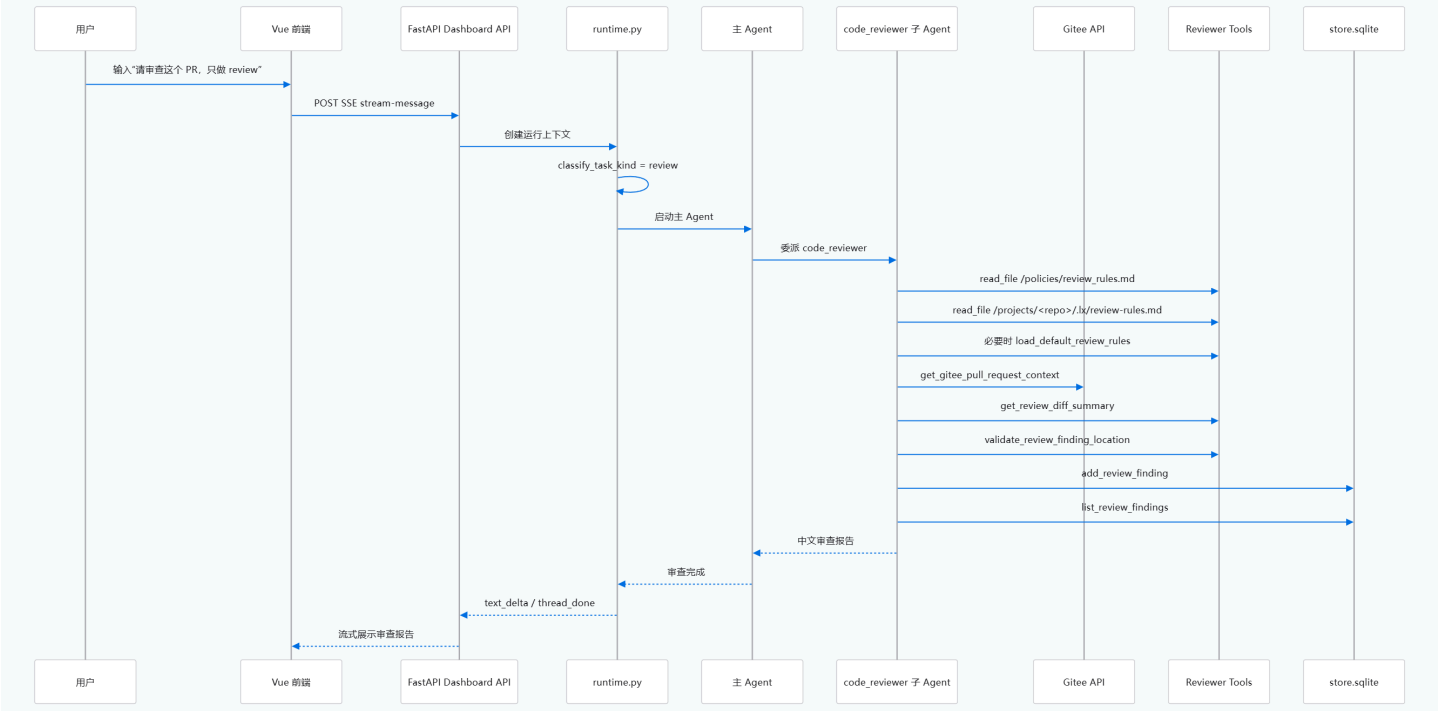
1、Reviewer的流程



它的核心流程是：

1. 按 `code-review` skill 读取审查规则；
2. 如果用户提供 Gitee PR，读取 PR 详情、提交、文件、评论；
3. 获取本地 diff 摘要和变更行号；
4. 只记录会导致真实风险的问题；
5. 校验 finding 位置；
6. 使用 `add_review_finding` 保存结构化问题；
7. 使用 `list_review_findings` 汇总；

8. 输出中文审查报告；
9. 不修改文件、不提交、不 push、不创建 Pull Request。



2、Reviewer 子 Agent 的工具和Skill

Reviewer 子 Agent 单独挂载了如下工具。

工具	来源文件	作用
get_gitee_pull_request_context	agent/tools/gitee_tools.py	读取 Gitee PR 标题、描述、
load_default_review_rules	agent/tools/reviewer_tools.py	读取项目内置默认审查规则
get_review_diff_summary	agent/tools/reviewer_tools.py	获取本地 diff 摘要和变更行
validate_review_finding_location	agent/tools/reviewer_tools.py	校验 finding 是否落在真实
add_review_finding	agent/tools/reviewer_tools.py	把审查发现写入业务 Store
list_review_findings	agent/tools/reviewer_tools.py	汇总当前 thread 的审查发

这三个红色工具分别解决 reviewer 子 Agent 的三个关键问题：**审查对象是什么、改了什么、发现的位置是否可信**。

`get_gitee_pull_request_context`：获取 PR 上下文
它负责从 Gitee 读取 PR 信息，例如标题、描述、源分支、目标分支、提交、变更文件、评论等。
Reviewer 子 Agent 不能只看本地 diff，因为用户经常输入的是“审查这个 PR 地址”。它需要先知道这个 PR 对应哪个仓库、哪个分支、哪些文件发生了变化。

`get_review_diff_summary`：获取 diff 摘要

它负责读取本地分支和目标分支之间的 diff，整理出“哪些文件改了、哪些行是新增/删除、变更范围在哪里”。
代码审查不能只凭 PR 标题或文件名判断风险，必须看实际 diff。这个工具给 reviewer 一个可审查的变更范围，避免它审查整个仓库，浪费调用次数，也避免跑偏。

`validate_review_finding_location`：校验 finding 位置
它负责确认 reviewer 发现的问题是否落在本次 diff 的有效文件和行号范围内。
比如 reviewer 说 `main.py:89` 有问题，这个工具会检查这行是否真的属于本次 PR 的变更范围。这样可以减少“幻觉式审查”：模型指出一个不存在的行号、旧代码行、或者不属于本次 PR 的问题。

Reviewer 子 Agent 挂载了：

```
代码块
1   /skills/code-review/SKILL.md
```

该 skill 定义了审查方法论，包括：

- 角色定位；
- 输出语言；
- 审查输入；
- 规则读取顺序；
- PR 上下文读取；
- diff 读取；
- finding 记录；
- 行号校验；
- 审查报告格式；
- 禁止事项。

3、Review findings 存在哪里

Reviewer 子 Agent 发现的问题会通过 `add_review_finding` 写入业务 Store。

数据库位置：

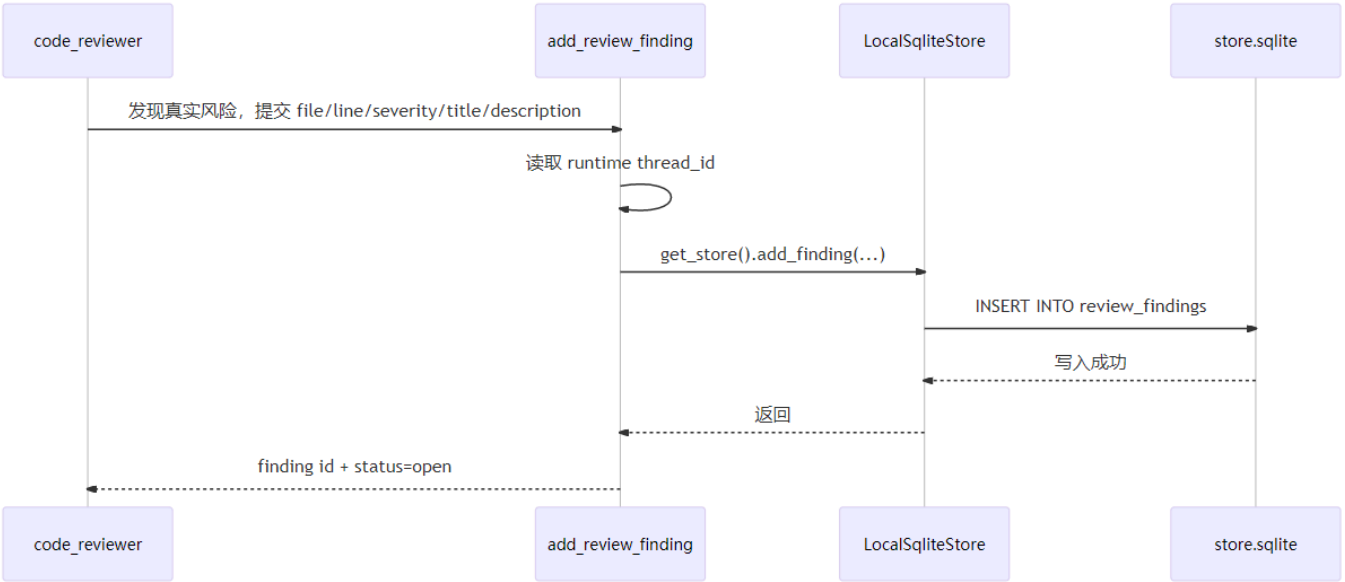
表名：

review_findings

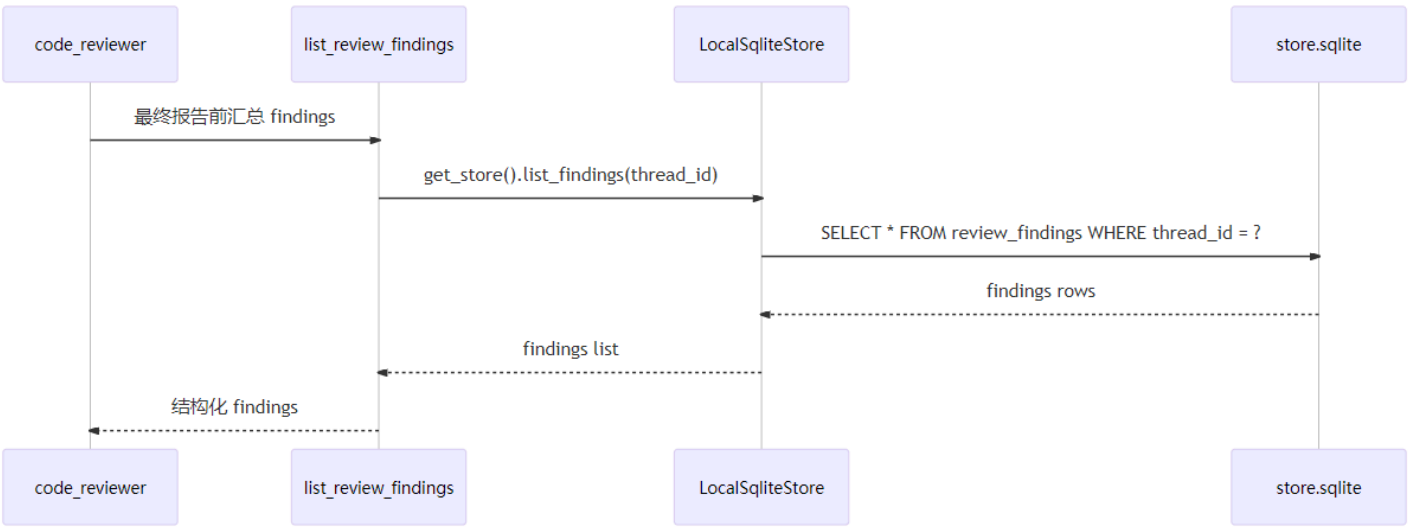
表字段包括：

字段	说明
id	finding id，例如 finding-xxxx
thread_id	所属会话 / 任务
file	问题文件
line	问题行号，可为空
severity	严重级别
title	问题标题
description	风险说明和建议
status	状态，默认 open
created_at	创建时间
updated_at	更新时间

写入链路：



读取链路：

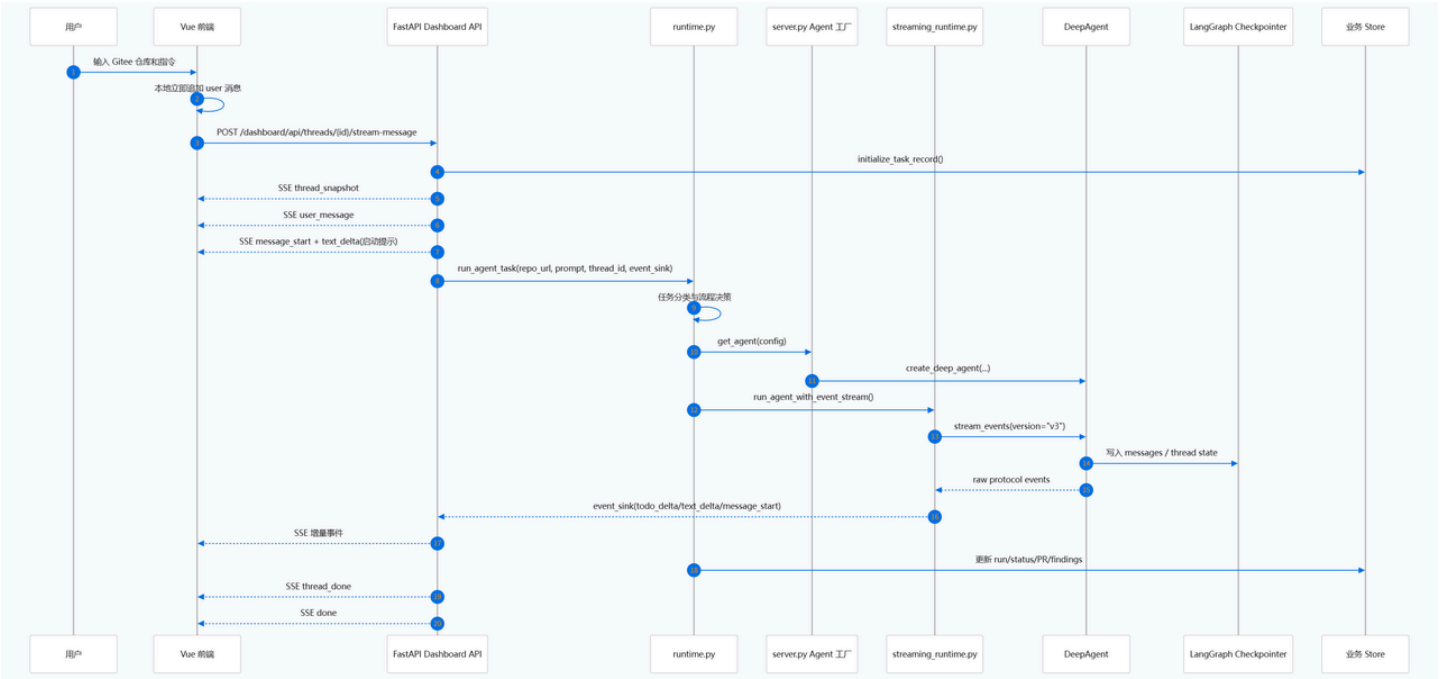


注意：

review findings 属于业务 Store，不属于 checkpoint。

checkpoint 保存的是 Agent 会话状态和消息历史；Store 保存的是结构化业务数据。

第十一章、FastAPI的后端接口



1、FastAPI介绍

FastAPI 是一个基于 Python 类型注解构建 Web API 的框架。它底层依赖 Starlette 处理 ASGI Web 服务能力，依赖 Pydantic 做请求体和响应数据校验。

在 LX-AICODING项目 中，FastAPI 的定位不是“业务系统框架”，而是 **Agent 产品服务外壳**。它负责接收前端请求、校验参数、启动 Agent 任务、返回实时流、暴露历史查询和删除接口。

FastAPI 在当前项目里的作用可以拆成五层：

层次	FastAPI 提供的能力	当前项目中的体现
路由层	用 @router.get/post/delete 定义 HTTP 接口	dashboard_me、dashbc dashboard_stream_exist
参数校验层	用 Pydantic BaseModel 校验 JSON 请求体	DashboardThreadMessa
响应层	返回普通 JSON 或流式响应	普通接口返回 dict/list，流
异常层	抛出 HTTPException 生成标准 HTTP 错误	仓库地址不合法返回 400，
ASGI 服务层	由 Uvicorn 运行异步 Web 应用	start_all.py 启动后端 Uvi

简化后的 FastAPI 写法如下：

代码块

```python
from fastapi import APIRouter, HTTPException
from fastapi.responses import StreamingResponse
from pydantic import BaseModel

router = APIRouter(prefix="/dashboard/api")

class MessageRequest(BaseModel):
    content: str
    repo: str | None = None

@router.post("/threads/{thread_id}/stream-message")
async def stream_message(thread_id: str, body: MessageRequest):
    if not body.content.strip():
        raise HTTPException(status_code=400, detail="content 不能为空")
    return StreamingResponse(event_iter(), media_type="text/event-stream")
```

这里有几个初学者需要理解的点：

1. `APIRouter` 用于把一组接口集中管理 当前项目把 Dashboard 相关接口都放在 `dashboard_router` 下，最后由应用入口统一挂载。
2. `BaseModel` 让请求体变成结构化对象 前端传 JSON，FastAPI 自动解析成 `DashboardThreadMessageRequest`，代码里可以直接访问 `body.content` 和 `body.repo`。
3. `async def` 不等于所有代码都自动并发 FastAPI 可以运行异步接口，但如果接口内部直接执行同步阻塞的 Agent 任务，仍然会卡住事件循环。所以当前项目额外使用后台线程运行 Agent。
4. `StreamingResponse` 是 FastAPI 对流式 HTTP 响应的封装 它不会一次性返回完整 JSON，而是不断把 `event_iter()` 产生的内容推给浏览器。

5. FastAPI 不知道 DeepAgents 的内部逻辑 它只负责把 Web 请求转成 Python 函数调用。Agent 的任务分类、模型、工具、中间件、checkpoint 都在其它模块中完成。

2、SSE流式输出

SSE 全称是 Server-Sent Events，中文可以理解为“服务器推送事件”。它建立在普通 HTTP 连接之上，**服务端可以不断向浏览器发送文本事件**。

标准 SSE 格式是：

```
代码块
1   event: text_delta
2   data: {"content": "正在生成方案..."}
3
4   event: done
5   data: {}
```

每个事件之间用空行分隔。浏览器或前端解析器按顺序读取这些事件，然后更新页面。

SSE 的核心特点：

特点	说明
单向通信	服务端持续向客户端推送，客户端不能在同一条连接上反向发送消息
基于 HTTP	不需要升级协议，部署和代理兼容性较好
文本协议	默认传输 UTF-8 文本，很适合 token、日志、状态、JSON 事件
自动顺序	同一个连接里的事件天然按发送顺序到达
实现简单	后端 yield 文本，前端读取流即可

在 Agent 项目中，SSE 很适合这些场景：

- 模型 token 流式输出；
- 任务计划 `todo_delta`；
- 工具执行状态；
- 子 Agent 生命周期；
- 最终完成或错误事件。

当前项目使用的是 **POST SSE**，不是浏览器原生 `EventSource` 的 GET SSE。原因是本项目每次运行都需要提交用户输入、仓库地址、模型配置等请求体，用 POST 更合适。

3、WebSocket方案

WebSocket 是一种浏览器和服务器之间的双向长连接协议。连接建立后，客户端和服务端都可以随时发送消息。

WebSocket 的典型连接过程是：

```
代码块
1  浏览器 HTTP 请求
2    -> 请求升级协议 Upgrade: websocket
3    -> 服务端接受升级
4    -> 双方进入 WebSocket 双向通信
```

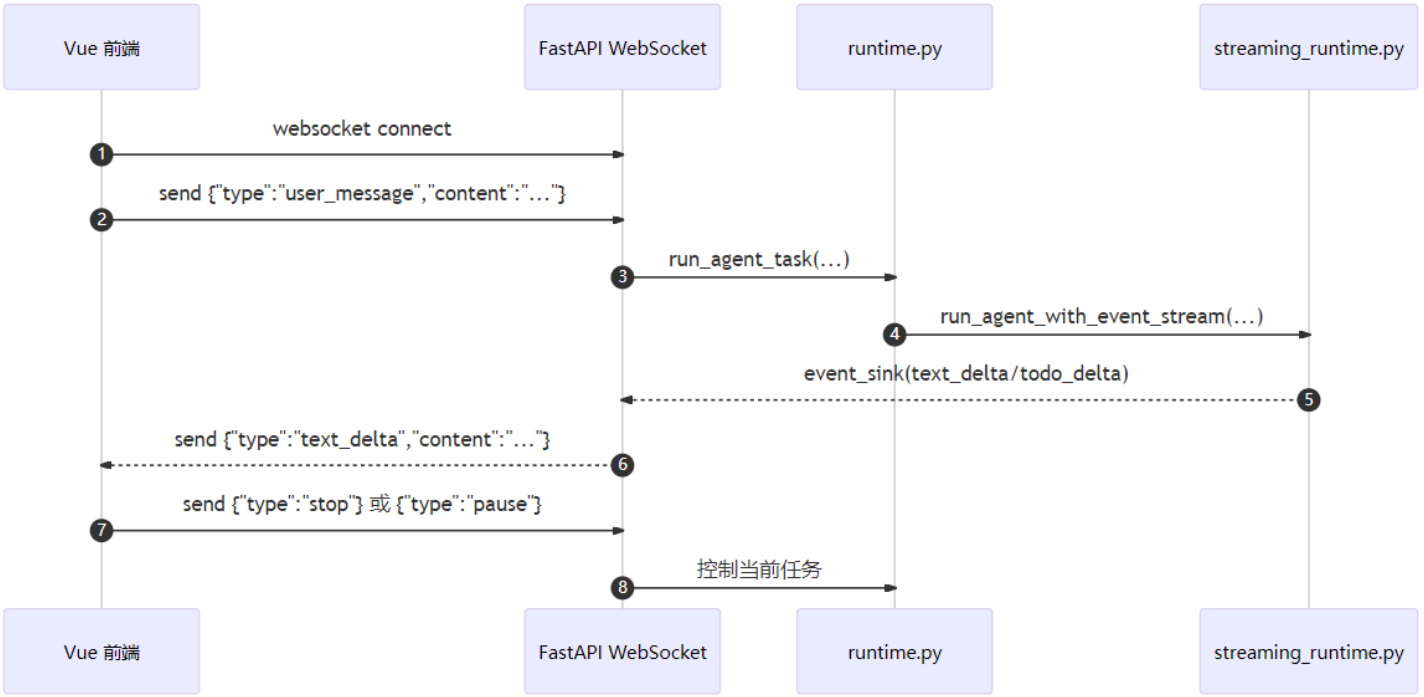
WebSocket 的核心特点：

特点	说明
双向通信	客户端和服务端可以在同一条连接上互相发送消息
长连接	建立后持续存在，适合频繁实时交互
低延迟	比反复 HTTP 请求更适合高频消息
协议更复杂	需要处理连接生命周期、心跳、重连、断线、并发房间等问题
部署要求更高	反向代理、负载均衡、超时策略需要额外配置

WebSocket 更适合这些场景：

- 实时聊天系统；
- 多人协作编辑；
- 在线游戏；
- 实时语音/消息控制；
- 前端需要随时向后端发送取消、暂停、继续、调整参数等控制命令；
- 一个连接中有多路双向事件。

如果 LX-AICODING 使用 WebSocket，可能会变成下面这种结构：

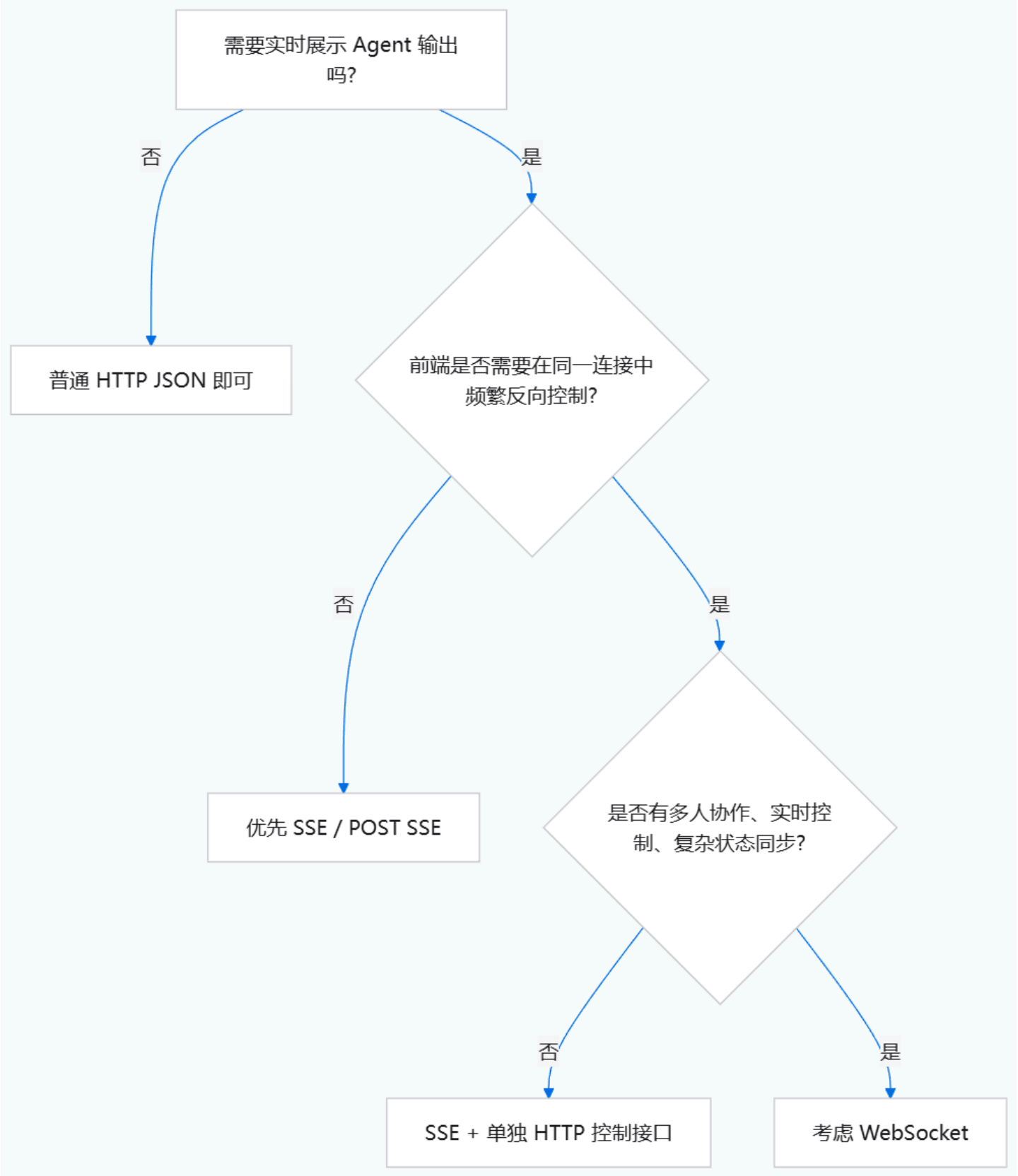


4、SSE 和 WebSocket 对比

两者都能实现“实时更新页面”，但适用场景不同。

对比项	**SSE**
通信方向	服务端 -> 客户端单向推送
底层协议	HTTP 文本流
浏览器支持	原生 EventSource 支持 GET；POST SSE 可用 fetch 流读取
请求体支持	原生 EventSource 不支持 POST；fetch POST SSE 支持请求体
实现复杂度	相对简单
代理兼容	通常更容易
断线重连	EventSource 自带部分重连；fetch POST 需要自己处理
消息格式	文本事件，适合 JSON
适合 Agent token 输出	很适合
适合实时协作/双向控制	一般
当前项目适配成本	低

从 Agent 产品角度看，可以这样判断：



5、SSE中最重要的`StreamingResponse`

普通 HTTP 响应是“一次请求，一次完整返回”。这不适合 Agent 项目，因为 Agent 可能需要：

- 读取仓库文件；
- 生成任务计划；
- 调用模型多轮推理；

- 执行命令；
- 创建或复用 PR；
- 输出很长的技术方案或审查报告。

如果等全部完成后再返回，用户会长时间看不到任何反馈。

所以当前项目使用 FastAPI 的：

StreamingResponse(event_iter(), media_type="text/event-stream")

它允许后端在一个 HTTP 响应中持续 `yield` 多段 SSE 文本。前端可以边接收边渲染。

简化伪代码：

代码块

```
async def event_iter():
    yield sse("user_message", {"content": content})
    yield sse("message_start", {"message_id": message_id})

    while True:
        event, data = await queue.get()
        yield sse(event, data)  # 发送数据给客户端
        if event == "done":
            break

return StreamingResponse(event_iter(), media_type="text/event-stream")
```

这里的关键点是：`event_iter()` 是异步生成器，每次 `yield` 都会向浏览器推送一段内容。

6、`dashboard_routes.py` 的详细职责

`agent/api/dashboard_routes.py` 是 FastAPI 和 Vue Dashboard 之间的适配层。它不是 Agent 核心推理层，也不是 DeepAgents 工厂层，而是一个 **Web API 边界层**。

可以把它理解成四件事：

职责	具体说明	相关函数
页面初始化	给前端提供当前用户、模型下拉框、默认模型等信息	dashboard_me()、d
历史会话读取	给左侧会话列表和历史详情提供稳定数据	dashboard_threads(
当前轮实时运行	接收用户输入，创建或复用 thread，返回 POST SSE 实时事件	dashboard_stream_ dashboard_stream_
删除会话	删除 Store 业务数据和 checkpoint 历史	dashboard_delete_t

它和 `runtime.py` 的边界非常重要：

文件	更接近哪一层	主要问题
dashboard_routes.py	HTTP/SSE 产品接口层	前端怎么请求、
runtime.py	Agent 运行调度层	当前用户输入应
server.py	Agent 工厂层	这个 Agent 应该
streaming_runtime.py	DeepAgents 事件适配层	如何消费 stream

一句话总结：

> `dashboard_routes.py` 只负责“接住前端请求并把后端事件推回前端”，不负责判断任务策略，也不负责组装 Agent。

路由结构

当前 Dashboard API 统一挂载在：

```
代码块
1  dashboard_router = APIRouter(prefix="/dashboard/api")
```

也就是说，文件中的路由都会自动加上 `/dashboard/api` 前缀。

主要接口如下：

HTTP 方法	路径	后端函数
GET	/dashboard/api/me	dashboard_me()
GET	/dashboard/api/options	dashboard_options()
GET	/dashboard/api/threads	dashboard_threads()
GET	/dashboard/api/threads/{thread_id}	dashboard_thread_deta
POST	/dashboard/api/threads/stream-message	dashboard_stream_new
POST	/dashboard/api/threads/{thread_id}/stream-message	dashboard_stream_exis
DELETE	/dashboard/api/threads/{thread_id}	dashboard_delete_thre

这套 API 的重点不是 REST 资源建模，而是为了让前端拥有清晰的三类操作：

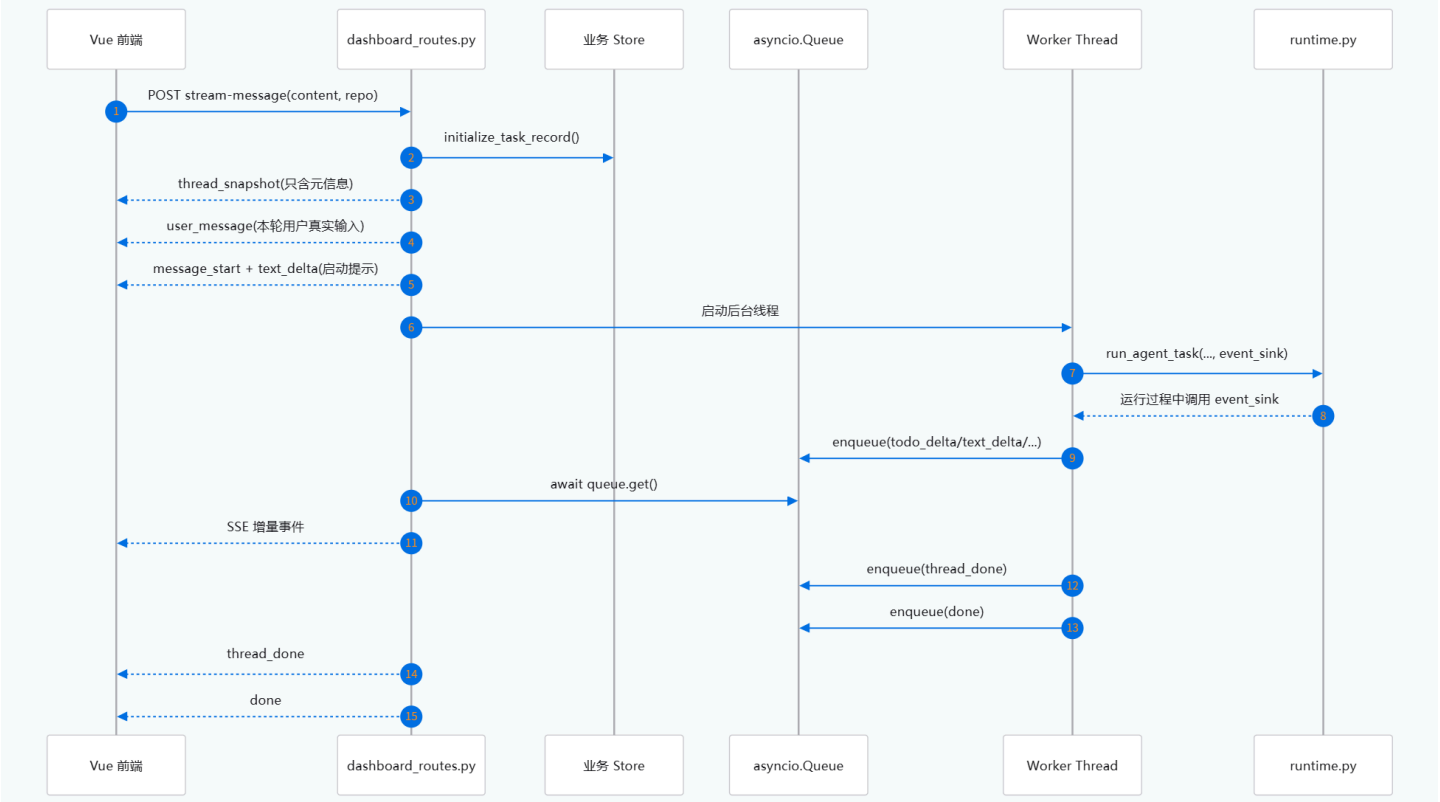
1. **读配置**：`/me`、`/options`
2. **读历史**：`/threads`、`/threads/{thread_id}`
3. **跑任务**：`POST /stream-message`

4. **删会话：** `DELETE /threads/{thread_id}`

`_post_streaming_response()` 的执行顺序

`_post_streaming_response()` 是当前 Dashboard 后端最关键的函数。它把一次前端输入转换成一条实时 SSE 流。

核心顺序如下：



几个设计点：

1. `thread_snapshot` 不包含 `messages` 它只更新当前会话的仓库、状态、分支、PR 等元信息，不能覆盖前端已经追加的本轮用户输入。
2. `user_message` 必须尽早发送 用户输入进入后端后，后端立即通过 `user_message` 确认这轮输入，前端据此稳定本地临时消息。
3. Agent 在后台线程运行 避免阻塞 FastAPI 的异步响应生成器。
4. `event_sink` 是 runtime 到 SSE 的桥 `runtime.py` 和 `streaming_runtime.py` 不知道 FastAPI 的存在，只调用 `event_sink(event, data)` 。
5. `done` 是 SSE 结束信号 前端收到 `done` 后关闭本轮 streaming 状态，输入框恢复可用。

`asyncio.Queue` 和后台线程

这里容易让初学者困惑：为什么不用一个普通函数直接跑？

原因是 FastAPI 的 SSE 发送发生在异步世界，而 Agent 运行发生在同步世界。

部分	执行模型
StreamingResponse(event_iter())	async/await，运行在事件循环中
run_agent_task()	同步阻塞函数
DeepAgents 工具调用	可能执行文件、命令、网络请求，耗时不可控

如果在 `event_iter()` 里直接调用 `run_agent_task()`：

代码块

```
async def event_iter():
    run_agent_task(...)  # 阻塞
    yield sse(...)
```

那么 `yield` 要等 Agent 全部结束后才执行，页面又会变成“一次性输出”。

当前做法是：

代码块

```
thread = threading.Thread(target=worker, daemon=True)
thread.start()

while True:
    event, data = await queue.get()
    yield _sse_part(event, data)
```

这样：

- worker 线程负责慢任务；
- async generator 负责持续发 SSE；
- `asyncio.Queue` 负责跨线程传递事件；
- `loop.call_soon_threadsafe()` 确保从 worker 线程安全地把事件放回事件循环。

注意：

注意：线程不是为了"支持并发"而加的——每个 SSE 请求本来就各起一个线程。线程的作用是"把阻塞搬离事件循环，保住流式推送给前端的通道"。

只把 `run_agent_task` 改成 `async def` 行不行

不行。 光改函数签名毫无作用。 `async def` 只有在函数内部真正 `await`（让出事件循环）时才不阻塞。你的调用链里最底层的 `agent.stream_events()` 是同步迭代器、工具是 `subprocess`/同步 HTTP/同步 SQLite——这些阻塞调用不会因为外层套了个 `async def` 就变非

阻塞。直接 `await run_agent_task(...)` 的结果是：事件循环照样被卡死，流式也没了，比现在更糟。

`dashboard_routes.py` 中几个辅助函数

函数	作用	讲课重
_normalize_dashboard_repo_url()	把 owner/repo 转成完整 Gitee URL	API 边
_sse_part()	生成标准 SSE 字符串	前后端
_timestamp_ms()	把后端时间转成前端毫秒时间戳	前端展
_status_for_frontend()	把后端状态映射成页面状态	后端状
_repo_full_name()	组装 owner/repo	侧边栏
_pr_payload()	把 PR URL 转成前端 PR 对象	PR 展
_message_payload()	从 checkpoint 生成历史 messages	历史正
_thread_payload()	生成完整 thread DTO	打开历
_thread_meta_payload()	生成不含 messages 的 thread 元信息	当前轮

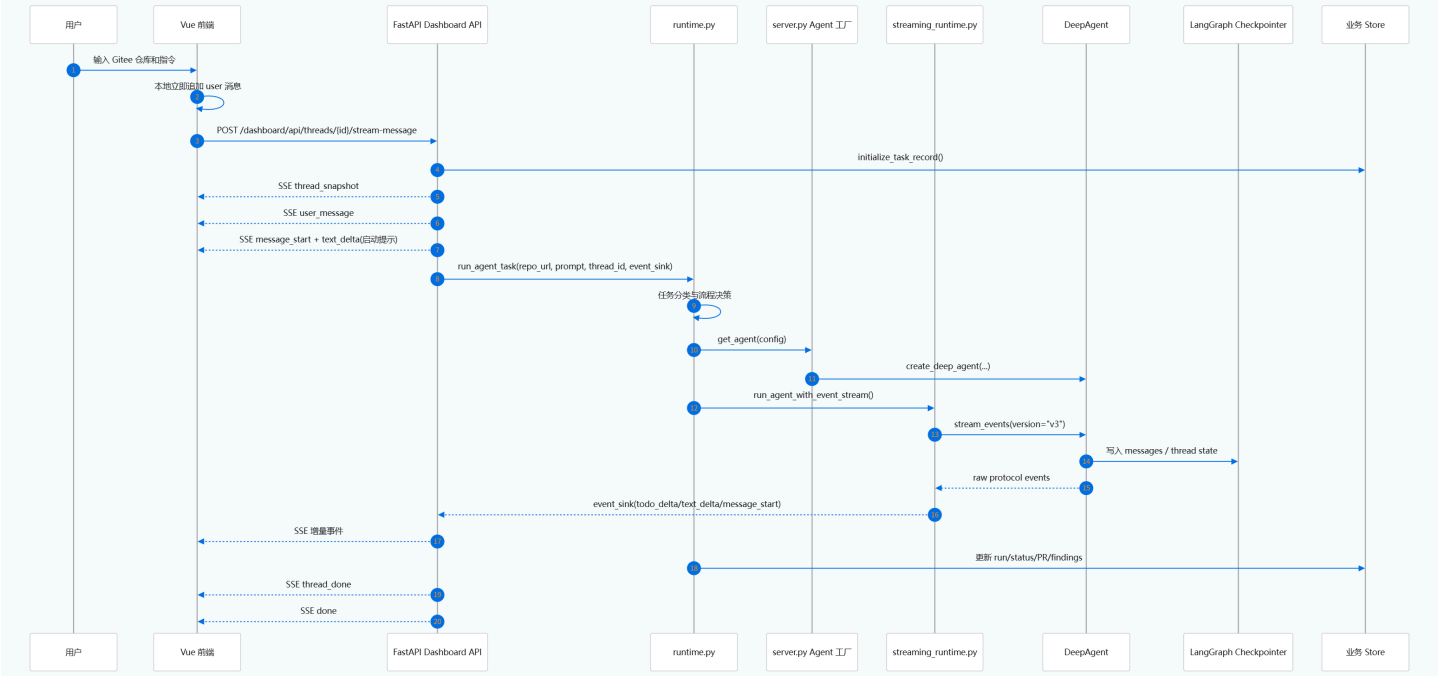
其中最重要的是 `_thread_meta_payload()`：

代码块

```
payload = _thread_payload(thread)
payload.pop("messages", None)
return payload
```

它的意义是：**当前轮实时流中，只允许更新元信息，不允许用后端快照覆盖前端当前消息列表。**

第十二章、Agent的运行调度



1、调度职责

`runtime.py`、`server.py`、`streaming_runtime.py`

这一组三个文件最容易混淆。可以先记住一句话：

> `runtime.py` 决定本轮任务怎么跑，`server.py` 负责把 DeepAgent 组装出来，`streaming_runtime.py` 负责真正驱动 Agent 运行并解析流式事件。

也就是说，它们不是平级重复关系，而是一个调用链：

FastAPI Dashboard API

-> runtime.py

-> server.py

-> create_deep_agent(...)

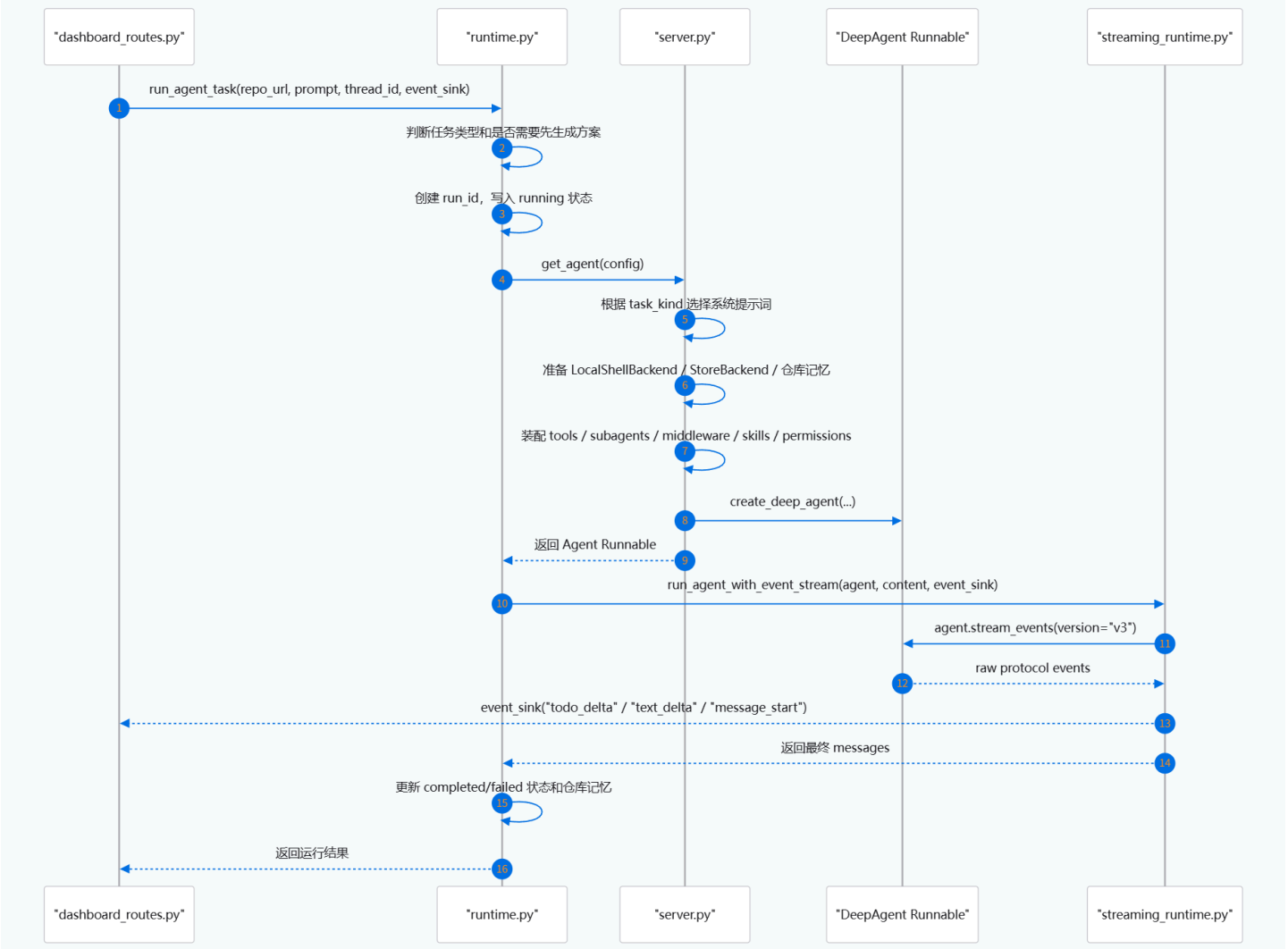
-> streaming_runtime.py

-> agent.stream_events(version="v3")

-> event_sink(...)

-> FastAPI SSE

-> Vue 前端



三个文件的职责对比

文件	角色	主要负责
agent/core/runtime.py	运行调度层	判断任务类型、决定 planning/coding/review/c 建 run 记录、更新任务状态、确认方案后切换 co 务成功后更新仓库记忆
agent/server.py	Agent 工厂层	根据 thread_id、task_kind、repo_url 创建 Dee 注入模型、工具、子 Agent、middleware、skill backend、permissions、checkpoint、store
agent/core/ streaming_runtime.py	Agent 执行与事件适配层	调用 agent.stream_events(version="v3")，解析 token、write_todos、subagent 事件，把它们转 message_start、text_delta、todo_delta 等前端

三个文件之间传递的关键对象

文件之间传递的关键对象

对象	从哪里来	传到哪里
thread_id	FastAPI / 前端当前会话	runtime.py、server.py、streaming_runtime.py、chec Store
task_kind	runtime.py 分类或流程改写	server.py、streaming_runtime.py
repo_url	前端输入或历史 thread fallback	runtime.py、server.py
agent	server.py 创建	runtime.py 再传给 streaming_runtime.py
event_sink	dashboard_routes.py 创建	runtime.py、streaming_runtime.py
run_id	runtime.py 创建	streaming_runtime.py、Store、前端 message_id

2、runtime.py

`agent/core/runtime.py` 是整个 Agent 项目的运行调度中心。它不负责解析 token，也不直接实现工具，它主要负责决定“本轮任务应该怎么跑”。

runtime.py 的职责边界

职责	是否属于 runtime.py
判断任务类型	是
判断是否需要先生成技术方案	是
判断用户是否确认上一轮方案	是
创建 thread/run 业务记录	是
构建 Agent 配置	间接，通过 server.py
解析 DeepAgents V3 事件	否，交给 streaming_runtime.py
维护 checkpoint	否，交给 LangGraph checkpointer
更新仓库记忆	是，coding 成功后基于最终回答更新
前端渲染	否

run_agent_task 总入口

核心函数：

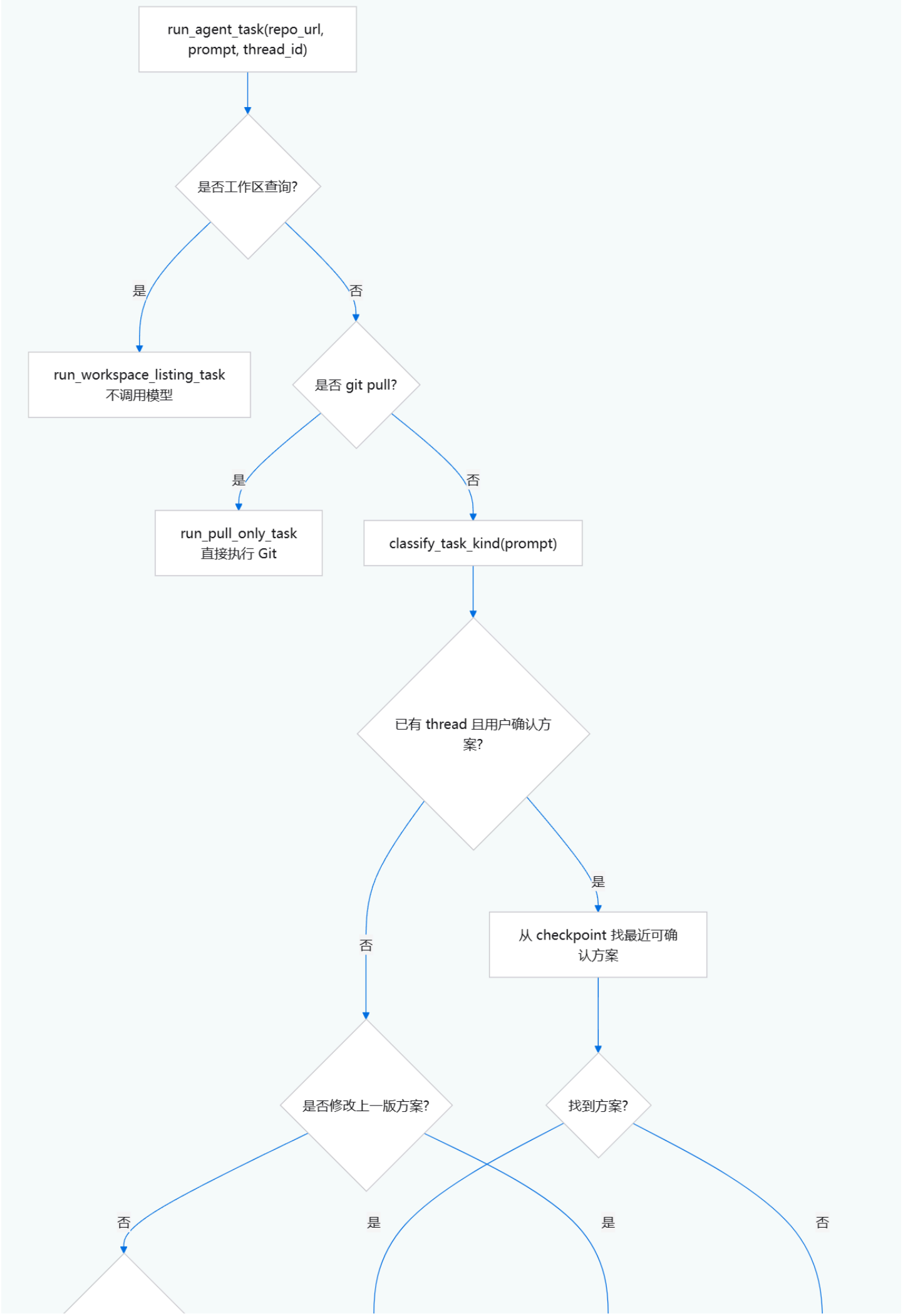
def run_agent_task(*, repo_url: str, prompt: str, thread_id: str | None = None, event_sink=None)

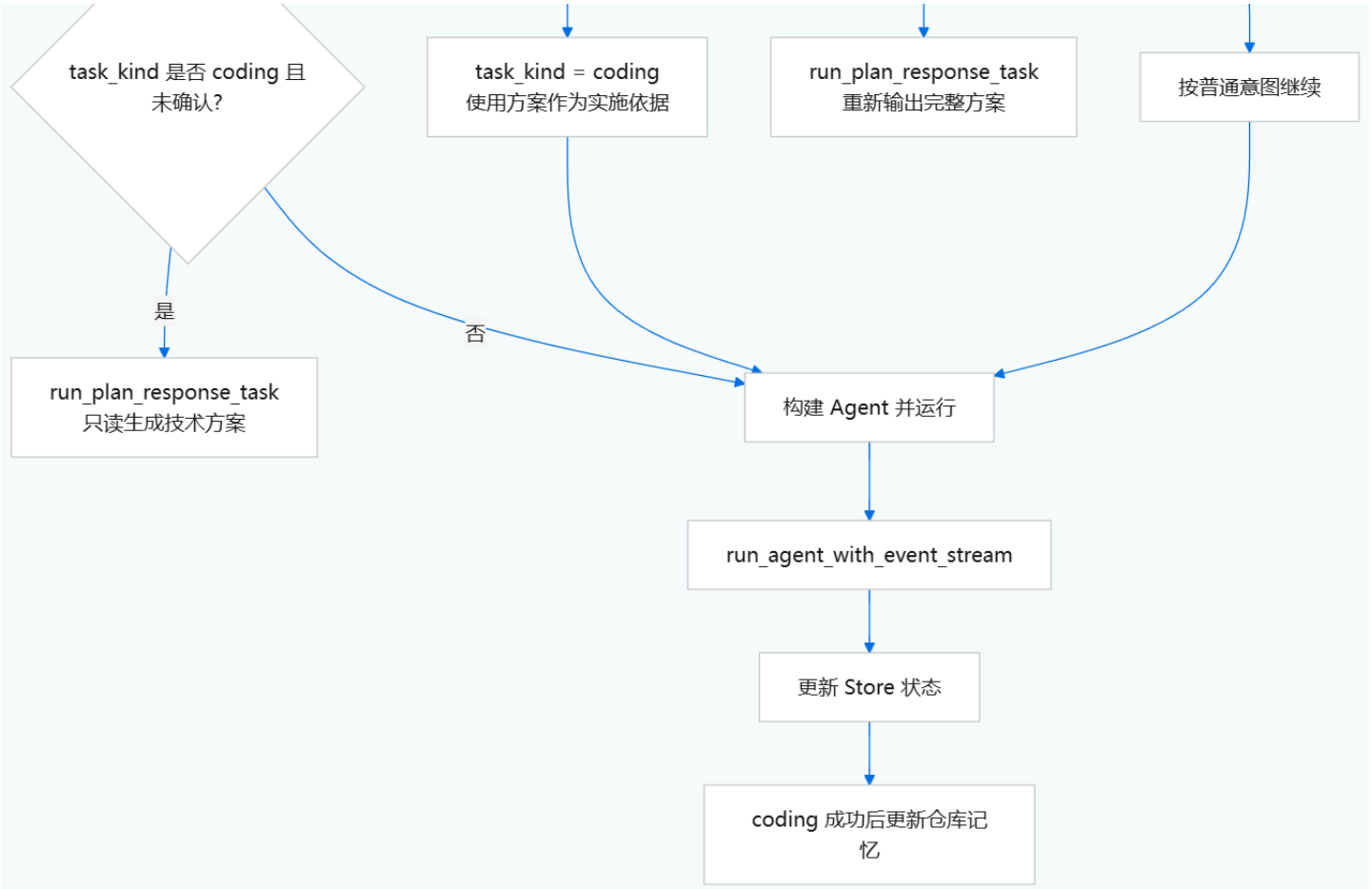
这个函数处理一轮用户输入。它的决策顺序可以理解成：

1. 是否是工作区查询任务。

2. 是否是 git pull 任务。
3. 调用 `classify_task_kind(prompt)` 做第一层意图分类。
4. 如果是 coding 需求但没有确认方案，先转 planning。
5. 如果用户输入“确认/开始实施”，从 checkpoint 找上一轮可确认方案。
6. 如果是方案修订，基于上一版方案重新生成完整新版方案。
7. 其它任务进入通用 Agent 执行分支。

runtime 调度流程图


run_agent_task(repo_url, prompt, thread_id)
是否工作区查询?
是
否
run_workspace_listing_task
不调用模型
是否 git pull?
是
否
run_pull_only_task
直接执行 Git
classify_task_kind(prompt)
已有 thread 且用户确认方案?
是
否
从 checkpoint 找最近可确认方案
是否修改上一版方案?
找到方案?
否
是
是
否



第十三章、DeepAgents V3 流式输出

DeepAgents 的 `stream_events(version="v3")` 会返回比较底层的 raw protocol events。它包含：

- assistant （AIMessage）文本块开始和结束。
- assistant 文本 token delta。
- 工具调用 chunk。（ToolMessage）
- `write_todos` 参数增量。
- 子 Agent 生命周期。

这些事件不能直接给前端展示。`agent/core/streaming_runtime.py` 的作用就是把官方事件转换成项目自己的 SSE 事件。

1、调用入口

runtime.py 调用：

代码块

```
result = run_agent_with_event_stream(
    agent=agent,
    thread_id=thread_id,
    run_id=run_id,
    content=content,
```

```
        task_kind=task_kind,
        event_sink=event_sink,
    )
```

`run_agent_with_event_stream()` 内部执行：

代码块

```
    stream = agent.stream_events(
        {"messages": [{"role": "user", "content": content}]},
        version="v3",
        config={"configurable": {"thread_id": thread_id}},
    )
```

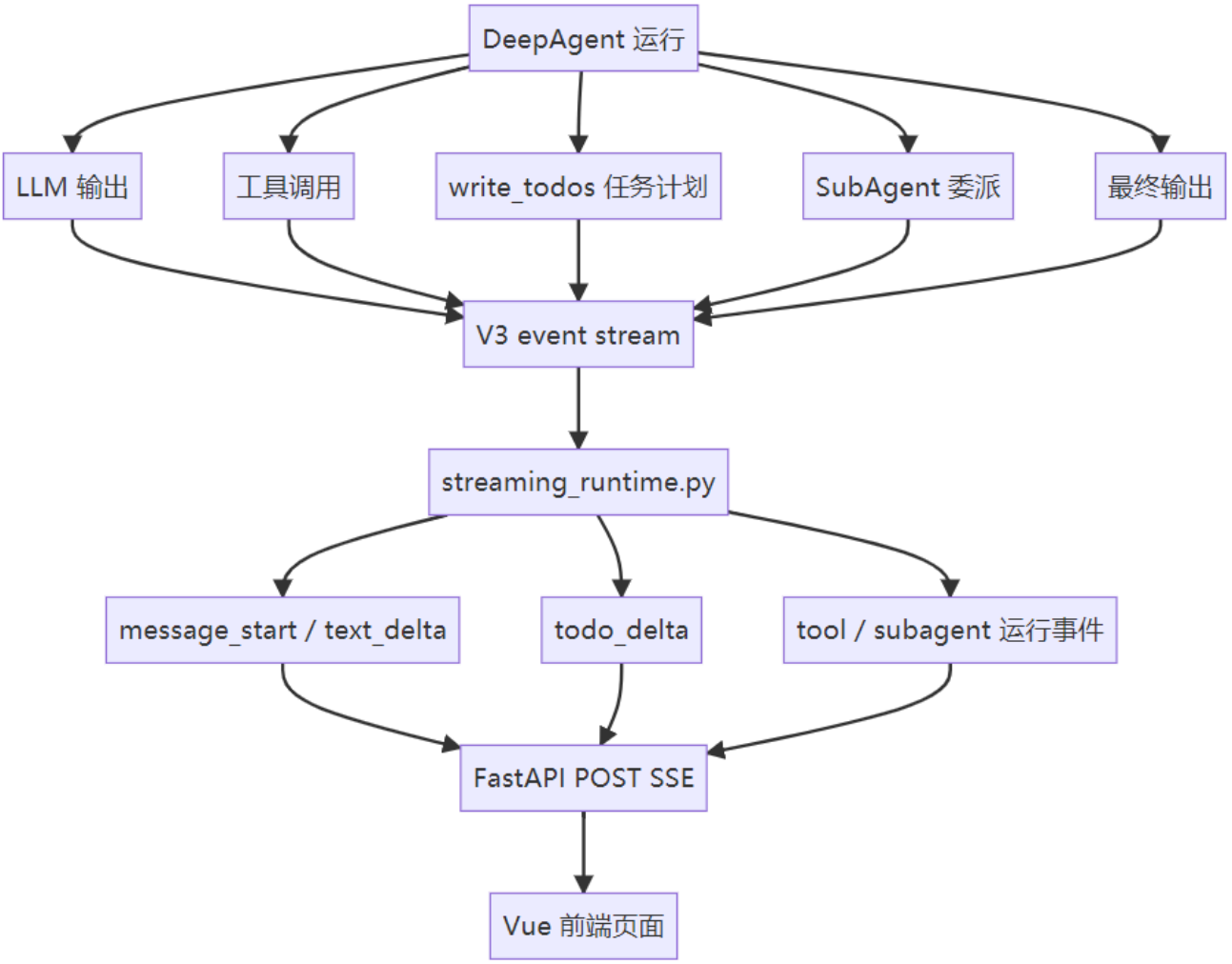
2、DeepAgents V3 event streaming

DeepAgents V3 event streaming 可以理解成：DeepAgents 在 LangGraph 运行图之上，提供了一套更适合 Agent 产品展示的流式事件接口。

传统流式输出通常只关注“模型正在吐 token”。但是 DeepAgents 的运行过程不只有模型文本，还包含：

- 主 Agent 的 assistant 文本。
- 工具调用的开始、参数增量、输出增量、结束状态。
- `write_todos` 这类特殊工具产生的任务计划。
- 子 Agent 的创建、运行、结束和输出。
- 图状态快照和最终输出。

因此，DeepAgents V3 event streaming 不是单纯的 token stream，而是一个“Agent 运行事件流”。当前项目正是基于这一点，把 DeepAgents 的底层事件转换成课程版前端可以理解的 SSE 事件。



V3 typed projections

DeepAgents V3 event streaming 暴露的不是单一列表，而是一组 typed projections。可以把它理解成：同一轮 Agent 运行，被框架整理成多个“观察视角”。

Projection	适合观察什么	典型用途
stream.messages	顶层主 Agent 的 assistant 消息	展示主 Agent 的自然语言
stream.tool_calls	顶层主 Agent 调用的工具	展示工具卡片、工具参数
stream.subagents	DeepAgents 子 Agent 运行情况	展示 reviewer、general 务
subagent.messages	某个子 Agent 的 assistant 消息	展示子 Agent 的分析或审
subagent.tool_calls	某个子 Agent 调用的工具	展示子 Agent 读取 diff、作
stream.values	LangGraph 状态快照	调试图状态，不适合直接
stream.output	流结束后的最终结果	任务完成后读取最终 me

如果只做一个简单聊天页面，读取 `stream.messages` 可能就够了。但是 LX-AICODING 不是普通聊天，它需要同时展示：

- 任务计划列表。
- 技术方案或总结正文。
- 代码审查子 Agent 的运行过程。
- 工具是否正在执行。
- 最终结果和 PR 链接。

所以当前项目没有只依赖某一个 projection，而是进一步读取 raw protocol events，再统一转换成自己的前端事件。

raw protocol events

官方 typed projections 更容易使用，但当前项目最终选择解析 raw protocol events，原因是前端有更强的实时性要求。

raw event 可以理解成框架最原始的事件协议。它通常包含：

代码块

```
{
    "method": "messages",
    "params": {
        "namespace": [],
        "data": {
            "event": "content-block-delta",
            "data": {
                "delta": {
                    "type": "text",
                    "text": "正在分析项目结构..."
                }
            }
        },
    },
}
```

当前项目主要关心几类 raw event：

raw event	当前项目处理
method=messages + content-block-start	创建或切换当前 assistant message
method=messages + content-block-delta	提取文本 delta，流式输出到前端
method=messages + content-block-finish	flush 当前 assistant 文本
method=messages 里的 tool call chunk	提取 write_todos 参数增量
method=tool_calls	兼容部分版本的工具调用事件
method=subagents	记录子 Agent 运行步骤

DeepAgents 会把 assistant 输出拆成类似这样的事件：

代码块

```
content-block-start
content-block-delta: "我"
content-block-delta: "先"
content-block-delta: "分析"
content-block-finish
```

项目里的处理思路是：

代码块

```
如果遇到文本块 start:
    准备一条新的 assistant 消息

如果遇到 text delta:
    拼到 current_assistant_text:  24个Token并接成一个Event
    如果满足 flush 条件:
        推给前端

如果遇到文本块 finish:
    把最后没推完的文本推给前端
    清空当前文本块
```

`streaming_runtime.py` 里对应的辅助函数包括：

函数	作用
_event_payloads()	从 raw event 中取 params.data
_text_delta_from_event()	提取 assistant 文本 token
_text_block_marker_from_event()	识别文本块开始和结束
_tool_chunk_from_message_event()	从 message event 中提取工具调用 chunk
_tool_call_from_raw()	兼容 method=tool_calls
_subagent_from_raw()	兼容 method=subagents
_todos_from_args_text()	从不完整 JSON 参数中尽早提取任务计划

这样做的好处是：**任务计划和 assistant 文本都可以真正按 chunk 实时进入前端**，不必等到 Agent 最终完成后再一次性展示。

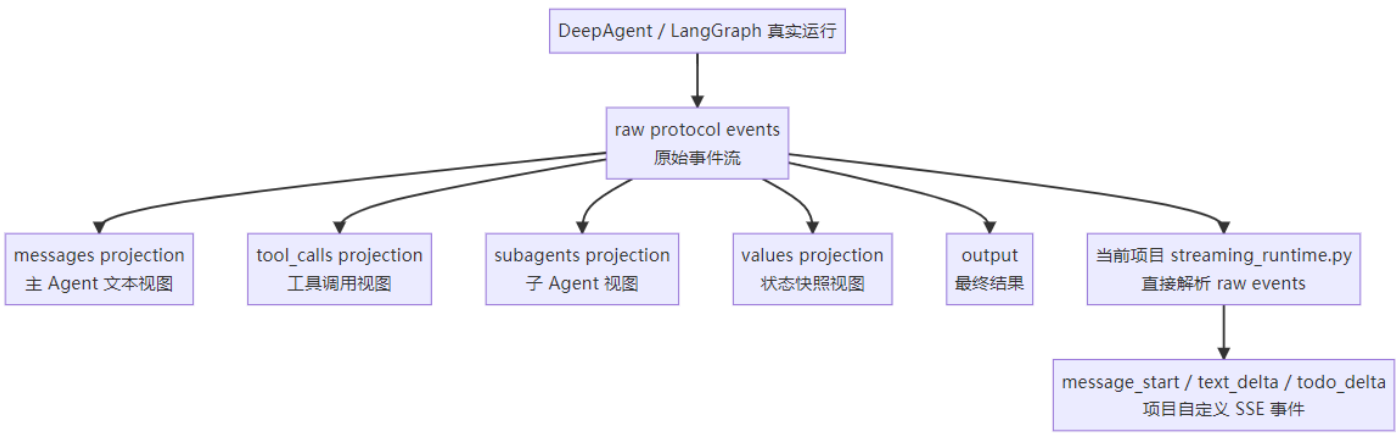
typed projections 和 raw protocol events 的关系

这两个概念不是互斥关系，也不是两套完全不同的流式系统。它们的关系可以这样理解：

> raw protocol events 是 DeepAgents / LangGraph 运行时吐出来的原始事件；typed projections 是框架在原始事件之上整理出来的“分类视图”。

换句话说：

- raw protocol events 更底层，信息更细，顺序更接近真实运行过程。
- typed projections 更上层，语义更清楚，用起来更简单。
- typed projections 通常可以看作对 raw events 的筛选、归类和包装。
- 如果项目只需要普通文本、工具结果、子 Agent 状态，用 typed projections 更省事。
- 如果项目要精确控制 token chunk、`write_todos` 参数增量、前端消息 ID、输出顺序，就更适合直接读 raw events。



从这张图可以看出：typed projections 和 raw protocol events 的数据源是同一轮 Agent 运行。区别在于，typed projections 是官方帮你“整理好的视图”，raw protocol events 是你自己拿到底层事件

后自行解释。

对比项	typed projections	raw protocol events
抽象层级	高层 API	底层协议事件
使用难度	低	高
字段语义	更稳定、更面向业务概念	更贴近运行协议，字段更细
事件粒度	通常较粗，按消息、工具、子 Agent 组织	更细，可以看到 content block、chunk
输出顺序	不同 projection 需要 interleave 才能统一顺序	原始迭代天然接近事件到达顺序
适合场景	快速做 UI、调试工具/子 Agent 状态	精确控制前端流式展示和任务计
当前项目定位	可作为参考视图	主链路采用

用 typed projections 的伪代码

如果只想做一个简单的 Agent 运行页，可以直接消费 typed projections。代码会比较清晰：

代码块

```
stream = agent.stream_events(
    {"messages": [{"role": "user", "content": user_input}]},
    version="v3",
)

# 读取主 Agent 的文本输出。
for message in stream.messages:
    ui.append_assistant_message(message.text)

# 读取主 Agent 工具调用。
for tool_call in stream.tool_calls:
    ui.show_tool_card(
        name=tool_call.tool_name,
        input=tool_call.input,
        completed=tool_call.completed,
        error=tool_call.error,
    )

# 读取子 Agent 的运行输出。
for subagent in stream.subagents:
    ui.show_subagent_status(
        name=subagent.name,
        status=subagent.status,
    )

```

```
        for sub_message in subagent.messages:
            ui.append_subagent_message(
                subagent_name=subagent.name,
                text=sub_message.text,
            )
```

这种写法的优点是直观。开发者不需要理解 `content-block-delta`、`namespace`、tool call chunk 这些细节。

但是它也有明显不足：如果 `write_todos` 的参数是逐步增长的 JSON 字符串，typed projections 可能更偏向“工具调用对象最终长什么样”，而不是“参数每次增长了什么”。任务计划可能不会足够早地出现在页面上。

用 raw protocol events 的伪代码

当前项目采用的是更底层的写法：

代码块

```
stream = agent.stream_events(
    {"messages": [{"role": "user", "content": user_input}]},
    version="v3",
)

for event in stream:
    # 1. assistant 文本块开始：前端先创建一个 AI 消息容器。
    if is_content_block_start(event):
        message_id = new_message_id()
        sse_send("message_start", {"message_id": message_id, "role": "assistant"})

    # 2. assistant 文本 token：前端立即更新正文。
    elif is_content_block_delta(event):
        text = extract_text_delta(event)
        sse_send("text_delta", {"message_id": message_id, "content": text})

    # 3. write_todos 工具参数 chunk：尽早解析任务计划。
    elif is_write_todos_args_chunk(event):
        todos = parse_partial_todos(event)
        sse_send("todo_delta", {"todos": todos})

    # 4. 子 Agent 事件：转成简短运行步骤。
    elif is_subagent_event(event):
        step = summarize_subagent_event(event)
        sse_send("run_event", step)

    # 5. 工具失败：转成可读错误。
```

```
        elif is_tool_error(event):
            error = extract_tool_error(event)
            sse_send("run_event", {"level": "error", "message": error})
```

这种写法更复杂，但它能解决当前项目最关键的几个问题：

项目需求	typed projections 是否方便	raw events 是否
assistant 内容按 chunk 实时输出	可以，但控制粒度有限	很方便
write_todos 任务计划尽早出现	不稳定	很方便
多轮消息必须严格追加到当前 run 后面	需要额外封装	可用 run_id/me 定
子 Agent 过程要简洁展示	可以	可以，同时能按
避免工具输出刷屏	需要过滤 projection	可以在 raw eve

可以用下面这段伪代码串起来：

代码块

```
def run_agent_with_event_stream(agent, content, event_sink):
    # 1. 启动 DeepAgents V3 事件流
    stream = agent.stream_events(
        {"messages": [{"role": "user", "content": content}]},
        version="v3",
    )

    # 2. 按真实到达顺序消费 raw event
    for event in stream:
        if is_assistant_text_delta(event):
            event_sink("text_delta", extract_text(event))

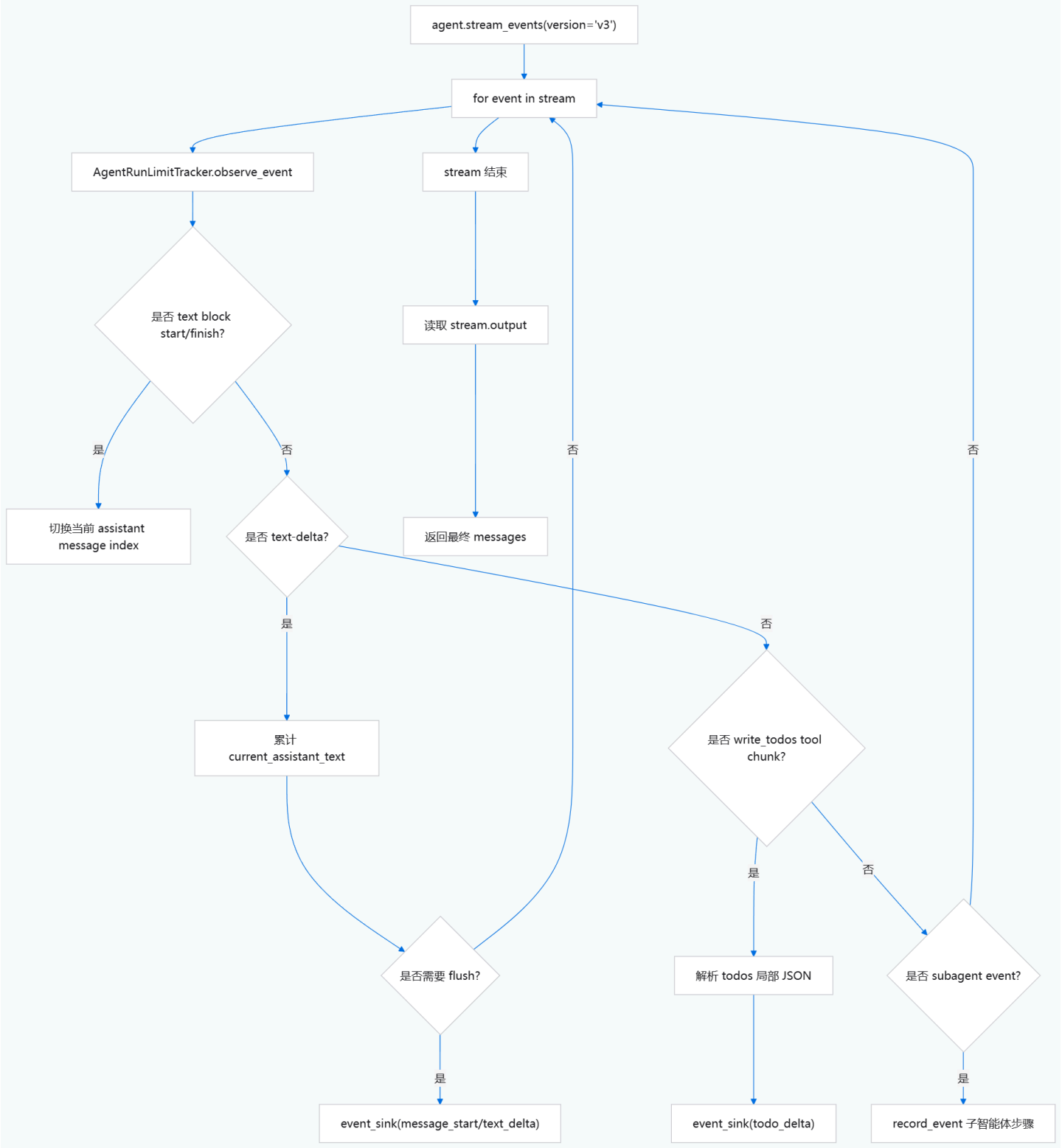
        elif is_write_todos_chunk(event):
            event_sink("todo_delta", extract_todos(event))

        elif is_subagent_event(event):
            event_sink("run_event", summarize_subagent(event))

    # 3. 流结束后读取最终输出，交给 runtime 后处理
    return stream.output
```

这段伪代码表达的是当前项目的核心思想：**DeepAgents 负责产生事件，`streaming_runtime.py` 负责翻译事件，FastAPI 负责传输事件，Vue 前端负责展示事件。**

3、 V3 事件消费流程



最小伪代码串联

前端

代码块

```
submit(prompt) {
  messages.push(user(prompt))
  fetch('/dashboard/api/threads/{id}/stream-message', { method: 'POST' })
    .readSSE((event, data) => {
      if (event === 'user_message') stabilizeUserMessage(data)
      if (event === 'todo_delta') upsertTodo(data)
      if (event === 'message_start') ensureAgentMessage(data.message_id)
      if (event === 'text_delta') appendText(data.message_id, data.content)
      if (event === 'thread_done') mergeThreadMeta(data)
      if (event === 'done') streaming = false
    })
}
```

FastAPI

代码块

```
async def event_iter():
    yield user_message
    start_worker_thread(run_agent_task)
    while True:
        event = await queue.get()
        yield sse(event)
        if event.name == "done":
            break
```

runtime

代码块

```
def run_agent_task(repo_url, prompt, thread_id, event_sink):
    task_kind = classify_task_kind(prompt)

    if task_kind == "coding" and not confirmed_plan:
        return run_plan_response_task(...)

    agent = get_agent(config)
    result = run_agent_with_event_stream(
        agent=agent,
        content=wrapped_user_content,
        event_sink=event_sink,
    )
    update_status_and_memory(result)
```

streaming_runtime

```
for event in agent.stream_events(version="v3"):
    if is_text_delta(event):
        accumulate_text()
        if should_flush():
            event_sink("text_delta", ...)
    elif is_write_todos(event):
        event_sink("todo_delta", ...)
    elif is_subagent(event):
        record_event(...)
```

4、assistant 文本如何变成前端消息

关键函数：

```
_record_assistant_stream_message(...)
```

它会产生两个前端事件：

1. `message_start`：保证前端有一个指定 `message_id` 的 AI 消息容器。
2. `text_delta`：把当前累计文本发给前端。 （加入人工的内容）

当前实现中，`text_delta` 的 `mode` 通常是 `replace`，原因是后端已经累计了一段完整文本。前端收到后直接替换当前文本块，避免 token 重复追加。

```
event_sink(
    "text_delta",
    {
        "message_id": message_id,
        "content": text,
        "mode": "replace",
    },
)
```

5、 为什么不是每个 token 都写一次

`_should_flush_stream_text()` 做了轻量合并：

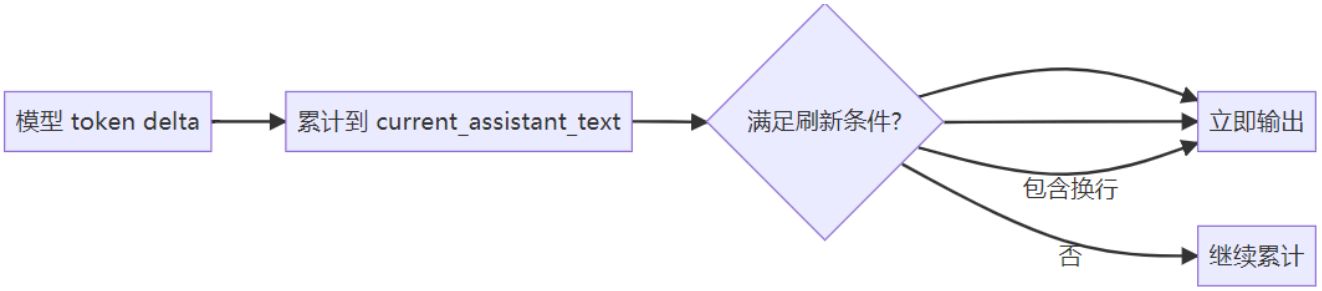
- 第一段内容立即输出。
- 累计新增约 24 个字符再刷新。
- 遇到换行立即刷新。

这样可以兼顾实时性和 SQLite/浏览器压力。

所以总结就是：

> 首段立即输出；之后每新增约 24 个字符刷新一次；遇到换行也立即刷新；没有正文则不刷新。

这样做是为了兼顾实时性和性能，避免每个 token 都写事件、推前端。



write_todos 如何流式展示

DeepAgents 的 `write_todos` 工具会把任务计划写成工具调用参数。V3 raw event 里，参数可能不是一次性完整 JSON，而是逐步增长的 JSON 字符串。

模型可能不是一次性给完整 JSON，而是一段一段吐：

```
代码块
1  {"todos": [
2    {"content": "读取项目结构", "status": "completed"},
3    {"content": "修改代码", "status": "in_progress"}
4  ]}
```

但流式过程中可能先拿到半截：

```
代码块
1  {"todos": [{"content": "读取项目
```

项目中使用两种方式兼容：

```
代码块
```

```
如果 args 是 dict:
    说明已经完整，直接解析 todos

如果 args 是 str:
    说明可能是半截 JSON
    尝试用 _todos_from_args_text 提前提取已经闭合的 todo
```

然后发给前端：

代码块

```
event_sink("todo_delta", {"todos": todos})
```

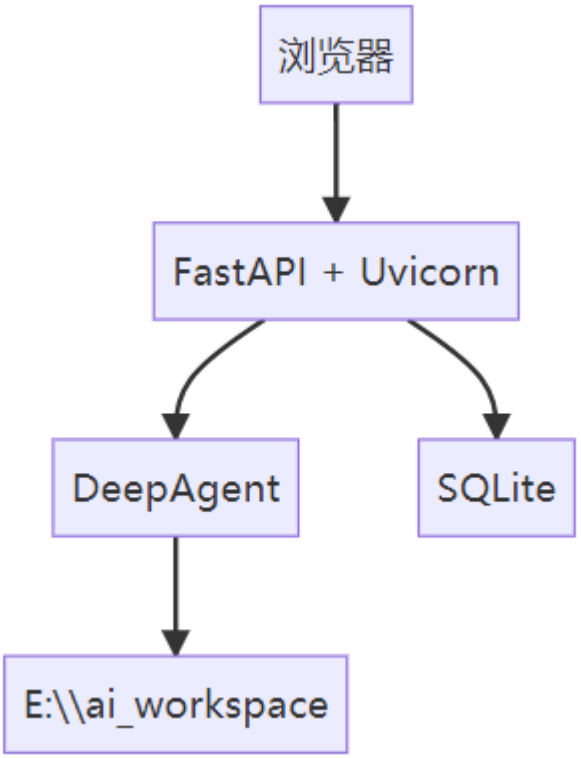
情况	处理函数
参数已经是 dict	_extract_todos()
参数是逐步增长的字符串	_todos_from_args_text()

`_todos_from_args_text()` 会用保守正则提取已经闭合的 `content/status` 对象，让前端更早看到任务计划，而不是等完整 JSON 结束。

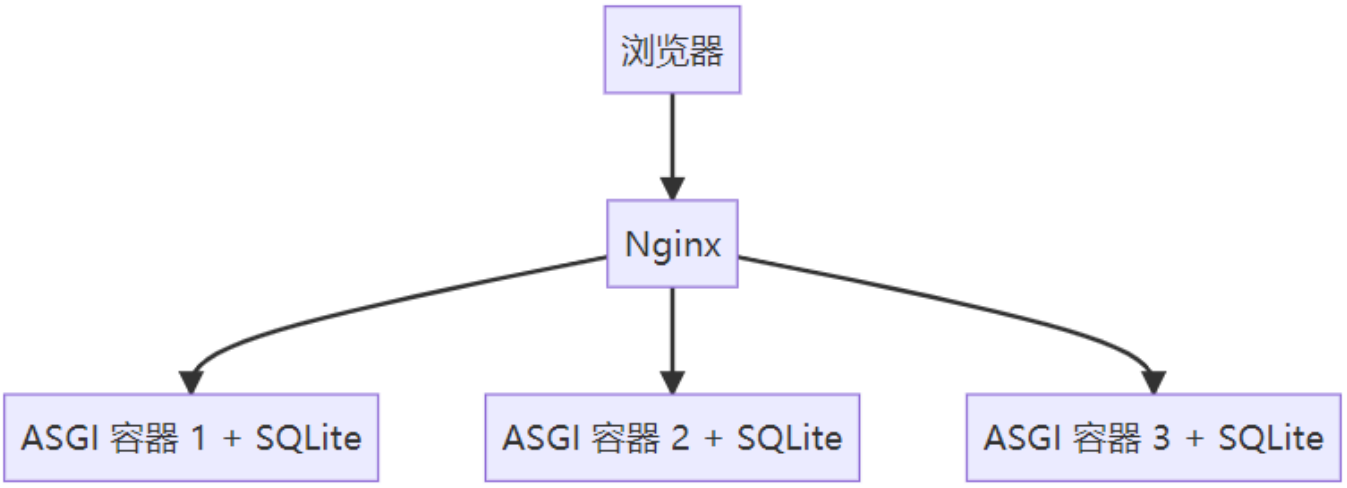
第十四章、Agent项目的并发解决方案

1、本地部署

本地教学版适合单机、少量用户、低并发：



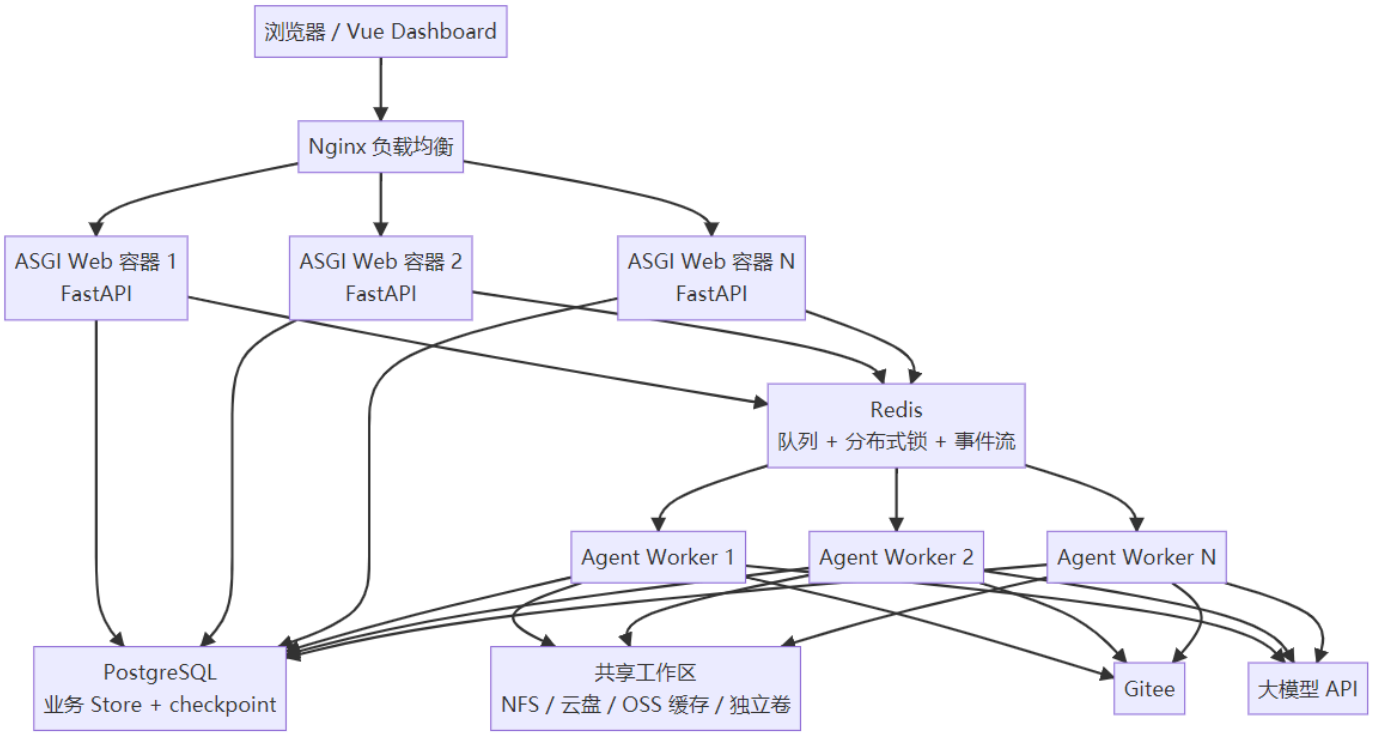
云端多容器不能简单变成：



原因是：

- 每个容器的 SQLite 文件不同，状态不一致。
- 每个容器的内存锁不同，不能保护跨容器并发。
- SSE 连接在哪个容器上，Agent 就必须在哪个容器里跑，容器重启会中断任务。
- 如果多个容器挂载同一个 SQLite 文件，并发写会更容易触发锁冲突。

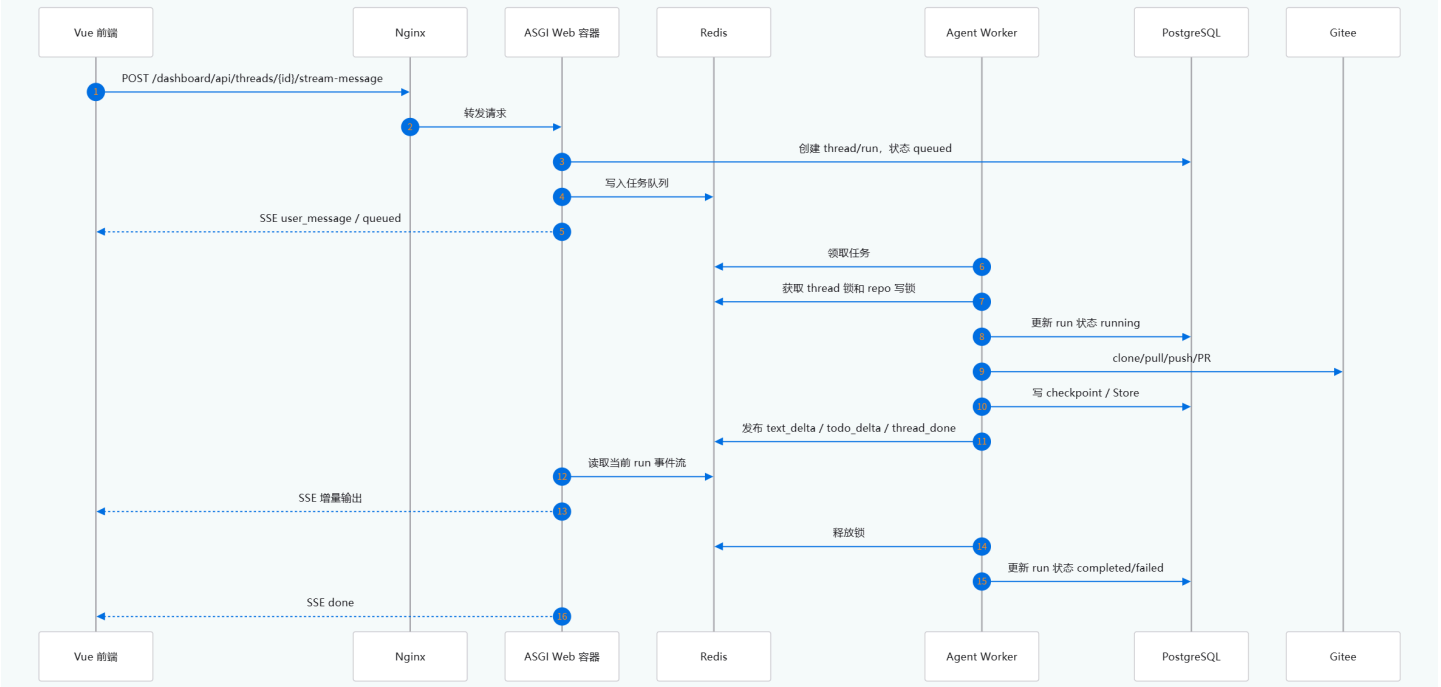
2、服务器集群部署方案



2.2 各组件职责

组件	职责
Nginx	静态资源、反向代理、负载均衡、SSE 长连接代理
ASGI Web 容器	接收 HTTP 请求、校验参数、创建任务、提供 SSE 接口、查询历史
Agent Worker	执行 DeepAgent、读写仓库、调用工具、运行测试、push/PR
PostgreSQL	保存 thread、run、review findings、checkpoint、仓库记忆等持久化数据
Redis	保存任务队列、分布式锁、实时事件流、限流计数
共享工作区	保存仓库源码、skills、policies、reviews、runtimes
Gitee	代码仓库、PR、评论
大模型服务	deepseek-v4-pro 等模型调用

2.3 数据流总览



3、Redis消息队列

3.1 Redis 承担三类职责

职责	Redis 功能
任务队列	List / Stream / Celery broker / RQ queue
分布式锁	SET key value NX EX seconds
实时事件总线	Redis Stream 或 Pub/Sub

3.2 任务队列

当用户提交任务时，ASGI 容器不直接运行 Agent，而是：

1. 在 PostgreSQL 中创建 `run`，状态为 `queued`。
2. 把任务写入 Redis 队列。
3. SSE 先返回 `queued` 事件。
4. Agent Worker 从队列取任务执行。

任务消息示例：

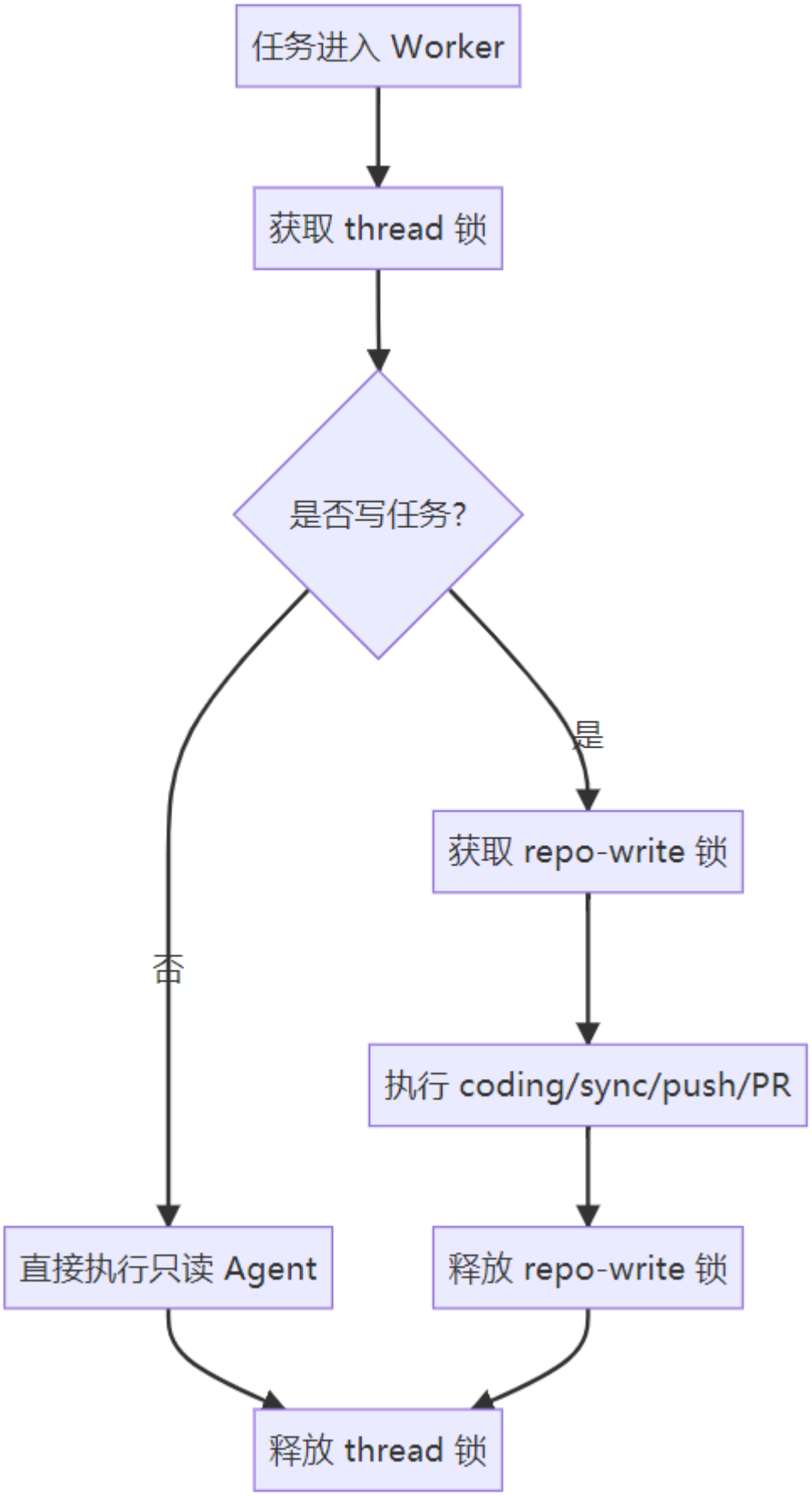
代码块

```
{
  "run_id": "run-uuid",
  "thread_id": "thread-uuid",
  "repo_url": "https://gitee.com/msb-goldbin/efg-test.git",
```

```
    "prompt": "用户输入",
    "task_kind": "coding",
    "created_at": "2026-08-16T12:00:00+08:00"
  }
```

3.3 分布式锁

必须有两类锁。



会话锁

同一个 `thread_id` 同一时间只允许一轮任务运行。

Redis key：

```
lock:thread:{thread_id}
```

作用：

- 防止用户连续点击发送导致同一会话两轮任务并发。
- 防止第二轮任务计划插到第一轮输出中。
- 防止 checkpoint 同一 thread 被同时写入。

仓库写锁

同一个 Gitee 仓库同一时间只允许一个写任务。

Redis key：

```
代码块
1   lock:repo-write:{owner}/{repo}
```

需要加写锁的任务：

任务类型	是否需要仓库写锁
coding	需要
sync / pull / push	需要
创建 PR	需要
planning	通常不需要
qa	不需要
analysis	不需要
review	通常不需要，除非要发布 PR 评论

3.4 实时事件总线

建议把当前 `event_sink(event, data)` 改成写 Redis Stream。

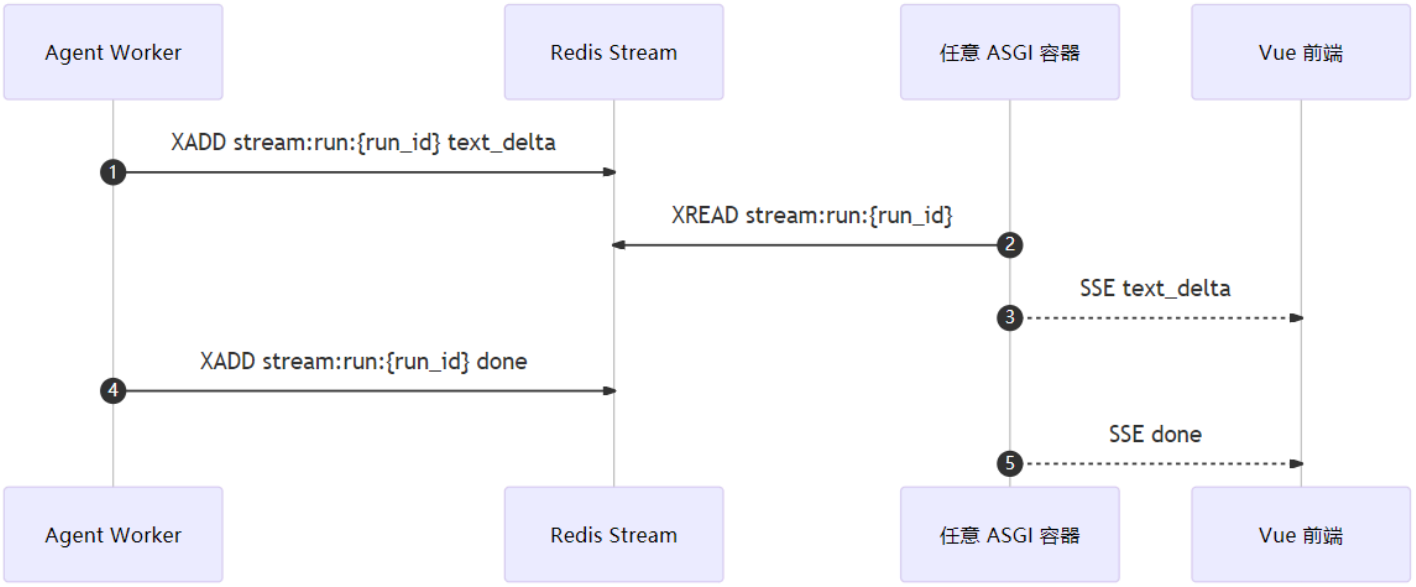
Redis Stream key：

```
代码块
1   stream:run:{run_id}
```

事件示例：

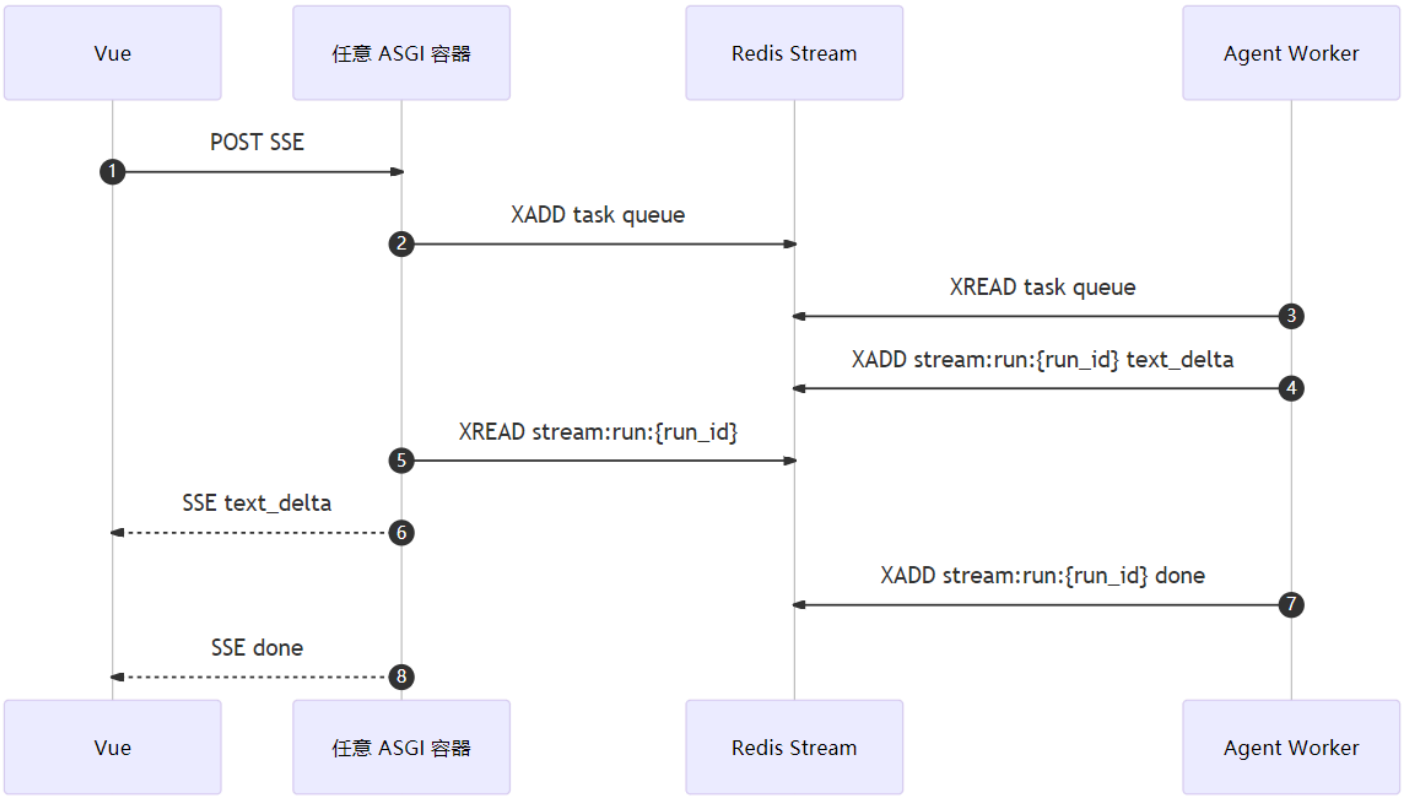
```
代码块
    "event": "text_delta",
    "thread_id": "thread-uuid",
    "run_id": "run-uuid",
    "message_id": "thread-live-assistant-run-1",
    "content": "正在读取项目结构...",
    "mode": "replace"
  }
```

SSE 接口不再等待本进程队列，而是读 Redis Stream：



这样前端连接到任意 ASGI 容器，都能读取同一个 run 的事件。

4、Agent Worker设计



4.1 为什么要拆 Worker

如果 ASGI 容器直接运行 Agent，会有几个问题：

问题	说明
Web 请求被长时间占用	Agent 运行可能几分钟甚至更久
容器重启影响任务	Web 容器重启会直接中断 Agent
扩容职责混乱	Web 扩容和 Agent 执行扩容绑在一起
并发控制困难	每个 Web 容器都要维护任务执行状态

拆出 Worker 后：

- Web 容器负责短请求和 SSE。
- Worker 负责长任务。
- Worker 数量可以按模型额度、CPU、内存、工作区 IO 单独扩容。

4.2 Worker 主循环伪代码

代码块

```
def worker_loop():
    while True:
        task = redis_queue.pop()
        if task is None:
```

```
            sleep(1)
            continue

        run_id = task["run_id"]
        thread_id = task["thread_id"]
        repo_url = task["repo_url"]

        with acquire_thread_lock(thread_id):
            with acquire_repo_lock_if_needed(task):
                try:
                    mark_run_running(run_id)
                    run_agent_task(
                        repo_url=repo_url,
                        prompt=task["prompt"],
                        thread_id=thread_id,
                        event_sink=redis_stream_event_sink(run_id),
                    )
                    mark_run_completed(run_id)
                except Exception as exc:
                    mark_run_failed(run_id, exc)
                    publish_error_event(run_id, exc)
                finally:
                    publish_done_event(run_id)
```

第十五章、项目经验模板和相关面试题

介绍：这个 AI Coding 项目不是一个简单的聊天 Agent，而是一个围绕 Gitee 仓库构建的工程化 Agent Harness。它通过多层意图识别控制任务入口，通过非单例 Agent 动态装配 Prompt、SubAgent 和 Skill，通过 checkpoint 与仓库记忆保证上下文连续，通过自定义文件后端和沙箱思路保证安全边界，通过 DeepAgents V3 + SSE 实现过程可观测，最后通过 Worker 集群、Redis 和 PostgreSQL 支持高并发扩展。

1、项目亮点

一、多层意图识别与分层任务决策

项目痛点

AI Coding 项目不是普通聊天系统。用户输入一个需求之后，系统不能只判断“这是写代码还是问答”就结束了。

因为一次真实的开发任务里，Agent 需要连续做很多判断：

- 用户输入到底是问答、方案、代码实施，还是代码审查？

- 这个任务是否应该由主 Agent 自己完成，还是交给专门的子 Agent?
- 用户的复杂需求应该拆成哪些步骤?
- 每一个步骤到底应该读文件、查仓库、写代码、运行测试，还是提交 PR?
- 某一步失败以后，是继续、重试，还是调整方案?

如果只做一层意图识别，Agent 很容易变成“模型自由发挥”。在企业场景里，这会带来执行不可控、工具滥用、任务流程混乱和成本不可控的问题。

所以当前项目更适合描述为：做了多层意图识别和分层任务决策。

解决方案

当前项目可以分为四层意图识别和任务决策。

代码块

```
第一层：用户输入意图分类
第二层：主 Agent 的任务转交判断
第三层：用户任务拆解成任务计划
第四层：任务清单中每一步的意图决策
```

工程化价值

这个设计的好处是，Agent 不再是单轮问答模型，而是一个具备分层调度能力的工程系统。

第一，它让任务入口可控。系统先判断任务类型，避免问答、方案、代码实施、审查混在一起。

第二，它让能力边界更清晰。主 Agent 负责调度，子 Agent 和 skill 负责专门任务，工具负责具体操作。

第三，它让复杂任务可追踪。用户可以在前端看到任务计划，而不是只看到模型最后给一个结果。

第四，它让执行过程更稳定。每一步都可以根据工具反馈继续决策，比如测试失败后继续修复，而不是一次性生成代码后就结束。

第五，它方便后续扩展。比如以后要增加安全扫描、部署检查、数据库迁移审查，只需要新增任务类型、子 Agent 或 skill，而不是推翻整体架构。

面试口述版本

我在这个 AI Coding 项目里做的不是单层意图识别，而是多层意图识别和分层任务决策。

第一层是用户输入意图分类。系统会先判断用户是在要技术方案、代码实施、代码审查、项目问答，还是仓库同步。这样可以避免用户只是问问题时误触发代码修改，也能保证写代码任务必须先走方案确认。

第二层是主 Agent 的任务转交判断。主 Agent 不是什么都自己做，而是更像一个调度者。比如 PR 审查会交给 `code_reviewer` 子 Agent，通用项目分析可以交给 general-purpose 子 Agent，仓库初始化分析可以走 skill，Gitee 操作则走专门工具。

第三层是任务计划拆解。对于复杂开发任务，Agent 会通过 DeepAgents 的 `write_todos` 把用户需求拆成结构化任务清单，比如读取项目、设计方案、修改代码、增加测试、运行验证、提交 PR。这个任务计划会实时展示在前端，也方便追踪执行进度。

第四层是任务清单中每一步的动作级决策。比如执行到“修改代码”这一步时，Agent 还要继续判断读哪些文件、改哪些模块、是否需要补测试、测试失败后如何修复、什么时候可以提交 PR。

这样设计的价值是让 Agent 从一个简单聊天模型，变成了一个分层调度的工程系统。它既能保证任务入口可控，又能让复杂开发过程可追踪、可扩展、可恢复，也更符合企业内部对 AI Coding 工具的落地要求。

二、第一层用户意图识别采用“小规模 LLM + 规则判断”的任务分类方案

项目痛点

AI Coding 项目的第一层意图识别非常关键，因为它决定了用户输入进入哪条执行链路。

如果第一层分类不准，后面的流程都会错。

比如：

- 把普通问答误判成 coding，可能导致 Agent 尝试改代码；
- 把 coding 误判成 qa，用户明明想开发功能，系统却只回答文字；
- 把 review 误判成普通问答，就不会进入 reviewer 子 Agent；
- 把“确认实施”误判成普通聊天，就无法衔接上一轮技术方案。

所以第一层任务分类既要理解自然语言，又要足够稳定，不能完全依赖模型自由发挥。

解决方案

当前项目采用的是“小规模 LLM + 规则判断”的组合方案。

它不是单纯关键词匹配，也不是完全依赖大模型，而是分成两步。

第一步，优先使用轻量模型做语义分类。

模型会根据用户输入，判断当前任务大致属于哪一类：

```
代码块
1  planning：生成技术方案
2  coding：代码实施
3  review：代码审查
4  qa：项目问答
5  pull：同步仓库
6  workspace：查询工作区
```

LLM 对自然语言语义理解更强，可以识别“用户真实想做什么”。

第二步，使用规则判断做兜底和强约束。也就是说，LLM 负责理解语义，规则负责兜底和确定性控制。

```
用户输入
  -> 小规模 LLM 做语义任务分类
  -> 检查分类结果是否合法
  -> 如果结果稳定，使用 LLM 分类结果
  -> 如果模型失败、结果非法或命中特殊规则，走规则兜底
  -> runtime 根据任务类型进入对应流程
```

工程化价值

这个方案的价值在于平衡了“智能”和“可控”。

小规模 LLM 提供语义理解能力，可以覆盖自然语言表达的多样性。规则判断提供确定性和兜底能力，避免模型误判导致错误执行。

对企业级 Agent 项目来说，这种组合方案比单纯 Prompt 或单纯关键词更可靠：

- 分类准确率更高；
- 规则维护成本更低；
- 异常情况下有兜底；
- 简单任务可以快速识别；
- 高风险任务可以做强约束；
- 后续扩展任务类型更方便。

面试口述版本

在第一层用户意图识别上，我没有只做关键词匹配，也没有完全依赖大模型，而是采用了“小规模 LLM + 规则判断”的组合方案。

原因是 AI Coding 场景里的用户输入非常灵活。比如用户可能说“帮我加一个功能”，也可能说“这个模块缺一个查询接口”，这些表达不一定有明确关键词，如果只靠规则，后面会变成大量 if/else，维护成本很高。所以我会优先使用轻量模型做语义分类，判断用户输入属于 planning、coding、review、qa、pull、workspace 等哪类任务。

但是我也没有完全相信模型。因为第一层分类直接决定后面的执行链路，如果模型把普通问答误判成 coding，就可能触发代码修改流程。所以项目里保留了规则判断作为兜底，尤其是像“确认实施”“pull 仓库”“workspace 查询”“PR 审查链接”这类高确定性场景，会用规则做强约束。如果模型返回异常、分类不合法或置信度不足，也会回退到规则判断。

这样设计的好处是，LLM 负责理解自然语言，规则负责保证流程安全和稳定。整体既比纯关键词更智能，也比纯 LLM 更可控，更符合企业里对 Agent 系统稳定性和安全边界的要求。

三、仓库记忆文件，避免 AI Coding 过程中的记忆断层

项目痛点

AI Coding 不是一次性问答，而是围绕同一个代码仓库持续工作的过程。

在真实开发中，用户可能今天让 Agent 实现登录功能，明天让它增加部门模块，后天让它 review 一个 PR。每一轮任务都和同一个仓库的历史状态有关。

如果 Agent 每次启动都从零开始分析仓库，就会出现几个问题：

- 每次都重复读取项目结构、启动方式、测试方式；
- 上一轮已经踩过的坑，下一轮还会重复踩；
- 已经确认过的技术决策，后续任务又重新推翻；
- Agent 不知道之前哪些功能已经完成、哪些 PR 已经创建；
- 长对话被压缩或新会话打开后，项目上下文容易断掉。

这就是 AI Coding 场景里的“记忆断层”问题。

普通聊天历史只能保存某个会话里的上下文，但企业开发往往是围绕仓库长期持续推进的，不能只依赖单个会话。

解决方案

当前项目设计了“仓库级记忆文件”。

它不是简单把所有聊天记录塞进 Prompt，而是为每一个 Gitee 仓库维护一份长期记忆。每个仓库对应一份独立记忆，内容记录这个仓库长期有价值的信息，包括：

```
代码块
1  项目结构
2  启动方式
3  测试命令
4  关键模块
5  已完成能力
6  最近分支
7  最近 PR
8  历史技术决策
9  已知问题
10 注意事项
```

Agent 每次处理某个仓库任务时，会先根据仓库地址定位对应的记忆文件，然后把这份记忆注入到 Agent 上下文中。

这样 Agent 在新一轮任务开始时，不需要完全重新认识项目，而是可以直接知道：

- 这个仓库是什么类型的项目；

- 后端入口在哪里；
- 测试应该怎么跑；
- 上一次做到了哪个功能；
- 哪些设计已经确认过；
- 哪些问题之前已经遇到过。

同时，任务成功结束后，系统会把本轮有价值的信息整理回仓库记忆中。例如：

- 新增了哪些功能；
- 改了哪些关键文件；
- 测试结果是什么；
- 创建了哪个 PR；
- 有没有新的注意事项。

这样仓库记忆会随着项目迭代持续更新。

工程化价值

仓库记忆文件的价值在于，它让 AI Coding 从“单轮执行”变成“持续协作”。

第一，减少重复分析。

Agent 不需要每次都重新扫项目结构、测试方式和关键模块，可以更快进入任务。

第二，避免技术决策断层。

比如之前已经决定使用 SQLite、某个接口风格、某个分支策略，后续任务可以延续这些决策，而不是每次重新设计。

第三，提升多轮开发稳定性。

哪怕新开一个会话，只要仓库相同，Agent 仍然能读取仓库记忆，继续基于已有上下文工作。

第四，降低 token 成本。

仓库记忆是压缩后的关键信息，不是完整聊天记录。相比每次塞入大量历史消息，更节省上下文。

第五，更适合企业团队场景。

企业内部一个仓库可能被多个任务、多个会话、甚至多个开发人员反复操作。仓库级记忆可以作为这个项目的长期知识资产。

面试口述版本

在这个 AI Coding 项目里，我设计了仓库级记忆文件，主要是为了解决 AI Coding 过程中的记忆断层问题。

因为 AI Coding 不是普通聊天。它通常是围绕同一个代码仓库持续工作的，比如今天实现登录，明天增加部门管理，后天做代码审查。如果每次 Agent 都从零开始分析项目，就会重复读取目录、重复判断启动方式、重复踩之前已经踩过的问题，甚至推翻上一轮已经确认的技术决策。

所以我没有只依赖会话历史，而是为每个 Gitee 仓库维护一份独立的仓库记忆文件。这个记忆文件会记录项目结构、启动命令、测试方式、关键模块、已完成功能、最近分支、最近 PR、历史技术决策和已知问题。

每次 Agent 处理某个仓库任务时，会根据仓库地址定位对应的记忆文件，并把它注入到上下文里。任务成功结束后，系统会把本轮有价值的信息再整理回记忆文件，比如新增了什么功能、修改了哪些文件、测试是否通过、创建了哪个 PR。

我把 checkpoint 和仓库记忆做了区分。checkpoint 负责会话历史恢复，仓库记忆负责仓库长期知识沉淀。这样即使用户换了一个会话，只要还是同一个仓库，Agent 仍然能延续之前的项目上下文。

这个设计的工程价值是，它让 Agent 从一次性执行工具变成了长期协作助手。它能减少重复分析、降低 token 成本、避免技术决策断层，也更符合企业内部围绕一个仓库持续迭代的真实开发场景。

四、写代码和代码审查拆分成两个不同的 SubAgent

在 AI Coding 项目里，“写代码”和“审查代码”看起来都和代码有关，但它们本质上是两种完全不同的任务。

如果让同一个 Agent 同时负责写代码和审查代码，就会出现几个问题。

第一，角色冲突。

写代码的 Agent 倾向于推动任务完成，而审查 Agent 应该保持质疑和风险意识。一个角色既当开发者又当 reviewer，容易对自己生成的代码过于乐观。

第二，Prompt 复杂度过高。

写代码需要关注实现路径、文件修改、测试和提交；审查代码需要关注 diff、风险等级、审查规则、证据位置。如果都塞进一个主 Agent，系统提示词会越来越长，行为也会变得不稳定。

第三，工具权限不同。

写代码需要写文件、运行测试、提交、push、创建 PR；代码审查更多是读取 PR 上下文、读取 diff、校验 findings、生成报告，默认不应该修改代码。两者权限边界不同。

第四，输出形态不同。

写代码最终输出的是实现总结、测试结果、分支和 PR；代码审查最终输出的是结构化 findings 和中文审查报告。混在一个 Agent 里，前端展示和后端数据结构都会复杂。

所以当前项目把写代码和代码审查拆成不同的 SubAgent / 能力单元，而不是让一个 Agent 包办所有任务。

解决方案

当前项目采用“主 Agent 调度 + 专业 SubAgent 执行”的方式。

工程化价值

这个拆分有几个非常明显的工程化价值。

第一，职责单一。

coding 只关注把需求正确落地，reviewer 只关注发现问题和风险。职责越清晰，Prompt 越容易维护，Agent 行为越稳定。

第二，权限更安全。

写代码和审查代码的工具权限可以分开。reviewer 默认不需要修改文件，避免审查任务误触发代码修改。

第三，输出更专业。

reviewer 可以按照代码审查规则输出结构化 findings，包括文件、行号、风险等级、问题原因和修改建议，而不是泛泛地说“代码看起来不错”。

第四，便于形成闭环。

reviewer 发现问题后，主 Agent 可以读取审查报告和 findings，再生成修复方案，用户确认后进入 coding 链路修改代码。这样就形成：

```
代码块
1   写代码 -> 创建 PR -> reviewer 审查 -> 生成 findings -> 主 Agent 生成修复方案 -> 再次 coding
```

这更接近企业真实研发流程。

第五，便于后续扩展。

未来如果要增加：

- 安全审查 Agent；
- 性能审查 Agent；
- 测试生成 Agent；
- 文档生成 Agent；
- 部署检查 Agent；

都可以沿用这个“主 Agent 调度 + 专业 SubAgent”的架构。

面试口述版本

在这个项目里，我把写代码和代码审查拆成了两个不同的 SubAgent 或能力链路，而不是让一个 Agent 同时完成所有事情。

原因是写代码和审查代码虽然都和代码有关，但角色完全不同。写代码更像开发者，目标是理解需求、修改文件、补测试、提交 PR；代码审查更像 reviewer，目标是基于 diff 找 bug、安全风险、兼容性问题和测试缺口。如果让同一个 Agent 既写代码又审查代码，容易出现角色冲突，也会让 Prompt 和工具权限变得很混乱。

所以我采用了主 Agent 调度的方式。主 Agent 先判断任务类型，如果是功能实现，就进入 coding 链路，负责生成方案、等待确认、修改代码、运行测试、提交 PR。如果是代码审查，就委派给

`code_reviewer` 子 Agent，读取 PR 上下文和 diff，加载审查规则，逐文件分析变更，最后输出结构化 findings 和中文审查报告。

这个拆分的价值是职责更清晰、权限更安全、输出更专业。coding 链路可以拥有写文件和提交代码的能力，而 reviewer 默认以只读审查为主，避免审查任务误修改代码。同时 reviewer 的输出可以结构化保存，后续主 Agent 可以基于这些 findings 再生成修复方案，形成“实现、审查、修复”的闭环。

我觉得这个设计比较符合企业内部研发流程，因为真实团队里开发和 review 本来就是两个角色。把它们拆成不同的 Agent，也让整个 AI Coding 系统更容易扩展和治理。

五、基于自定义Backend的权限控制

项目痛点

AI Coding 项目一定会涉及文件操作，例如读取仓库代码、修改源文件、运行测试、生成审查报告、读取技能文件等。

如果不做权限控制，Agent 拥有过大的文件系统访问能力，就会有明显风险，尤其是 AI Coding Agent 具备工具调用能力，如果文件操作权限只靠 Prompt 约束，比如“不要访问敏感文件”，这是不够的。企业项目里必须在工具层和后端层做强约束。

解决方案

当前项目不是直接让 Agent 操作真实文件系统，而是通过自定义文件后端来统一管理文件访问。

这个自定义文件后端可以理解成 Agent 和本地文件系统之间的一层“安全代理”。

工程化价值

这个设计的工程价值很明显。

第一，提高安全性。

Agent 不能随意访问整个磁盘，只能在允许的虚拟目录内操作。敏感文件、系统文件、其它项目文件都不会暴露给 Agent。

第二，提高可维护性。

所有路径都统一通过后端管理，后续从 Windows 本地迁移到 Linux Docker 或云端共享存储时，只需要改后端映射，不需要改 Agent 逻辑。

第三，支持多任务权限隔离。

不同任务类型、不同子 Agent 可以配置不同的读写权限。比如代码审查默认只读，写代码任务才允许改源码。

第四，降低误操作风险。

Agent 写文件时必须经过路径校验，可以避免把文件写到错误目录，也可以避免误删或覆盖关键配置。

第五，更符合企业落地要求。

企业内部使用 AI Coding 工具时，安全边界是重点。这个方案说明项目不是简单给模型一个 shell，而是设计了文件访问网关和权限控制层。

面试口述版本

在这个项目里，我没有让 Agent 直接操作真实文件系统，而是做了一层自定义文件后端，用来统一管理文件访问和权限控制。

原因是 AI Coding Agent 会读写代码、运行测试、生成报告，如果直接暴露本机路径，风险很高。比如模型可能访问 `.env`、系统目录，或者把文件写到错误位置。单靠 Prompt 告诉模型“不要访问敏感文件”是不可靠的，所以必须在工具层做强约束。

我的做法是把真实工作区抽象成一组虚拟目录，比如 `/projects/`、`/skills/`、`/policies/`、`/reviews/`、`/memories/`、`/tmp/`。Agent 只能看到这些虚拟路径，后端负责把它们映射到真实的工作区目录。所有读写操作都会经过路径解析和权限校验，确保路径不能逃逸出工作区。

同时，不同任务可以配置不同权限。比如 planning 阶段主要是只读，不能修改源码；coding 阶段才允许写 `/projects/`；reviewer 子 Agent 主要读取 PR diff 和审查规则，并把报告写到 `/reviews/`。这样就把不同任务和不同 Agent 的权限边界隔离开了。

这个设计的价值是，让 Agent 的文件操作从“模型自由调用工具”变成了“受控的文件访问”。它提升了安全性，也方便未来迁移到 Docker 或云端，因为 Agent 使用的是虚拟路径，不依赖具体 Windows 路径。整体上，这更符合企业内部对 AI Coding 工具的安全和可维护性要求。

六、通过中间件矩阵解决 Tool 调用参数预防和错误处理

项目痛点

在 AI Coding 项目里，Tool 调用是 Agent 真正产生行动的地方。

模型本身只是推理和决策，但真正会影响项目代码和外部系统的是工具，比如：

```
代码块
1  读文件
2  写文件
3  执行命令
4  操作 Gitee
5  创建 PR
6  发布评论
7  读取记忆
8  运行测试
```

所以 Tool 调用的稳定性非常关键。

如果没有统一治理，容易出现几个问题：

- 模型传入错误参数，工具直接报错；
- 工具异常直接中断整轮任务；
- 同一个失败工具被反复调用，形成循环；
- 工具返回内容过大，污染上下文；
- 工具输出中夹杂敏感信息；
- ToolMessage 和 AIMessage 顺序异常，导致下一轮模型请求报错；
- 不同工具各自处理异常，逻辑分散，难维护。

如果每个工具里都单独写一套防护逻辑，代码会越来越散。工具越多，重复代码越多，行为也不一致。

所以这个项目没有只在工具函数内部做 try/except，而是设计了一个“中间件矩阵”，在 Agent 运行链路中统一处理 Tool 调用前后的风险。

解决方案

当前项目把 Tool 调用治理拆成多类中间件，每个中间件负责一个横切问题。

可以理解成：工具调用不是直接从模型到工具，而是要经过一组中间件防护层。

代码块

```
模型决策
  -> 参数/上下文中间件
  -> 权限/消息清洗中间件
  -> 工具异常处理中间件
  -> 调用次数保护中间件
  -> 上下文压缩中间件
  -> 工具执行
  -> 工具结果再进入 Agent 循环
```

工程化价值

这个设计的工程价值在于，Tool 调用治理变成了统一机制，而不是散落在每个工具里的重复代码。

第一，降低工具开发复杂度。

新增工具时，不需要每个工具都重复实现调用次数保护、消息清洗、上下文压缩等逻辑，只需要关注自己的业务参数和业务结果。

第二，提升运行稳定性。

即使某个工具失败，中间件也可以把异常转换成 Agent 可理解的错误，让 Agent 有机会继续调整策略，而不是整轮崩掉。

第三，避免无限循环。

运行保护中间件可以发现模型调用或工具调用次数异常，及时终止任务，避免资源浪费。

第四，增强安全边界。

权限控制和路径校验可以阻止危险参数真正执行。即使模型传入越权路径，也会被后端拦截。

第五，方便统一观测和排查。

中间件层可以统一记录工具调用、异常、压缩、清洗等行为，后续排查问题时更容易定位是哪一层出了问题。

面试口述版本

在 Tool 调用这一块，我没有只在每个工具函数里简单写 try/except，而是设计了一套中间件矩阵来统一治理工具调用的风险。

因为 AI Coding 项目的工具很多，包括读写文件、执行测试、操作 Gitee、创建 PR、发布评论等。真正会对外部系统产生影响的是工具，不是模型本身。如果工具调用没有统一治理，很容易出现模型传错参数、工具异常中断、重复调用失败工具、ToolMessage 顺序不合法、工具输出过大污染上下文等问题。

所以我把这些问题拆成多个横切面，用中间件来处理。比如**上下文注入中间件**负责在调用前注入仓库记忆和工作区规则，减少模型传错参数；**消息清洗中间件**负责清理孤立 ToolMessage，避免下一轮模型请求报错；**工具异常处理中间件**负责把底层异常转换成 Agent 可理解的错误；**运行保护中间件**负责限制模型和工具调用次数，防止循环；**上下文压缩中间件**负责控制工具输出长度，避免上下文膨胀。

这样做的好处是，工具本身只需要关注自己的业务逻辑，比如 Gitee API 怎么调用、文件怎么读取，而通用的稳定性、安全性和上下文治理交给中间件矩阵统一处理。

我觉得这个设计比较符合企业级 Agent 系统的落地要求。因为企业项目里工具会越来越多，如果每个工具都单独处理异常和保护逻辑，最后会很难维护。通过中间件矩阵，可以让工具调用具备统一的安全边界、错误恢复能力、运行保护和可观测性。

七、非单例 Agent 实例，支持动态提示词、SubAgent 和 Skill 弹性装配

项目痛点

AI Coding 项目不是一个固定问答机器人，它面对的任务类型非常多。

如果整个系统只有一个单例 Agent，就会遇到一个问题：所有任务都要塞进同一个系统提示词、同一批工具、同一套 skill、同一组 subagent。

这样会导致：

- Prompt 越来越长，维护困难；
- 不同任务规则互相干扰；
- review 的规则影响 coding；
- coding 的写入能力影响 planning；
- 新增 skill 或 subagent 时必须改全局 Agent；

- 很难针对不同任务做弹性装配。

所以当前项目没有采用单例 Agent，而是采用“每轮任务动态创建 Agent 实例”的方式。

解决方案

当前项目把 Agent 设计成“运行时动态装配”的对象。

每一轮任务开始时，runtime 会先判断任务类型，然后 Agent 工厂根据任务上下文动态创建 Agent。这意味着不同任务可以拥有不同的 Agent 形态。也就是说，Agent 不是一个固定实例，而是根据任务类型动态组装出来的执行器。

Agent 可以每轮重新创建，但上下文不会依赖这个 Python 对象本身。项目中主要通过三类机制保证上下文持续性和统一性。

第一，使用 thread_id + checkpoint 保证会话上下文连续

每一轮用户输入都会带上同一个 `thread_id`。

Agent 虽然是新创建的，但运行时会绑定同一个 checkpointer，这样 DeepAgents / LangGraph 在运行时可以根据 `thread_id` 恢复这个会话之前的状态。虽然每一轮可能都是新的 Agent 实例，但它们都通过同一个 `thread_id` 读取同一个 checkpoint，因此会话历史是连续的。这解决的是“会话级上下文连续性”。

第二，使用仓库记忆文件保证仓库级上下文连续

checkpoint 解决的是某个会话内部的上下文，但 AI Coding 还需要跨会话、跨任务记住同一个仓库的信息。

第三，使用统一 Agent 工厂保证上下文注入方式一致

非单例 Agent 不代表每次随意创建。当前项目通过统一的 Agent 工厂创建 Agent。

每次创建 Agent 时，都会按照统一规则注入。

也就是说，虽然 Agent 实例是新的，但创建流程是统一的。

这可以保证：

- planning、coding、review 使用各自合适的 Prompt；
- 同一个仓库总是注入同一份仓库记忆；
- 同一个 thread 总是绑定同一个 checkpoint；
- 权限和 tools 根据 task_kind 动态装配；
- SubAgent 和 Skill 的加载方式一致。

这解决的是“多 Agent 实例之间的配置统一性”。

工程化价值

这个设计的工程价值在于，它同时兼顾了“弹性”和“连续性”。

第一，任务能力可以动态装配。

不同任务可以使用不同 Prompt、不同权限、不同 SubAgent 和 Skill。

第二，会话上下文不会丢。

同一个 `thread_id` 通过 checkpoint 恢复历史消息和图状态。

第三，仓库上下文不会断层。

同一个 `repo_url` 通过仓库记忆文件恢复项目长期知识。

第四，多 Agent 实例不会配置混乱。

所有 Agent 都通过统一工厂创建，保证上下文注入、权限配置和能力加载方式一致。

第五，更适合生产化部署。

未来 Agent Worker 化以后，Worker 从队列里拿到任务，只要有 `thread_id` 和 `repo_url`，就能重新创建 Agent 并恢复上下文，不依赖某个常驻内存对象。

面试口述版本

在这个项目里，我采用的是非单例 Agent 设计。每一轮任务都会根据 task_kind 动态创建 Agent，这样可以按任务装配不同的系统提示词、工具权限、SubAgent 和 Skill。比如 planning 任务只读生成方案，coding 任务允许修改代码和提交 PR，review 任务会加载 code_reviewer 子 Agent。

但是非单例设计有一个关键问题，就是每轮都重新创建 Agent，怎么保证上下文不断层。我的处理方式是，不把上下文保存在 Agent 对象里，而是放到外部持久化层。

具体来说，会话级上下文通过 `thread_id` 和 checkpoint 保证。每一轮 Agent 虽然是新实例，但只要绑定同一个 `thread_id`，LangGraph checkpointer 就能恢复之前的 messages 和图状态。比如上一轮生成方案，下一轮用户说“确认实施”，系统可以从 checkpoint 里找到上一轮方案并继续执行。

仓库级上下文通过仓库记忆文件保证。每个 Gitee 仓库都有自己的记忆文件，记录项目结构、测试方式、历史决策、最近 PR 和已完成功能。即使用户换了新会话，只要还是同一个仓库，Agent 也能读取这份仓库记忆，避免从零开始分析。

另外，所有 Agent 都通过统一的 Agent 工厂创建。每次创建时都会按统一规则注入 thread_id、repo_url、task_kind、checkpoint、仓库记忆、skills、subagents、permissions 和 middleware。这样虽然 Agent 实例不是单例，但上下文注入和能力装配是统一的。

所以这个设计不是简单地每次 new 一个 Agent，而是把 Agent 当成本轮任务的临时执行器，把上下文放到 checkpoint 和仓库记忆里。这样既能保证动态装配的弹性，又能保证多轮任务和多仓库场景下的上下文连续性，更适合企业级 AI Coding 系统。

如果面试官问：

> **你说通过 thread_id 和 checkpoint 恢复上下文，那历史很长怎么办？是不是全部塞给模型？**

你可以回答：

```
代码块
```

```
1  不是。checkpoint 负责恢复 LangGraph thread 的图状态，它保存的是会话状态，不等于每次都
   会把所有历史原样送进模型。真正进入模型上下文之前，还需要做 context engineering，比如消息
   清洗、历史压缩、保留最近关键对话、删除工具噪声，并结合仓库级 repo memory 提供长期项目知
   识。
2  在我的项目里，checkpoint 主要解决会话连续性，repo memory 解决仓库长期记忆。长上下文不会
   完全依赖 checkpoint 原文，而是通过中间件和记忆文件做提炼和注入，避免超出模型上下文，也避
   免历史噪声影响当前任务。
```

一句话总结：

> checkpointer 保证状态能恢复，但不代表所有历史都会无限制进入 LLM；企业级 Agent 项目必须在 checkpointer 之上做上下文裁剪、压缩和长期记忆抽取。

八、基于消息队列的 SSE 流式输出

项目痛点

AI Coding 任务通常不是秒级完成的。所以 AI Coding 项目必须做流式输出，让用户看到 Agent 正在做什么。

但这里还有一个工程问题：Agent 执行逻辑和 HTTP SSE 响应不应该强耦合。如果 Agent 在后台线程中运行，而 FastAPI SSE 负责把内容推给浏览器，中间就需要一个稳定的事件缓冲层。

解决方案

当前项目采用了“消息队列 + SSE”的流式输出方案。

整体思路是：

代码块

```
1  Agent 执行线程
2    -> 把运行事件写入队列
3    -> FastAPI SSE 从队列读取事件
4    -> 推送给 Vue 前端
```

也就是说，Agent 不直接操作 HTTP Response，而是通过 `event_sink` 把事件投递到队列。

加了队列之后：

代码块

```
1  Agent 只管 event_sink(event_type, payload)
2  SSE 只管 queue.get() 后发送
```

两边职责更清楚。

工程化价值

这个设计的工程价值主要体现在稳定性、解耦和可扩展。

第一，用户体验更好。

用户能实时看到 Agent 输出、任务计划和运行状态，不会觉得系统卡死。

第二，后端职责更清晰。

Agent 只负责执行和产生事件，FastAPI 只负责把事件流式推给前端。中间通过队列解耦。

第三，异常处理更稳定。

Agent 执行失败时，可以往队列里写入 `error` 和 `done`，SSE 能正常结束，而不是连接直接挂掉。

第四，方便后续 Worker 化。

本地版本可以用内存队列。未来云端多容器部署时，可以把内存队列替换成 Redis Stream，让 Agent Worker 和 Web 容器解耦。

第五，支持多类型事件。

不仅能流式输出文本，还能流式输出任务计划、状态变化、错误信息和子 Agent 进度。

面试口述版本

在这个项目里，我做了基于消息队列的 SSE 流式输出。因为 AI Coding 任务通常不是秒级完成的，它可能要读仓库、分析结构、生成方案、修改代码、跑测试、创建 PR。如果前端只能等任务结束后一次性拿结果，用户体验会很差，也很难判断 Agent 是卡住了还是正在工作。

所以我的设计是让 Agent 执行线程和 FastAPI SSE 响应解耦。Agent 不直接写 HTTP Response，而是通过 `event_sink` 把运行过程中的事件写入队列，比如用户消息、assistant 文本增量、任务计划、任务状态、错误和完成事件。FastAPI 这边的 SSE generator 持续从队列里读取事件，再推送给 Vue 前端。

这样 Agent 是事件生产者，队列是缓冲层，SSE 是传输层，前端是展示层。职责比较清晰。

当前项目里还结合了 DeepAgents V3 event streaming。DeepAgents 会产生 raw events，`streaming_runtime.py` 会把 assistant token 转成 `text_delta`，把 `write_todos` 转成 `todo_delta`，再通过队列推给前端。所以前端不仅能看到最终答案，还能看到任务计划和 AI 输出的实时变化。

这个设计的工程价值是提高用户体验和系统稳定性。即使 Agent 报错，也可以通过队列发送 error 和 done 事件，让 SSE 正常收尾。后续如果部署到多容器环境，也可以把本地内存队列替换成 Redis Stream，让 Web 容器和 Agent Worker 解耦。所以这个方案不是简单做一个流式输出，而是为后续生产化部署预留了架构扩展空间。

项目场景面试题

1. Agent 项目如何保证数据安全和权限控制？

总的来说，Agent 项目的安全不能只靠 Prompt。Prompt 只能告诉模型“应该怎么做”，但不能真正阻止模型调用工具、访问文件或执行命令。真正的安全边界必须放在沙箱、文件后端权限、工具网关和运行时治理上。

第一，文件访问要经过受控后端，而不是直接暴露宿主机文件系统。
在当前 AI Coding 项目里，Agent 看到的是 `/projects`、`/skills`、`/policies`、`/reviews`、`/memories` 这类虚拟目录，真实路径由自定义文件后端映射。所有路径都会做越界校验，避免模型访问工作区外的系统文件、`.env`、私钥或其它项目目录。也就是说，即使模型生成了危险路径，后端也不会执行。

第二，不同任务和不同 Agent 要有不同权限。

planning 任务主要只读，不能修改代码；coding 任务才允许写项目文件、运行测试和提交 PR；reviewer 子 Agent 主要读取 PR diff 和审查规则，默认不应该改代码。这个权限隔离不能只写在 Prompt 里，而要落实到 backend permission 和工具可用范围上。

第三，如果是更高安全要求的企业场景，我会进一步引入沙箱执行。

可以参考我之前 ERP Harness 项目的做法：Agent 的代码执行、依赖安装、文件生成、技能运行都放在 OpenSandbox 这类隔离环境里。沙箱相当于 Agent 的执行容器，命令和文件操作都发生在隔离环境中，宿主机只负责调度和结果下载。即使模型执行了有风险的命令，也被限制在沙箱内部，不会直接影响宿主机和其它业务系统。

第四，沙箱本身也需要生命周期和健康治理。

企业里不能只创建一个沙箱就不管了。需要有沙箱初始化、目录播种、依赖预装、健康检查、异常重建和熔断机制。比如沙箱不可用时自动重连或重建；连续沙箱错误达到阈值后触发熔断，避免 Agent 无限重试。这能保证安全隔离的同时，也保证系统可用性。

总结一下，我对 Agent 安全的理解是纵深防御：Prompt 负责行为引导，权限后端负责文件边界，沙箱负责执行隔离，中间件负责健康检查和熔断，Secret 和日志脱敏负责外围安全。真正的核心安全边界还是：宿主机不直接暴露给 Agent，所有执行都经过受控 backend 或沙箱。

2. 为什么项目中的 Agent 不是单例？

我没有把 Agent 做成单例，主要是为了支持动态装配和上下文隔离。AI Coding 项目不是固定问答机器人，不同任务需要不同的提示词、工具权限、SubAgent、Skill 和记忆上下文，如果做成全局单例，后期会很难维护，也容易出现上下文污染。

第一，非单例可以支持动态提示词。

比如 planning 任务强调只读分析和输出方案，coding 任务强调按确认方案修改代码、测试、提交 PR，review 任务强调从审查视角分析 diff。每类任务的 Prompt 规则不同，动态创建 Agent 可以按 task_kind 装配对应提示词，而不是把所有规则塞进一个超长系统提示词里。

第二，非单例方便动态加载能力。
不同任务需要的 SubAgent 和 Skill 不一样。代码审查任务需要 `code_reviewer` 子 Agent 和 code-review skill；普通 coding 任务可能更关注代码实施 skill；项目分析任务可以使用通用分析子 Agent。按任务创建 Agent，可以只加载本轮需要的能力，减少干扰，也方便后续扩展新的安全审查、测试生成、部署检查等能力。

第三，非单例更适合权限隔离。

planning、coding、review 的工具权限不一样。planning 主要只读，coding 允许写项目和运行测试，review 默认以读取 diff 和生成审查报告为主。每轮动态创建 Agent，可以按任务注入不同 backend、permissions 和 tools，权限边界更清楚。

第四，非单例不会导致上下文丢失，因为上下文不绑定在 Agent 对象上。
项目里会话级上下文由 `thread_id + checkpoint` 保证，仓库级长期上下文由 repo memory 保证。Agent 只是本轮任务的执行器，每次创建时通过统一 Agent 工厂注入 thread_id、repo_url、task_kind、checkpoint、仓库记忆、middleware、skills 和 subagents。这样即使 Agent 实例是新的，上下文仍然是连续和统一的。

总结来说，Agent 非单例不是简单为了“每轮 new 一个对象”，而是为了让系统具备运行时弹性装配能力。它可以让不同任务拥有不同 Prompt、不同工具权限、不同 SubAgent 和 Skill，同时通过 checkpoint 和仓库记忆保证上下文连续。这个设计更适合多任务、多仓库、多会话和未来 Worker 化部署的企业级 AI Coding 场景。

3. 你的Agent项目为什么要使用 SubAgent架构？有什么特色？

我使用 SubAgent 架构，是因为 AI Coding 项目里的任务已经不适合全部交给一个主 Agent 完成。主 Agent 更适合作为调度者，负责理解用户目标、选择流程和汇总结果；而代码审查、项目分析这类专业任务，交给专门的 SubAgent 会更稳定、更容易扩展。

第一，SubAgent 可以解决角色职责混乱的问题。

写代码和代码审查虽然都和代码有关，但思维方式完全不同。coding 更偏实现，目标是修改代码、补测试、提交 PR；review 更偏风险识别，目标是基于 diff 找 bug、安全问题、兼容性问题和测试缺口。如果让一个 Agent 同时负责开发和审查，容易角色冲突，也会让 Prompt 越来越复杂。

第二，SubAgent 可以实现专业能力隔离。
当前项目里有 `code_reviewer` 子 Agent，专门负责读取 PR 上下文、分析 diff、加载审查规则、生成结构化 findings 和中文审查报告。它不是泛泛地回答“代码看起来不错”，而是按照代码审查流程输出问题位置、风险等级、原因和建议。主 Agent 只负责调度它、接收结果并汇总。

第三，SubAgent 和权限、Skill 可以一起动态装配。

不同 SubAgent 可以绑定不同提示词、工具和权限。比如 reviewer 默认以只读为主，只需要读取 PR diff、审查规则和写 review 报告，不应该拥有随意修改代码的权限。后续如果增加安全审查、测试生成、部署检查等能力，也可以扩展成独立 SubAgent，而不是让主 Agent 变成一个越来越臃肿的万能 Agent。

第四，SubAgent 可以形成工程闭环。

比如 coding Agent 完成功能并创建 PR 后，reviewer 子 Agent 可以审查 PR，输出 findings；主 Agent 再基于 findings 生成修复方案，用户确认后重新进入 coding 流程。这样就形成“实现、审查、修复、再验证”的闭环，更接近真实研发流程。

总结来说，我的 SubAgent 架构不是为了炫技，而是为了把复杂 Agent 系统拆成可治理的专业角色。主 Agent 负责调度，SubAgent 负责专业任务，Skill 提供可插拔能力，工具负责具体执行。这样系统职责更清晰，权限更安全，扩展也更方便，比较符合企业级 AI Coding 平台的架构思路。

4. Agent 项目中工具失败怎么办？或者出现循环调用怎么办？

总的来说，工具失败和循环调用是 Agent 工程化里必须重点处理的问题。我的原则是：工具失败不能直接让系统失控，循环调用必须有硬性保护。也就是说，工具层要可恢复，运行层要可熔断。

第一，工具失败要转成 Agent 可理解的业务错误。

比如 Gitee 认证失败、路径越界、PR 已存在、测试命令失败、文件不存在，这些不能简单把 Python traceback 抛给模型或前端。工具内部会把异常转换成明确的错误信息，让 Agent 能判断下一步是重试、换路径、复用已有 PR，还是停止任务。

第二，通用异常治理放到中间件矩阵里。

不是每个工具都重复写一套 try/except。项目里通过中间件处理工具异常、消息清洗、上下文压缩和运行保护。比如清理不合法 ToolMessage，避免下一轮模型请求因为消息顺序错误失败；压缩过大的工具输出，避免污染上下文；统一记录异常，方便排查。

第三，循环调用要有硬限制。

Agent 有时会因为工具失败或上下文不清晰，反复调用同一个工具。项目里设置了模型调用次数和工具调用次数保护，一旦超过阈值，就会中断本轮任务，返回明确错误，而不是无限消耗 token 和外部 API。

第四，高并发和生产环境下还要加队列与锁。

如果多个 Agent 同时操作同一个仓库，工具失败可能不是参数问题，而是并发冲突。所以生产方案里会用 Redis 分布式锁控制同一仓库写任务串行，用队列控制 Agent Worker 并发，避免多个任务同时 push、commit 或修改同一工作区。

总结来说，我不会把工具失败当成普通异常处理，而是把它作为 Agent 运行时治理的一部分。工具负责返回可理解错误，中间件负责统一清洗和保护，运行层负责调用次数熔断，高并发场景下再用队列和分布式锁控制并发。这样才能避免 Agent 从“会调用工具”变成“乱调用工具”。

5. 为什么要设计 Agent 的长期记忆，当前项目的长期记忆有什么特点？

我设计长期记忆，是因为 AI Coding 不是一次性问答，而是围绕同一个仓库持续迭代的开发过程。只靠当前对话上下文，很容易出现记忆断层，比如重复分析项目结构、忘记历史技术决策、不了解之前已经完成的功能。

第一，长期记忆解决的是仓库级上下文连续性。

checkpoint 解决的是某个会话的历史恢复，但一个仓库可能跨多个会话、多个任务持续开发。所以当前项目为每个 Gitee 仓库维护独立的仓库记忆文件，记录项目结构、启动方式、测试命令、关键模块、已完成功能、最近分支、最近 PR、历史决策和已知问题。

第二，长期记忆不是简单保存聊天记录。

聊天记录里有大量过程信息、工具输出和临时推理，不适合长期注入模型。当前项目会把任务完成后的关键信息提炼成仓库知识，比如本轮改了哪些核心文件、测试结果是什么、采用了什么设计、创建了哪个 PR。这样记忆更短、更稳定，也更适合后续任务复用。

第三，长期记忆按仓库维度隔离。

记忆文件路径和 Gitee 仓库绑定，比如按 owner/repo 组织。这样不同仓库之间不会互相污染。同一个仓库即使换了新会话，也能读取同一份记忆，避免 Agent 每次从零开始理解项目。

第四，长期记忆和 checkpoint 分工明确。

checkpoint 保存会话状态，保证同一个 thread 可以继续；仓库记忆保存项目长期知识，保证同一个 repo 可以持续协作。两者结合起来，既能保证短期对话连续，也能保证长期项目上下文不丢。

总结来说，当前项目的长期记忆不是普通聊天记忆，而是仓库级工程记忆。它把项目结构、技术决策、测试方式和历史成果沉淀下来，让 Agent 在多轮、多会话、多任务中持续理解同一个代码仓库。这能减少重复分析、降低 token 成本，也让 AI Coding 更接近企业内部长期协作工具。

6. Agent 项目有多层用户意图识别吗？怎么做的？

必须有，而且我认为 AI Coding 项目不能只做一层意图分类。用户输入只是起点，真正的 Agent 执行过程还包括任务转交、任务拆解和每一步动作决策，所以我把它设计成多层意图识别和分层任务决策。

第一层是用户输入任务分类。

系统会先判断用户输入属于 planning、coding、review、qa、pull、workspace 等哪类任务。这里采用小规模 LLM 做语义分类，再用规则兜底。LLM 负责理解自然语言，规则负责处理高确定性场景，比如“确认实施”“pull 仓库”“PR 审查链接”等。

第二层是主 Agent 的任务转交判断。
进入 Agent 后，主 Agent 不一定自己完成所有事情。比如代码审查会委派给 `code_reviewer` 子 Agent；通用项目分析可以交给 general-purpose 子 Agent；某些固定流程可以通过 skill 执行。主 Agent 更像调度者，而不是所有任务都自己做。

第三层是任务计划拆解。
对于复杂开发任务，Agent 会通过 DeepAgents 的 `write_todos` 把用户需求拆成任务清单，比如读取项目、设计方案、修改代码、补测试、运行验证、提交 PR。这个任务计划会流式展示在前端，也让任务过程可追踪。

第四层是任务清单中每一步的动作级决策。

比如执行到“修改代码”这一步时，Agent 还要继续判断读哪些文件、改哪些模块、是否需要补测试、测试失败后如何修复、是否满足提交 PR 条件。这一层是基于工具反馈不断调整的。

总结来说，我的意图识别不是只在入口做一次分类，而是贯穿整个 Agent Loop。入口分类决定大流程，主 Agent 判断是否委派，`write_todos` 拆解任务计划，每一步再根据工具反馈做动作级决策。这样系统更像一个可调度、可追踪的工程 Agent，而不是简单的聊天机器人。

7. 怎么保证 SSE 的流式输出稳定性?

SSE 稳定性不能只靠前端 EventSource 或后端简单 yield 文本。我的做法是把它拆成三层：Agent 事件生产、队列缓冲、SSE 传输和前端消费。每一层都做边界隔离，避免 Agent 执行过程直接绑死 HTTP 响应。

第一，Agent 不直接写 HTTP Response。
项目里 DeepAgents 通过 `stream_events(version="v3")` 产生 raw events，后端的 `streaming_runtime.py` 负责把 assistant token、`write_todos`、SubAgent 状态转换成业务事件，比如 `text_delta`、`todo_delta`、`message_start`。Agent 只通过 `event_sink` 发送事件，不直接操作 SSE 连接。

第二，中间用消息队列做缓冲。

FastAPI 层维护队列，Agent 执行线程把事件写入队列，SSE generator 从队列读取并推给前端。这样 Agent 执行和 HTTP 流式响应解耦。即使 Agent 输出很快，队列也能缓冲；即使前端断开，也可以统一收尾。

第三，事件协议要稳定。
前端不直接消费 DeepAgents 原始事件，而是消费项目自己的事件类型，比如 `user_message`、`message_start`、`text_delta`、`todo_delta`、`thread_status`、`error`、`done`。这样 DeepAgents 底层协议变化时，只需要改后端适配层，不影响前端。

第四，要处理异常和结束信号。
Agent 成功时必须发送 `done`，失败时发送 `error` 后再发送结束事件，避免前端一直等待。后端还要捕获运行保护异常、工具异常、模型异常，把它们转换成可展示的错误消息。

第五，生产部署要考虑 Nginx 和多容器。

如果通过 Nginx 代理 SSE，需要关闭 buffering，设置合理的 read timeout，避免代理层把流式响应缓存住。多容器场景下，建议把本地队列升级成 Redis Stream，Agent Worker 负责生产事件，任意 ASGI 容器都可以读取事件并推给前端。

总结来说，我保证 SSE 稳定性的核心不是简单“后端 yield token”，而是把 DeepAgents 原始事件转换成稳定业务事件，通过队列解耦 Agent 和 HTTP，再用明确的 error/done 协议保证前端收尾。未来高并发部署时，再用 Redis Stream 替代内存队列，实现 Web 容器和 Agent Worker 的彻底解耦。

8. SSE 和 WebSocket有什么区别，为什么你的项目没有采用WebSocket

SSE 和 WebSocket 都能做实时通信，但它们适合的场景不一样。SSE 更适合服务端持续向浏览器推送文本事件，WebSocket 更适合高频双向通信。我的 AI Coding 项目主要是“用户提交任务，

服务端持续推送 Agent 输出”，所以 SSE 更匹配。

第一，从通信模型看，SSE 是单向服务端推送。
浏览器通过 HTTP 建立连接后，后端持续发送事件，比如 `text_delta`、`todo_delta`、`done`。用户下一次输入再发新的 HTTP POST 请求即可。这个模式和 AI Coding 非常贴合，因为 Agent 执行时主要是后端不断把状态推给前端。

第二，WebSocket 是全双工连接。

它适合聊天协作、多人在线编辑、实时控制、频繁双向消息，比如前端要不断发送控制指令、后端也不断回消息。如果当前项目要做多人协作、实时打断、暂停恢复、远程终端交互，那 WebSocket 会更合适。

第三，SSE 的工程复杂度更低。

SSE 基于 HTTP，容易和 FastAPI、Nginx、浏览器 fetch 流结合，也更容易做超时、鉴权、日志和错误处理。当前项目已经把用户提交用 POST，请求返回用 SSE，前端只需要消费标准事件流，不需要维护长生命周期双向连接状态。

第四，当前项目没有强双向实时需求。

AI Coding 的交互模式是：用户提交一轮任务，Agent 流式输出；用户如果要继续，就再提交下一轮。停止任务可以用 AbortController 或 cancel 接口实现，不需要 WebSocket 常驻双向通道。所以 WebSocket 在当前阶段会增加复杂度，但收益不明显。

总结来说，我没有采用 WebSocket，是因为当前项目的核心需求是服务端单向流式输出 Agent 过程，而不是高频双向协作控制。SSE 更简单、更稳定、更贴合 HTTP 架构，也更容易和 FastAPI、Nginx、Redis Stream 组合。等未来出现多人协作、实时终端、复杂控制通道时，再引入 WebSocket 会更合理。

9. 你的项目有做高并发处理吗？可以支持多少人的并发，怎么做的？

Agent项目做了高并发处理，在三个AgentWorker节点，两个ASGI节点的情况下，最大并发数是170左右（planning / qa 轻任务和coding / review 重任务）。同时执行 coding/review 的用户有 20 到 30 个

我的高并发方案是把 Web 请求并发和 Agent 执行并发拆开治理。Web 层用多 FastAPI/Uvicorn 容器横向扩展，Agent 执行层用独立 Agent Worker 集群处理长任务，状态统一放在 PostgreSQL，队列、事件流、分布式锁和限流放在 Redis。

第一，整体架构是 Web 层无状态、Agent 层队列化。

用户请求先进 Nginx，再分发到多个 FastAPI 容器。FastAPI 不直接跑长时间 Agent，而是创建 run 记录，把任务写入 Redis Queue，然后通过 SSE 从 Redis Stream 读取事件推给前端。Agent Worker 从队列消费任务，执行 DeepAgent、调用模型、运行测试、提交 PR，并把过程事件写回 Redis Stream。

第二，通过 Redis 锁保证并发安全。

即使 Worker 有 6 个重任务并发，也不能让多个 Agent 同时修改同一个仓库。所以要加两类锁：`thread_lock` 保证同一会话串行，`repo_write_lock` 保证同一仓库写任务串行。不同仓库可以并发，同一仓库的 coding/push/PR 必须排队。

如果模型额度、CPU 和测试资源允许，Worker 从 3 个扩到 6 个，但我会同时设置全局模型调用限流、单用户限流、单仓库写锁和任务超时，避免扩容后把外部模型 API 或 Gitee API 打爆。我的设计重点是 Web 层横向扩展、Agent 层队列化、Redis Stream 做事件流、Redis Lock 做并发保护、PostgreSQL 做统一状态。

Vllm + sglang的推理优化和代码解析-----> 新项目（13个RAG）------> 第14个项目： Harness架构的Agent中台

DSH： Harness架构的Agent平台。 TS， 插件。====> FDE。